USC UNIVERSIDADE DE SANTIAGO DE COMPOSTELA
FACULTADE DE MATEMÁTICAS

Traballo Fin de Grao

La Transformada Rápida de Fourier

Pablo Díaz Viñambres

Julio, 2025

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

GRAO DE MATEMÁTICAS

Traballo Fin de Grao

La Transformada Rápida de Fourier

Pablo Díaz Viñambres

Julio, 2025

UNIVERSIDADE DE SANTIAGO DE COMPOSTELA

Trabajo propuesto

Área de Coñecemento: Análise Matemática
Título: A transformada rápida de Fourier
Breve descrición do contido
Este traballo centrarase no estudo da transformada rápida de Fourier. En primeiro lugar, estudarase a transformada de Fourier, tanto na súa versión continua coma discreta, analizando as súas propiedades e as da súa transformada inversa. A continuación, incluirase o algoritmo FFT e algunha aplicación do mesmo.

Índice

Resumen vi

Introducción ix

1. Preliminares 1

1.1. Espacios L^p 1

1.2. Series de Fourier 4

1.3. Convoluciones 5

2. La transformada de Fourier 9

2.1. Aproximando funciones no periódicas 9

2.2. La transformada de Fourier en L^1 10

2.3. Extensión al espacio L^2 15

2.4. Generalización a varias dimensiones 17

3. Transformando señales discretas 19

3.1. Procesamiento de señales 19

3.2. La transformada discreta de Fourier (DFT) 20

3.3. DFTs bidimensionales 26

4. La transformada rápida de Fourier 31

4.1. FFTs en base 2 32

4.1.1. El algoritmo de Cooley-Tukey 32
4.1.2. Decimación en tiempo (DIT) y en frecuencia (DIF) 34
4.1.3. Análisis de complejidad 36
4.1.4. Diagramas de mariposa y computación *in-place* 37
4.2. Otras FFTs 40
4.2.1. Cooley-Tukey para enteros compuestos 40
4.2.2. FFTs en otras bases y para números primos 41
4.2.3. Otros detalles y librerías para el cálculo de FFTs 42

5. Aplicaciones **45**
5.1. Filtros digitales de convolución 45
5.2. Algoritmos de compresión: formato JPEG 49

I. Código fuente **59**
I.1. Implementación del filtro digital de audio 59
I.2. Análisis de la compresión JPEG 62

Bibliografía **65**

Resumen

Desde su redescubrimiento en los años 50, la clase de algoritmos conocida como transformadas rápidas de Fourier (FFTs) ha sido fundamental en numerosos campos de las matemáticas, la ciencia y la ingeniería. No es de sorprender que el algoritmo de Cooley-Tukey (conocido comúnmente como *la* FFT) sea ampliamente reconocido como uno de los algoritmos más importantes del siglo XX. En este trabajo, buscamos ofrecer un enfoque estructurado y bien fundamentado para el desarrollo de FFTs. Comenzamos con los fundamentos matemáticos de los espacios L^p y la transformada continua de Fourier, que proporcionan una nueva forma de ver funciones a través de su espectro de frecuencias. Más adelante, introducimos la transformada discreta de Fourier (DFT) como una herramienta numérica que permite aplicar los métodos de Fourier. El cálculo de la DFT para entradas de gran tamaño solo es viable gracias a la FFT. Finalmente, presentamos una breve introducción a dos de los grandes ámbitos de aplicación: el procesamiento de señales digitales y la compresión de datos. En particular, revisamos los filtros de audio digital y examinamos el papel de las FFTs en la compresión de imágenes JPEG.

Abstract

Since their rediscovery in the fifties, the class of algorithms known as Fast Fourier Transforms (FFTs) have been fundamental in numerous fields within mathematics, science and engineering. It comes as no surprise that the Cooley-Tukey algorithm (commonly known as *the* FFT) is widely recognized as one of the most important algorithms of the 20th century. On this project, we aim to provide an structured and well-grounded approach to the development of FFTs. We begin with the mathematical foundations of L^p spaces and the continuous Fourier Transform, that provide a new way to look into functions via their spectrum of frequencies. Later, we introduce the Discrete Fourier Transform (DFT) as a numerical tool to enable Fourier Methods. Computing the DFT for large input sizes is only doable because of FFT algorithms. Finally, we present a brief overview of two major application domains: digital signal processing and data compression. In particular, we review digital audio filters and examine the role of FFTs in JPEG image compression.

Introducción

El objetivo fundamental de este trabajo es dar una visión comprensiva y autocontenida de las Transformadas Rápidas de Fourier (FFTs). Para ello, vemos necesario ir capa por capa, comenzando por un repaso de las series de Fourier y el análisis funcional, centrándonos en las propiedades de los espacios L^p. A continuación, describiremos la teoría detrás de la Transformada de Fourier (FT). Esta transformada aporta una nueva forma de ver las funciones de $L^1(\mathbb{R})$ y $L^2(\mathbb{R})$, a través de una variable compleja ω que representa la magnitud de cada frecuencia, con fase y amplitud, que compone la función original. Hablaremos de dominio temporal, en el que normalmente se define una función real $f(x)$ y dominio de frecuencias o espectro, $\hat{f}(\omega)$, al cual podemos llegar mediante la FT.

Más adelante, introduciremos la Transformada Discreta de Fourier (DFT) que, de cierta forma, adapta la FT al mundo real, permitiendo su cálculo numérico. Las DFTs actúan sobre secuencias finitas de mediciones reales o complejas, que a menudo llamaremos señales. En esta transformada se basan gran parte de las aplicaciones del análisis de Fourier y es gracias a los algoritmos FFT que podemos computarlas eficientemente, incluso para grandes conjuntos de datos de entrada. Además, mediante al importante Teorema de Convolución, obtendremos un método rápido para calcular convoluciones discretas, el cual es de inmensa utilidad en la práctica. Finalmente, nos adentraremos en dos de las principales ramas de aplicación: el procesamiento de señales y la compresión de información. En la Tabla 1 resumimos el contenido de este trabajo.

Tabla 1: Resumen de la estructura del trabajo.

Capítulo	Objeto de estudio	Dominio
1	Series de Fourier y convoluciones	Espacios L^p y de Hilbert
2	Transformada de Fourier (FT)	Funciones de $L^1(\mathbb{C})$ y $L^2(\mathbb{C})$
3	Transformada Discreta de Fourier (DFT)	Vectores de $\mathbb{C}^N$
4	Transformada Rápida de Fourier (FFT)	Vectores de $\mathbb{C}^N$
5	Aplicaciones de la DFT/FFT y variantes	Señales representadas en $\mathbb{R}^N$

Aunque el desarrollo del primer algoritmo FFT se suele atribuir a la publicación, aparecida en 1964, "An algorithm for the machine calculation of complex Fourier series." de James W. Cooley y John W. Tukey ([5]), existe evidencia anterior de uso de algoritmos similares. Alrededor del año 1805, Gauss ya aplicó un algoritmo prácticamente equivalente a casos muy pequeños, con el objetivo de calcular la trayectoria del asteroide Pallas mediante interpolación trigonométrica ([11]). A comienzos del siglo XX se utilizaron algoritmos de descomposición trigonométrica similares, con el objetivo de realizar cálculos en problemas como la dispersión de rayos X ([4]) o cálculos estadísticos. Sin embargo, no fue hasta la publicación de Cooley y Tukey, en el comienzo de la era de la computación, que se llegó a apreciar la importancia de las FFTs. La interpretación dada por estos dos científicos fue la más general hasta la fecha, pudiéndose aplicar sobre cualquier N compuesto, en vez de tan solo para el caso $N = 2^k$.

Posteriormente, muchas otras variantes y generalizaciones del algoritmo de Cooley-Tukey fueron publicadas, incluyendo versiones para N primo, variaciones para el cálculo paralelo, para la obtención de un conjunto selecto de frecuencias y para datos estrictamente reales. Por otro lado, se han desarrollado implementaciones altamente optimizadas y especializadas para obtener el mayor rendimiento sobre hardware moderno, adaptando su cálculo a todo tipo de dispositivos.

Destacamos que el estudio de los algoritmos FFTs no solo permite dominar una técnica clave en procesamiento de señales, sino también desarrollar una comprensión más profunda de cómo el análisis, el álgebra y la informática convergen en el desarrollo de herramientas concretas. Hoy en día, la ubicuidad de las DFTs y sus variantes reales (DCTs y DSTs), equipadas con algoritmos rápidos, es difícil de ignorar. De esta forma, las FFTs se han convertido en una herramienta fundamental en un vasto conjunto de campos, desde las ciencias experimentales hasta la teoría de la información, y desde la ingeniería eléctrica hasta la interpretación de pruebas médicas.

Aunque no las trataremos en este trabajo, también existen generalizaciones de la transformada de Fourier permitiendo observar cómo varían las frecuencias a lo largo del tiempo, con métodos como las Transformadas Modificadas de Cosenos (MDCTs) y las Transformadas de Fourier de Tiempo Corto (STFTs), que permiten la contrucción de *espectrogramas*. Estas variantes son esenciales en tareas como la compresión, el análisis de audio o el procesamiento del habla humana. En su implementación, los algoritmos FFT juegan un papel fundamental.

Capítulo 1

Preliminares

1.1. Espacios L^p

En esta primera sección, buscaremos generalizar el concepto de espacios vectoriales de dimensión finita sobre el cuerpo de los complejos, $\mathbb{C}$, a una visión continua en la que, en lugar de vectores, tendremos funciones, sobre las que impondremos ciertas condiciones.

Definición 1.1. Sean $z \in \mathbb{C}^n$ y $p \in \mathbb{R}$, $p \geq 1$. La *p-norma* de z viene dada por

$$\|z\|_p = \left(\sum_{i=1}^{n} |z_i|^p \right)^{1/p}.$$

La *p-norma* satisface las propiedades de no negatividad ($\|z\|_p \geq 0$), definición positiva ($\|z\|_p = 0 \Leftrightarrow z = 0$), homogeneidad con los escalares ($\lambda \in \mathbb{C} \Rightarrow \|\lambda z\|_p = |\lambda| \, \|z\|_p$) y desigualdad triangular ($\|x + y\|_p \leq \|x\|_p + \|y\|_p$). Llamamos al par $(\mathbb{C}^n, \|\cdot\|_p)$ *espacio vectorial normado.*

Recordemos que podemos definir el producto escalar $\langle a, b \rangle = \sum_{i=1}^{n} a_i \bar{b_i}$, que cumple $\|a\|_2 = \langle a, a \rangle^{1/2}$. Por lo tanto, la 2-norma puede expresarse en función del producto escalar, particularidad que será importante más adelante.

A lo largo de este trabajo, consideraremos funciones Lebesgue-medibles definidas en ciertos subconjuntos $S \subset \mathbb{R}^n$ sobre los que podremos integrar usando la integral de Lebesgue. A continuación enunciamos un resultado central en teoría de medida que nos será de gran utilidad, cuya demostración puede verse en ([25]).

Teorema 1.2 (Teorema de la convergencia dominada de Lebesgue)**.** Sea $\{f_n\}_{n \in \mathbb{N}}$ una sucesión de funciones $f_n : S \to \mathbb{C}$ convergiendo puntualmente a una función f en todo punto. Entonces, si existe una función integrable ϕ tal que $|f_n(x)| \leq \phi(x)$, para todo $x \in S$ y $n \in \mathbb{N}$, se tiene que

$$\int_S f_n(x) \, dx \to \int_S f(x) \, dx.$$

Definición 1.3. Dado $S \subset \mathbb{R}^n$, definimos $\mathcal{L}^p(S)$ como el espacio vectorial de funciones $f : S \to \mathbb{C}$ medibles para las que se cumple

$$\|f\|_p := \left(\int_S |f(x)|^p \, dx\right)^{1/p} < \infty,$$

junto con la suma de funciones y el producto escalar usual.

Esta definición es un primer intento de generalizar al caso funcional el espacio $\mathbb{C}^n$. Ahora el espacio vectorial formado por estas funciones será de dimensión infinita. Claramente, la aplicación $f \to \|f\|_p$ satisface las propiedades de no negatividad y homogeneidad con escalares. También se tiene la desigualdad triangular, conocida en estos espacios como la desigualdad de Minkowski ([25, th. 8.10]). Sin embargo, esta aplicación no es una norma válida, tan solo una *seminorma*, ya que no satisface la propiedad de definición positiva, es decir, existen funciones no idénticamente nulas con $\|f\|_p = 0$. En concreto, cualquier función con valor distinto de 0 en tan solo un conjunto de medida nula incumplirá esta propiedad.

Definición 1.4. Dada la relación de equivalencia

$$f \sim g \iff f(x) = g(x) \text{ para casi todo } x \in S,$$

definimos $L^p(S) = \mathcal{L}^p(S)/\sim$.

En $L^p(S)$ la seminorma $f \to \|f\|_p$ sí cumplirá la propiedad de definición positiva y por tanto $(L^p(S), \|f\|_p)$ es un espacio vectorial normado. La métrica inducida por esta norma nos permite dar una nueva noción de distancia entre funciones. Así, podemos definir la convergencia en norma en $L^p(S)$:

Definición 1.5. Sea $\{f_n\}$ una sucesión de funciones en $L^p(S)$, tendremos que $f_n \longrightarrow f$ en p-norma si y solo si $\|f_n - f\|_p \longrightarrow 0$.

Un importante resultado con demostración en ([25, th 8.14]) nos dice además que los $L^p(S)$ son espacios métricos completos (espacios de Banach):

Teorema 1.6. Si $\{f_n\}$ es una sucesión de Cauchy en $L^p(S)$, entonces existe $f \in L^p(S)$ tal que $f_n \longrightarrow f$.

Ahora centrémonos en el caso de $L^2(S)$. Al igual que en $\mathbb{C}^n$, podemos definir el producto escalar $\langle f, g\rangle = \int_S f(x)\overline{g(x)}dx$ y obtener la norma de $L^2(S)$ a partir de él: $\|f\|_2 = \langle f, f\rangle^{1/2}$. En general, se puede dar la siguiente definición, central en análisis funcional:

Definición 1.7. Un espacio de Hilbert (complejo) $\mathcal{H}$ es un espacio vectorial en el que existe un producto escalar $\langle a, b\rangle$ con valores en $\mathbb{C}$ satisfaciendo las siguientes propiedades:

1. $\langle a, b\rangle = \overline{\langle b, a\rangle}$, $\forall a, b \in \mathcal{H}$,
2. $\langle za + wb, c\rangle = z\langle a, c\rangle + w\langle b, c\rangle$, y $\langle a, zb + wc\rangle = \overline{z}\langle a, b\rangle + \overline{w}\langle a, c\rangle$, $\forall a, b, c \in \mathcal{H}$, $\forall z, w \in \mathbb{C}$,
3. $\langle a, a\rangle > 0$ si $a \neq 0$, $\langle a, a\rangle = 0$ si $a = 0$,

y cuya métrica asociada a la norma inducida por este producto escalar genera un espacio métrico completo. Es decir, un espacio de Hilbert es un espacio de Banach cuya métrica y norma son generadas por un producto escalar.

Por lo tanto, en virtud del Teorema 1.6, $L^2(S)$ será un espacio de Hilbert, siendo los demás $L^p(S)$, $p \neq 2$, tan solo espacios de Banach. Los espacios $\mathbb{C}^n$ también son de Hilbert, pero de dimensión finita.

Proposición 1.8. Sean f, $g \in L^2(S)$. Se cumplen las siguientes propiedades:

- Desigualdad de Cauchy-Schwarz: $|\langle f, g\rangle| \leq \|f\| \, \|g\|$, es decir,
$$\left| \int_S f(x)\overline{g(x)} \, dx \right| \leq \sqrt{\int_S |f(x)|^2 \, dx} \sqrt{\int_S |g(x)|^2 \, dx}.$$
- Desigualdad triangular: $\|f + g\| \leq \|f\| + \|g\|$, es decir,
$$\sqrt{\int_S |f(x) + g(x)|^2 \, dx} \leq \sqrt{\int_S |f(x)|^2 \, dx} + \sqrt{\int_S |g(x)|^2 \, dx}.$$
- Teorema de Pitágoras: si $\{\phi_k\}_{k=1}^n$ es una familia de funciones de $L^2(S)$ ortogonales dos a dos, esto es $\langle \phi_i, \phi_j\rangle = 0$, para todo $i, j \in \{1, \dots n\}$, $i \neq j$, entonces $\|\sum_{k=1}^n \phi_k\|^2 = \sum_{k=1}^n \|\phi_k\|^2$, es decir,
$$\int_S \left| \sum_{k=1}^n \phi_k(x) \right| \, dx = \sum_{k=1}^n \int_S |\phi_k(x)|^2 \, dx.$$

También podemos hablar de elementos ortogonales y ortonormales en espacios de Hilbert. Un conjunto ortonormal $\{\phi_i\}_{i \in I}$ será aquel donde todos los elementos sean ortogonales a pares y tengan norma unitaria.

Proposición 1.9 (Desigualdad de Bessel)**.** Sea $\{\phi_k\}_{k=1}^\infty$ un conjunto ortonormal de $\mathcal{H}$, entonces, para cualquier $f \in \mathcal{H}$, tendremos que
$$\sum_{k=1}^\infty |\langle f, \phi_k\rangle|^2 \leq \|f\|^2.$$

Además, aplicando Pitágoras, se tiene que
$$\left\| \sum_{k=1}^\infty \langle f, \phi_k\rangle \, \phi_k \right\|^2 = \sum_{k=1}^\infty |\langle f, \phi_k\rangle|^2 \, \|\phi_k\|^2 = \sum_{k=1}^\infty |\langle f, \phi_k\rangle|^2,$$
por lo que la serie infinita de proyecciones de f sobre $\{\phi_k\}_{k=1}^\infty$, $\sum_{k=1}^\infty \langle f, \phi_k\rangle \, \phi_k$, converge en $\mathcal{H}$.

1.2. Series de Fourier

En la Sección 1.1 hemos definido el espacio $L^2(S)$ como una especie de generalización de $\mathbb{C}^N$ a dimensión infinita. Cabe esperar que, al igual que en $\mathbb{C}^n$, podamos construir una base ortonormal que nos permita expresar cualquier función de $L^2(S)$ en un sistema de coordenadas. Sin embargo, $L^2(S)$ no tiene dimensión finita, por lo que no podemos saber a simple vista si un conjunto ortonormal de funciones $\{\phi_i\}_{i\in I}$ tendrá suficientes elementos como para poder recuperar cualquier función. Sin embargo, podemos definir lo siguiente:

Definición 1.10. Sea $\mathcal{H}$ un espacio de Hilbert. Una *base ortonormal* es un conjunto ortonormal $\{\phi_i\}_{i\in I}$ cuyo subespacio generado es denso en $\mathcal{H}$, es decir, $\overline{\langle\{\phi_i\}_{i\in I}\rangle} = \mathcal{H}$. Un espacio de Hilbert será *separable* si existe una base ortonormal numerable.

Un interesante resultado técnico ([25, th 8.35]) nos dice que entre todos los espacios de Hilbert separables de dimensión infinita existe una isometría, por lo que son isomorfos y se comportan esencialmente como $L^2(S)$, espacio en el que nos centraremos durante el resto de esta sección.

Proposición 1.11. Un conjunto ortonormal de funciones de $L^2(S)$, $\{\phi_i\}_{i\in I}$, será una base ortonormal si y solo si se cumple cualquiera de las siguientes condiciones equivalentes:

1. Si $\langle f, \phi_i\rangle = 0$, para todo $i \in I$, entonces $f = 0$.
2. Para cualquier $f \in L^2(S)$, tenemos que $f = \sum_{k=1}^{\infty} \langle f, \phi_k\rangle \phi_k$.
3. Para cualquier $f \in L^2(S)$, se satisface la *Identidad de Parseval*:

$$\|f\|^2 = \sum_{k=1}^{\infty} |\langle f, \phi_k\rangle|^2. \tag{1.1}$$

   Es decir, la desigualdad de Bessel se convierte en igualdad cuando el conjunto ortonormal de elementos forma una base.

Definición 1.12. Dada una base ortonormal $\{\phi_k\}$ de $L^2(S)$ y una función $f \in L^2(S)$, llamaremos *coeficientes de Fourier de f* a los productos $\langle f, \phi_k\rangle$. La suma $\sum_{k=1}^{\infty} \langle f, \phi_k\rangle \phi_k$ será la *serie de Fourier de f*.

Aquí hemos dado una definición general de los coeficientes y la suma de Fourier, en la que, a partir de una base ortonormal, podemos conseguir una cierta suma que convergerá a la función original en la norma de $L^2(S)$. En concreto, nos interesará el caso con $S = (-\pi, \pi)$ (o bien, el caso con cubos n-dimensionales de lado 2π centrados en el origen), ya que cualquier función definida en un intervalo (a, b) acotado (o en una región rectangular acotada) se puede trasladar y escalar hasta $L^2(-\pi, \pi)$.

A partir de propiedades de las funciones $L^2(S)$, se puede demostrar ([9, th 3.5]) que el conjunto de funciones exponenciales complejas $\left\{\frac{1}{\sqrt{2\pi}}e^{inx}\right\}_{n\in\mathbb{Z}}$ y el conjunto de funciones trigonométricas, $\left\{\frac{1}{\sqrt{2\pi}}\right\} \cup \left\{\frac{1}{\sqrt{\pi}}\cos(nx)\right\}_{n=1}^{\infty} \cup \left\{\frac{1}{\sqrt{\pi}}\operatorname{sen}(nx)\right\}_{n=1}^{\infty}$ son bases ortonormales de $L^2(-\pi,\pi)$. También tenemos las bases de $L^2(0,\pi)$ formadas por tan solo cosenos o senos, $\left\{\frac{1}{\sqrt{\pi}}\right\} \cup \left\{\frac{2}{\sqrt{\pi}}\cos(nx)\right\}_{n=1}^{\infty}$ y $\left\{\frac{1}{\sqrt{\pi}}\operatorname{sen}(nx)\right\}_{n=1}^{\infty}$, respectivamente.

Llamaremos *serie de Fourier compleja* de una función $f \in L^2(-\pi,\pi)$ a la serie de Fourier con respecto a la base exponencial compleja, es decir:

$$f(x) = \frac{1}{2\pi}\sum_{n\in\mathbb{Z}} c_n e^{inx}, \quad \text{donde } c_n = \int_{-\pi}^{\pi} f(x)e^{-inx}\,dx.$$

En el análisis de sumas de Fourier, normalmente se consideran funciones 2π-periódicas definidas en todo $\mathbb{R}$, o bien extensiones periódicas de funciones definidas en $(-\pi,\pi)$.

También es posible generalizar de nuevo estas ideas a funciones definidas en regiones *acotadas* $S \subset \mathbb{R}^n$. Al igual que en los intervalos acotados de $\mathbb{R}$, podemos trasladar y escalar cualquier función de $L^2(S)$ hasta un cubo n-dimensional de diámetro 2π centrado en el origen. Dentro de estos cubos, podemos construir bases ortonormales considerando el producto cartesiano de las ya dadas para $L^2(-\pi,\pi)$, o bien para $L^2(0,\pi)$.

1.3. Convoluciones

A lo largo de esta sección, introduciremos una operación que será de gran utilidad tanto en los resultados teóricos de la transformada de Fourier como en sus aplicaciones. De aquí en adelante, denotaremos $\int_{\mathbb{R}^n} f(x)\,dx = \int f(x)\,dx$.

Definición 1.13. Sean $f, g : \mathbb{R}^n \to \mathbb{C}$. Definimos su *convolución* como

$$\begin{aligned} f * g : \mathbb{R}^n &\longrightarrow \mathbb{C} \\ x &\longmapsto (f*g)(x) = \int f(x-y)g(y)\,dy = \int f(z)g(x-z)\,dz, \end{aligned}$$

siempre que la integral tenga sentido.

Intuitivamente, podemos pensar en la convolución como una media continua de f ponderada por g. En particular, dado $(a,b) \subset \mathbb{R}$, tenemos que la media de f en (a,b) viene dada por $\frac{1}{b-a}\int_a^b f(y)\,dy$. Ahora, considerando $w(x) = g(x-y)$ como función de peso, podemos definir la media ponderada por w como

$$\frac{\int_a^b f(y)g(x-y)\,dy}{\int_a^b g(x-y)\,dy}.$$

Finalmente, si tomamos $\int_a^b g(x-y)\,dy = 1$ y toda la recta real, llegamos precisamente a la definición de convolución unidimensional. Similarmente, podemos definir la *correlación cruzada* de f y g, denotada como $f \star g$, reflejando $f(x)$ o $g(x)$, es decir, tomando $f(-x)$ o $g(-x)$ en el caso real, o tomando el operador adjunto en el caso complejo:

$$(f \star g)(x) = \int \overline{f(y)} g(x+y)\,dy.$$

Ahora veamos cuándo la definición de convolución tiene sentido, es decir, cuándo la integral es convergente.

Proposición 1.14. Bajo cualquiera de siguientes condiciones, la integral $\int f(x-y)g(y)\,dy$ es absolutamente convergente, y por tanto $f * g$ está bien definida:

1. $f \in L^1(\mathbb{R}^n)$ y g acotada.
2. $f, g \in L^2(\mathbb{R}^n)$.
3. f localmente integrable (integrable en cualquier compacto $A \subset \mathbb{R}^n$) y g acotada y de soporte compacto.

Demostración. En efecto, si $|g(x)| \leq M$ y $f \in L^1(\mathbb{R}^n)$ entonces

$$\int |f(x-y)g(y)|\,dy \leq M \int |f(x-y)|\,dy < \infty,$$

cumpliéndose la primera condición. Ahora si $f, g \in L^2(\mathbb{R}^n)$, podemos utilizar la desigualdad de Cauchy-Schwarz para obtener la segunda condición

$$\int |f(x-y)g(y)|\,dy \leq \sqrt{\int |f(x-y)|^2\,dy}\sqrt{\int |g(y)|^2\,dy} < \infty.$$

Finalmente, al ser $y \to f(x-y)$ localmente integrable en el soporte A de g,

$$\int |f(x-y)g(y)|\,dy = \int_A |f(x-y)g(y)|\,dy \leq M \int_A |f(x-y)|\,dy < \infty,$$

demostrando la tercera condición. □

Existen más resultados acerca de la existencia y acotación de la convolución en espacios $L^p(\mathbb{R}^n)$. Por ejemplo, el siguiente resultado ([25, th. 9.2]) nos da una condición suficiente para la convergencia de la convolución en $L^p(\mathbb{R}^n)$:

Teorema 1.15 (de convolución de Young)**.** Sean $f \in L^p(\mathbb{R}^n)$ y $g \in L^q(\mathbb{R}^n)$, con $1/p + 1/q \geq 1$, entonces $f * g \in L^r(\mathbb{R}^n)$ con $\frac{1}{r} + 1 = \frac{1}{p} + \frac{1}{q}$. En particular, si $f, g \in L^1(\mathbb{R}^n)$, entonces $f * g \in L^1(\mathbb{R}^n)$ y está definida en casi todo punto de $\mathbb{R}^n$.

También tenemos la siguiente relación entre la diferenciación y la convolución:

Proposición 1.16. Sean f diferenciable y g tales que $f * g$ está bien definida, entonces $f * g$ es diferenciable y se tiene que

$$\frac{\partial}{\partial x_i}(f * g) = \frac{\partial f}{\partial x_i} * g.$$

Además, se cumplen las siguientes propiedades algebraicas:

Proposición 1.17. Sean f, g, h tales que $f * g$ y $f * h$ están bien definidas, entonces:

1. $f * g = g * f$.
2. $f * (ag + bh) = a(f * g) + b(f * h)$, $\forall a, b \in \mathbb{R}$.
3. $f * (g * h) = (f * g) * h$.

Estas propiedades se asemejan a las del producto escalar, pero no existe una función identidad para la convolución. Es decir, no podemos encontrar g tal que $f * g = f$. Sin embargo, sí podemos encontrar familias de funciones que se aproximan a una identidad tanto como queramos. Fijemos un $K \in L^1(\mathbb{R}^n)$, con $\int K(x)dx = 1$, al que llamaremos *núcleo de convolución*. Para $\varepsilon > 0$ definamos $K_\varepsilon(x) = \frac{1}{\varepsilon^n} K(\frac{x}{\varepsilon})$. Las funciones K_ε reducen el soporte de K y aumentan la altura de los picos de K cuando $\varepsilon < 1$. Además, la integral se mantiene constante para todo $\varepsilon \in \mathbb{R}$ ya que, aplicando el cambio de variable $y = x/\varepsilon$,

$$\int K_\varepsilon(x)\, dx = \int \frac{1}{\varepsilon^n} K\left(\frac{x}{\varepsilon}\right)\, dx = \int K(y)\, dy.$$

Por lo tanto las K_ε concentran, sin alterarlo, el volumen de la función en torno al origen cuando $\varepsilon \to 0$. Intuitivamente, para una función $f \in L^p(\mathbb{R}^n)$, $f * K_\varepsilon$ alterará cada vez menos la función cuando $\varepsilon \to 0$ debido a que calculará su media ponderada en un entorno cada vez más concentrado en torno al punto central. Además, si (haciendo un gran abuso de notación) tomamos la *función límite* cuando $\varepsilon \to 0$, podríamos entender a esa función K_0 como la delta de Dirac, definida con $\delta(x) = 0$ si $x \neq 0$ y $\int_{-a}^{a} \delta(x)dx = 1$, $\forall a > 1$. La delta de Dirac es una distribución o *función generalizada*, representando la idea detrás de la unidad en la convolución, y cómo nos podemos aproximar a ella a través de los núcleos K_ε.

Existen varios resultados que precisan cómo convergen los K_ε a la unidad. Para nuestros fines, nos sirve con enunciar los siguientes teoremas, que se pueden consultar en ([9, th 7.1]) y ([25, th 9.6]).

Teorema 1.18. Si $K \in L^1(\mathbb{R})$ es un núcleo de convolución con $\alpha = \int_{-\infty}^{0} K(x)\ dx$, $\beta = \int_{0}^{\infty} K(x)\ dx$, f es continua a trozos en $\mathbb{R}$ y además f es acotada o K es acotado y de soporte compacto, entonces:

$$\lim_{\varepsilon \to 0} f * K_\varepsilon(x) = \alpha f(x^+) + \beta f(x^-).$$

Teorema 1.19. Si $K \in L^1(\mathbb{R}^n)$ es un núcleo de convolución y $f \in L^p(\mathbb{R}^n)$, entonces $f * K_\varepsilon$ converge a f en norma, es decir,

$$\lim_{\varepsilon \to 0} \|(f * K_\varepsilon) - f\|_p = 0.$$

Uno de los núcleos de convolución más importantes es el *núcleo gaussiano*, definido en el caso unidimensional como

$$G(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}. \tag{1.2}$$

Es un resultado conocido que $\int_{-\infty}^{\infty} e^{-x^2} = \sqrt{\pi}$, por lo que $\int G(x)dx = 1$. Tenemos que G es par, por lo que, con la notación del Teorema 1.18, $\alpha = \beta = 1/2$. Además, tanto G como G_ε tienen derivadas integrables y acotadas, con $G^{(k)} = P_k(x)e^{-x^2}$, donde $P_k(x)$ es un polinomio de grado k. Por lo tanto, aplicando los teoremas anteriores, hemos llegado a una potente herramienta con la que podemos aproximar una función f, potencialmente irregular, por $f * G_\varepsilon \in C^\infty(\mathbb{R})$. En la Figura 1.1, podemos visualizar un ejemplo de esta aproximación.

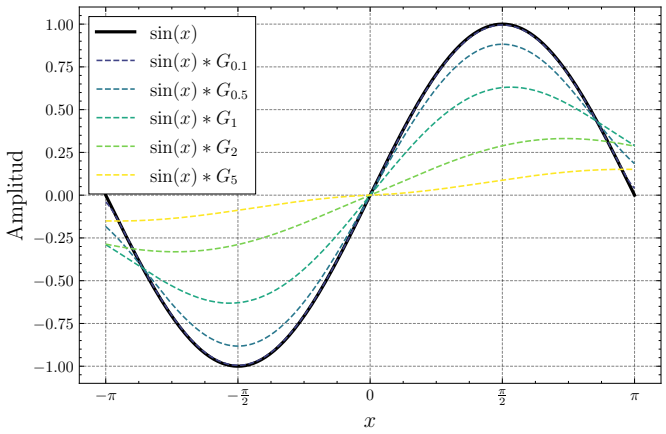


Figura 1.1: Convolución del núcleo G_ε con la función seno, para diferentes valores de ε.

Otro núcleo importante es el siguiente:

$$H(x) = \begin{cases} c\, e^{-\frac{1}{1-x^2}}, & x \in (-1,1) \\ 0, & x \in \mathbb{R} \setminus (-1,1) \end{cases}, \quad \text{con } c = \int_{-1}^{1} e^{-\frac{1}{1-x^2}}\, dx. \tag{1.3}$$

Utilizando el núcleo H podemos aproximar cualquier función de $L^p(\mathbb{R})$ tanto como queramos. En concreto, en la demostración del siguiente teorema, que se puede consultar en ([16, th 2.20]), se define la función $g = f_N * H_\varepsilon \in C^\infty(\mathbb{R})$, para un $\varepsilon > 0$ adecuado y siendo f_N la restricción de f a un intervalo $[-N, N]$ suficientemente grande.

Teorema 1.20 (densidad de C_c^∞ en L^p)**.** Sea $f \in L^p(\mathbb{R})$, entonces existe $g \in C^\infty(\mathbb{R})$ de soporte compacto tal que $\|f - g\|_p = 0$.

Capítulo 2

La transformada de Fourier

2.1. Aproximando funciones no periódicas

Como ya hemos visto, las series de Fourier nos permiten aproximar, bajo ciertas condiciones de regularidad, una función f en $L^2(-\pi, \pi)$, a partir de una suma de funciones exponenciales complejas (o bien, de funciones trigonométricas). Estas condiciones se pueden consultar en ([17]). Además, las series de Fourier se extienden naturalmente a funciones 2π-periódicas definidas en todo $\mathbb{R}$, que podemos identificar con las funciones definidas en el toro $f : \mathbb{T} \equiv \mathbb{R}/2\pi\mathbb{Z} \to \mathbb{C}$.

Una pregunta natural es si es posible también aproximar mediante funciones periódicas cualquier función $f : \mathbb{R} \to \mathbb{C}$, no necesariamente periódica. Sabemos que podemos aproximar una función f en $[-L, L]$ mediante su serie de Fourier (tan solo hace falta reescalar la expresión ya vista para funciones 2π-periódicas):

$$f(x) = \frac{1}{2L} \sum_{n \in \mathbb{Z}} c_{n,L} e^{in\pi x/L}, \quad \text{donde } c_{n,L} = \int_{-L}^{L} f(x) e^{-in\pi x/L} dx.$$

Notemos que el salto entre cada frecuencia de la serie viene dado por $\Delta\omega = \pi/L$ y cada una de estas frecuencias será $\omega_n = n\Delta\omega$. Podemos escribir entonces

$$f(x) = \frac{1}{2\pi} \sum_{n \in \mathbb{Z}} c_{n,L} e^{i\omega_n x} \, \Delta\omega, \quad \text{donde } c_{n,L} = \int_{-L}^{L} f(x) e^{-i\omega_n x} \, dx.$$

La idea para llegar a la definición de transformada es la siguiente: supongamos que $|f(x)| \to 0$ cuando $x \to \infty$ lo suficientemente rápido como para que $\int_{\mathbb{R}} f(x) e^{-i\omega_n x} dx$ esté acotada. Como veremos, es interesante estudiar los casos $f \in L^1(\mathbb{R})$ y $f \in L^2(\mathbb{R})$. Entonces, llevando al límite $L \to \infty$, el salto $\Delta\omega$ entre las frecuencias ω_n de la serie se hará arbitrariamente pequeño. Podemos considerar también los coeficientes $c_{n,L}$ como funciones definidas para cada una de las

frecuencias ω_n. Ahora realizamos, de manera informal, un proceso de paso al límite en el que $L \to \infty$, entonces $\Delta\omega \to 0$. A partir de la expresión de los coeficientes $c_{n,L}(\omega_n)$, llegamos a una función $\hat{f}(\omega)$:

$$c_{n,L}(\omega_n) = \int_{-L}^{L} f(x)e^{-i\omega_n x}\,dx \underset{\substack{L\to\infty\\ \Delta\omega\to 0}}{\rightsquigarrow} \hat{f}(\omega) = \int_{\mathbb{R}} f(x)e^{-i\omega x}\,dx.$$

Y, de forma casi simétrica, la serie de Fourier de f se convierte en

$$f(x) = \frac{1}{2\pi}\sum_{n\in\mathbb{Z}} c_{n_L}(\omega_n)e^{i\omega_n x}\,\Delta\omega \underset{\substack{L\to\infty\\ \Delta\omega\to 0}}{\rightsquigarrow} f(x) = \int_{\mathbb{R}} \hat{f}(\omega)e^{i\omega x}\,d\omega.$$

A lo largo de este capítulo, veremos hasta qué punto esta deducción heurística es válida, y cuándo las $\hat{f}$ y f se corresponden con la transformada de Fourier y su transformada inversa.

2.2. La transformada de Fourier en L^1

Definición 2.1. Dada $f \in L^1(\mathbb{R})$, la **transformada de Fourier** de f será la función $\hat{f} : \mathbb{R} \to \mathbb{C}$ definida como:

$$\hat{f}(\omega) := \int_{\mathbb{R}} f(x)e^{-ix\omega}\,dx, \quad \omega \in \mathbb{R}. \tag{2.1}$$

También emplearemos la notación $\mathscr{F}(f)$ para la transformada de Fourier.

En primer lugar, como $|f(x)e^{-ix\omega}| = |f(x)|$, entonces $|\hat{f}(\omega)| = \|f\|_1 < \infty$, ya que $f \in L^1(\mathbb{R})$, por lo que $\hat{f}$ está bien definida en todo punto. Si consideramos f como una función dependiente del tiempo, se suele decir que $f(x)$ está definida en el *dominio temporal*, mientras que $\hat{f}(\omega)$ está definida en el *dominio de frecuencia*.

A continuación, recogemos una lista de las propiedades más interesantes de la transformada de Fourier, las cuales referenciaremos a lo largo de todo el trabajo.

Proposición 2.2. Dada $f \in L^1(\mathbb{R})$, se tienen las siguientes propiedades:

1. Linealidad: para todo $\alpha, \beta \in \mathbb{R}$ se tiene que

$$\mathscr{F}(\alpha f + \beta g) = \alpha\hat{f} + \beta\hat{g}. \tag{2.2}$$

2. Traslación y modulación: para todo $x_0, \omega_0 \in \mathbb{R}$ definimos $f(x - x_0)$ la traslación de f por x_0 y $e^{-i\omega_0 x}f(x)$ la modulación de f por ω_0, entonces

$$\mathscr{F}(f(x - x_0))(\omega) = e^{-ix_0\omega}\hat{f}(\omega) \quad \text{y} \quad \mathscr{F}(e^{-i\omega_0 x}f(x))(\omega) = \hat{f}(\omega_0 + \omega). \tag{2.3}$$

3. Escalado: para todo $\delta \in \mathbb{R}/\{0\}$, definimos $f_\delta(x) = \delta^{-1} f(x/\delta)$, entonces

$$\hat{f}_\delta(\omega) = \hat{f}(\delta\omega) \quad \text{y} \quad \mathscr{F}(f(\delta x))(\omega) = (\hat{f})_\delta(\omega). \tag{2.4}$$

4. Diferenciación y producto: si f es continua y de clase 1 a trozos, y además $f' \in L^1(\mathbb{R})$, entonces

$$\mathscr{F}(f')(\omega) = i\omega \hat{f}(\omega). \tag{2.5}$$

   Por otro lado, si $xf(x) \in L^1(\mathbb{R})$, entonces

$$\mathscr{F}(xf(x))(\omega) = i(\hat{f})'(\omega). \tag{2.6}$$

Demostración. Las demostraciones de (2.2), (2.3) y (2.4) son inmediatas a partir de la definición. Para probar (2.5), usamos que, al ser $f \in L^1(\mathbb{R})$, entonces

$$\lim_{x\to\infty} f(x) = f(0) + \int_0^\infty f'(x)\, dx = 0,$$

y lo mismo ocurre cuando $x \to -\infty$. Por lo tanto podemos integrar por partes:

$$\mathscr{F}(f')(\omega) = \int_{\mathbb{R}} e^{-i\omega x} f'(x)\, dx = -\int_{\mathbb{R}} (-i\omega) e^{-i\omega x} f(x)\, dx = i\omega \hat{f}(\omega).$$

Por otro lado, si $xf(x) \in L^1(\mathbb{R})$, entonces, como $i(d/d\omega)e^{-i\omega x} = xe^{-i\omega x}$, obtenemos

$$\mathscr{F}(xf(x)) = \int_{\mathbb{R}} e^{-i\omega x} xf(x)\, dx = i\frac{d}{d\omega}\int_{\mathbb{R}} e^{-i\omega x} f(x)\, dx = i(\hat{f})'(\omega).$$

□

La siguiente propiedad, probablemente la más importante de la transformada de Fourier, nos permite transformar la operación de convolución en un simple producto de funciones.

Teorema 2.3 (Teorema de convolución)**.** Sean $f, g \in L^1(\mathbb{R})$, entonces:

$$\mathscr{F}(f * g)(\omega) = \hat{f}(\omega)\hat{g}(\omega). \tag{2.7}$$

Demostración. Por el Teorema 1.15 tenemos que $f * g \in L^1(\mathbb{R})$, por lo tanto podemos aplicar el Teorema de Fubini e intercambiar el orden de las integrales:

$$\begin{aligned}
\mathscr{F}(f * g)(\omega) &= \int_{\mathbb{R}} (f * g)(x) e^{-i\omega x}\, dx = \int_{\mathbb{R}} \left(\int_{\mathbb{R}} f(y) g(x-y)\, dy \right) e^{-i\omega x}\, dx \\
&= \int_{\mathbb{R}} f(y) \left(\int_{\mathbb{R}} g(x-y) e^{-i\omega(x-y)}\, dx \right) e^{-i\omega y}\, dy \\
&= \int_{\mathbb{R}} f(y) \left(\int_{\mathbb{R}} g(t) e^{-i\omega t}\, dt \right) e^{-i\omega y}\, dx = \hat{f}(\omega)\hat{g}(\omega).
\end{aligned}$$

□

Como veremos más adelante, el teorema de convolución y sus versiones discretas son la base de infinidad de aplicaciones en los campos de la combinatoria, el procesamiento de señales y la compresión de datos, entre muchos otros. Veamos ahora algunos ejemplos de cálculo explícito de la transformada.

Ejemplo 2.4. Sea $f = \chi_{(-a,a)}$ la función característica en el intervalo centrado en 0 con radio a. Su transformada de Fourier viene dada por

$$\hat{f}(\omega) = \int_{-a}^{a} e^{-i\omega x}\, dx = \frac{-e^{-i\omega x} + e^{i\omega x}}{i\omega} = \frac{2ia\sin(a\omega)}{ia\omega} = 2a\,\mathrm{sinc}(a\omega),$$

definiendo la función seno cardinal $\mathrm{sinc}(x)$ como la extensión continua a $\mathbb{R}$ de $\sin(x)/x$:

$$\mathrm{sinc}(x) = \begin{cases} \frac{\sin(x)}{x}, & x \neq 0, \\ 1, & x = 0. \end{cases} \tag{2.8}$$

Normalmente, no es posible calcular la transformada de Fourier directamente. En el siguiente ejemplo emplearemos ecuaciones diferenciales para lograrlo.

Ejemplo 2.5. Consideremos la distribución Gaussiana estándar,

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}, \quad x \in \mathbb{R}.$$

Es un resultado conocido que $\int_{\mathbb{R}} f(x)\, dx = 1$, por lo que $f \in L^1(\mathbb{R})$, ya que $f > 0$. Calculemos ahora su transformada de Fourier,

$$\hat{f}(\omega) = \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} e^{-x^2/2} e^{-i\omega x}\, dx.$$

Como $f'(x) + xf(x) = 0$, empleando las propiedades (2.5) y (2.6) obtenemos la siguiente ecuación diferencial en el dominio de frecuencia:

$$i\omega\hat{f}(\omega) + i(\hat{f})'(\omega) = 0.$$

Esta ecuación diferencial es lineal, y se puede resolver fácilmente por separación de variables:

$$\frac{(\hat{f})'(\omega)}{\hat{f}(\omega)} = -\omega \Longrightarrow \log(\hat{f}(\omega)) = -\frac{\omega^2}{2} + \log C \Longrightarrow \hat{f}(\omega) = Ce^{-\omega^2/2}.$$

Y como $C = f(0) = \int_{\mathbb{R}} f(x)\, dx = 1$, entonces

$$\hat{f}(\omega) = e^{-\omega^2/2}. \tag{2.9}$$

Observemos que $f = \sqrt{2\pi}\hat{f} = \sqrt{2\pi}\mathscr{F}(f)$. Como $\mathscr{F}$ es un operador lineal, es adecuado decir que f es una *autofunción* de la transformada de Fourier. Mediante las propiedades (2.3) y (2.4), es inmediato obtener la transformada de Fourier de una distribución Gaussiana general:

$$g(x) = f_\sigma(x-\mu) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \Longrightarrow \hat{g}(x) = e^{-i\mu\omega} e^{-\frac{\sigma^2\omega^2}{2}}.$$

.

La siguiente propiedad está relacionada con el comportamiento en el límite. Sabemos que, como $f \in L^1(\mathbb{R})$, entonces $f(x) \to 0$ cuando $|x| \to \infty$, veamos que a su transformada le ocurre lo mismo.

Lema 2.6 (de Riemann-Lebesgue)**.** Sea $f \in L^1(\mathbb{R})$, entonces $\lim_{|\omega|\to\infty} \hat{f}(\omega) = 0$.

Demostración. En primer lugar, consideremos una función simple, $s(x) = \sum_{j=1}^{N} c_j \chi_{A_j}(x)$, donde $c_j \in \mathbb{R}$ y $A_j = (x_j - a_j, x_j + a_j)$, con $x_j, a_j \in \mathbb{R}$. Entonces, usando las propiedades de linealidad y traslación y aplicando el Ejemplo 2.4, tendremos que $\hat{f} \to 0$, ya que

$$\mathscr{F}(\mathcal{X}_{A_j})(\omega) = 2a_j e^{-ix_j\omega} \operatorname{sinc}(a_j\omega) \to 0, \quad \text{para todo } j = 1, \ldots, N.$$

Ahora podemos emplear el teorema de aproximación de funciones medibles mediante funciones simples ([21, Th. 11.20]) y encontrar una sucesión $\{s_n\}$ de funciones simples tales que $\int |s_n(x) - f(x)|dx \to 0$. Pero entonces

$$\sup_{\omega\in\mathbb{R}} |\hat{s}_n(\omega) - \hat{f}(\omega)| = \sup_{\omega\in\mathbb{R}} |\mathscr{F}(s_n(\omega) - f(\omega))| \leq \int_{\mathbb{R}} |s_n(x) - f(x)| \, dx \to 0,$$

por lo que $\hat{s}_n \to \hat{f}$ uniformemente y entonces, como $\hat{s}_n \to 0$, se tendrá $\hat{f} \to 0$. □

Denotemos ahora como $C_0(\mathbb{R})$ al espacio lineal de funciones continuas $f : \mathbb{R} \to \mathbb{C}$ tales que $\lim_{|x|\to\infty} f(x) = 0$. Obviamente, si $f \in C_0(\mathbb{R})$, entonces f está acotada. Dotamos a este espacio con la norma del máximo:

$$\|f\|_{C_0(\mathbb{R})} = \max_{x\in\mathbb{R}} |f(x)|.$$

El siguiente teorema caracteriza la transformada de Fourier en $L^1(\mathbb{R})$ como un operador lineal que lleva funciones de $L^1(\mathbb{R})$ a funciones de $C_0(\mathbb{R})$.

Teorema 2.7. La transformada de Fourier $\mathscr{F}$ es un operador lineal continuo de $L^1(\mathbb{R})$ a $C_0(\mathbb{R})$. Además, la norma de este operador es 1, es decir, $1 = \text{ínf}\{c \geq 0 : \|\mathscr{F}f\|_{C_0(\mathbb{R})} \leq c\,\|f\|_1\}$.

Demostración. Ya hemos visto que $\mathscr{F}$ es lineal, veamos que es continuo. Sean $\omega, h \in \mathbb{R}$, entonces

$$|\hat{f}(\omega + h) - \hat{f}(\omega)| = \left| \int_{\mathbb{R}} f(x) e^{-i\omega x}(e^{-ihx} - 1) \, dx \right| \leq \int_{\mathbb{R}} |f(x)||e^{-ihx} - 1| \, dx.$$

Por otro lado, como $|e^{-ihx}| = 1$, entonces $|f(x)||e^{-ihx} - 1| \leq 2|f(x)| \in L^1(\mathbb{R})$. Se tiene también que $|e^{-ihx} - 1| \to 0$ cuando $h \to 0$, por lo que, tomando cualquier sucesión $\{h_n\} \to 0$, podemos aplicar el Teorema 1.2 y obtener:

$$\lim_{n\to\infty} |\hat{f}(\omega + h_n) - \hat{f}(\omega)| \leq \lim_{n\to\infty} \int_{\mathbb{R}} |f(x)||e^{-ih_n x} - 1| \, dx = \int_{\mathbb{R}} |f(x)| \left(\lim_{n\to\infty} |e^{-ih_n x} - 1| \right) \, dx = 0,$$

por lo que $\hat{f}$ es continua.

Este resultado, junto al Lema de Riemann-Lebesgue, concluye que $\hat{f} \in C_0(\mathbb{R})$. Para ver que la norma del operador es 1, primero observamos que para cualquier función $f \in L^1(\mathbb{R})$,

$$|\mathscr{F}(f)(\omega)| \leq \int_{\mathbb{R}} |f(x)|\, dx = \|f\|_1 \Longrightarrow \|\mathscr{F} f\|_{C_0(\mathbb{R})} \leq \|f\|_1 \,,$$

Finalmente, para ver que se alcanza el ínfimo en 1, tomamos $g(x)$ la distribución Gaussiana estándar. Ya vimos en (2.9) que $\hat{g}(\omega) = e^{-\omega^2/2}$, y como $\max_{\omega\in\mathbb{R}} |e^{-\omega^2/2}| = 1$, llegamos a $\|\mathscr{F} g\|_{C_0(\mathbb{R})} = \|g\|_1$. □

Hasta ahora hemos expuesto como pasar funciones al dominio de frecuencia y qué propiedades exhiben sus transformadas, veamos ahora cómo recuperar la función original. Esta operación será casi simétrica al cálculo de la transformada, lo cual facilitará los algoritmos para el cálculo en el caso discreto, que analizaremos en los Capítulos 3 y 4.

Teorema 2.8 (de inversión de Fourier)**.** Si $f \in L^1(\mathbb{R})$ es una función continua a trozos, definida como $f(x) = \frac{1}{2}(f(x^-) + f(x^+))$ en sus puntos de discontinuidad, entonces

$$f(x) = \frac{1}{2\pi} \lim_{\varepsilon\to 0} \int_{\mathbb{R}} e^{i\omega x} e^{-(\omega^2\varepsilon^2)/2} \hat{f}(\omega)\, d\omega, \quad x \in \mathbb{R}. \tag{2.10}$$

Además, si $\hat{f} \in L^1(\mathbb{R})$, entonces f será continua y

$$f(x) = \frac{1}{2\pi} \int_{\mathbb{R}} e^{i\omega x} \hat{f}(\omega)\, d\omega, \quad x \in \mathbb{R}. \tag{2.11}$$

Demostración. El problema principal con la fórmula (2.11) es que, si $\hat{f} \notin L^1(\mathbb{R})$, entonces la integral no tiene por qué converger. Para evitar este problema, la idea principal de esta demostración es multiplicar por una función que tiende a 0 rápidamente, llamada *función de corte*. Con este propósito, usaremos $g_\varepsilon(\omega) = e^{-\omega^2\varepsilon^2/2}$. Claramente, $\lim_{|\omega|\to\infty} g_\varepsilon(\omega) = 0$ y $\lim_{\varepsilon\to 0} g_\varepsilon(\omega) = 1$, puntualmente, para todo $\omega \in \mathbb{R}$. Entonces tenemos

$$\frac{1}{2\pi} \int_{\mathbb{R}} e^{i\omega x} e^{-\varepsilon^2\omega^2/2} \hat{f}(\omega)\, d\omega = \frac{1}{2\pi} \int_{\mathbb{R}} \int_{\mathbb{R}} e^{i\omega(x-y)} e^{-\varepsilon^2\omega^2/2} f(y)\, dy\, d\omega.$$

Como esta integral es absolutamente convergente, ya que $\hat{f}$ está acotada y $e^{-i\omega x-(\omega^2\varepsilon^2)/2} \in L^1(\mathbb{R}^2)$, podemos aplicar el Teorema de Fubini e intercambiar el orden de integración. Además, aplicando (2.9),

$$\int_{\mathbb{R}} e^{i\omega(x-y)} e^{-\varepsilon^2\omega^2/2}\, d\omega = \mathscr{F}(e^{-\varepsilon\omega^2/2})(y-x) = \frac{\sqrt{2\pi}}{\varepsilon} e^{-(x-y)^2/2\varepsilon^2}.$$

Usando ahora el núcleo de convolución gaussiano (1.2), tenemos que

$$\frac{1}{2\pi}\int_{\mathbb{R}}\int_{\mathbb{R}} e^{i\omega(x-y)}e^{-\varepsilon^2\omega^2}f(y)\,dy\,d\omega = \frac{1}{\varepsilon\sqrt{2\pi}}\int_{\mathbb{R}} f(y)e^{-(x-y)^2/2\varepsilon^2}\,dy = f * G_\varepsilon(x).$$

Finalmente, aplicando el Teorema 1.18, obtenemos que

$$\frac{1}{2\pi}\lim_{\varepsilon\to 0} f * G_\varepsilon(x) = \frac{1}{2}(f(x^-) + f(x^+)) = f(x).$$

Si $\hat{f} \in L^1(\mathbb{R})$, entonces podemos interpretar la integral en (2.11) como la transformada de Fourier de $\hat{f}$ evaluada en $-x$. Por otro lado, como $|e^{i\omega x}e^{-\omega^2\varepsilon^2/2}\hat{f}(\omega)| \le |\hat{f}(\omega)|$, podemos aplicar el teorema de convergencia dominada de Lebesgue:

$$\frac{1}{2\pi}\mathscr{F}(\hat{f})(-x) = \frac{1}{2\pi}\int_{\mathbb{R}} e^{i\omega x}\hat{f}(\omega)\,d\omega = \frac{1}{2\pi}\lim_{\varepsilon\to 0}\int_{\mathbb{R}} e^{i\omega x}e^{-\omega^2\varepsilon^2/2}\hat{f}(\omega)\,d\omega = f(x),$$

donde en el último paso hemos usado la continuidad de f, consecuencia del Teorema 2.7 aplicado a $\mathscr{F}(\hat{f})(-x)$. □

Es un corolario inmediato que la transformada de Fourier es única en $L^1(\mathbb{R})$.

Corolario 2.9 (Unicidad)**.** Si $\hat{f} = \hat{g}$, entonces $f = g$.

Demostración. Por la linealidad de la transformada, tendremos que $\mathscr{F}(f-g) = 0$, y por lo tanto $f - g = 0$. □

Nótese que existen más variantes del Teorema de Inversión de Fourier. En particular, se puede demostrar en casi todo punto para funciones de $L^1(\mathbb{R})$ no continuas a trozos, siguiendo un esquema similar en el que multiplicamos por otra función de corte, véase ([16, Th. 2.10]).

Llegados a este punto, cabe indicar que existen otras versiones de la transformada de Fourier en la literatura, cuya definición y propiedades son idénticas excepto en el factor de 2π que nos encontramos en el Teorema de Inversión. En concreto, tenemos las siguientes expresiones alternativas para la transformada y su inversa:

$$\begin{aligned}
\hat{f}(\omega) = \frac{1}{\sqrt{2\pi}}\int_{\mathbb{R}} f(x)e^{-i\omega x}\,dx \quad &\rightsquigarrow \quad f(x) = \frac{1}{\sqrt{2\pi}}\int_{\mathbb{R}} \hat{f}(\omega)e^{i\omega x}\,d\omega,\\
\hat{f}(\omega) = \int_{\mathbb{R}} f(x)e^{-2\pi i\omega x}\,dx \quad &\rightsquigarrow \quad f(x) = \int_{\mathbb{R}} \hat{f}(\omega)e^{2\pi i\omega x}\,d\omega.
\end{aligned} \tag{2.12}$$

2.3. Extensión al espacio L^2

Existen muchos ejemplos de funciones que no están en $L^2(\mathbb{R})$, pero si en $L^1(\mathbb{R})$, y viceversa. Consideremos, por ejemplo, la función $f(x) = \operatorname{sinc}(x)$, $x \in \mathbb{R}$, definida en (2.8). Es sencillo

demostrar que $\operatorname{sinc}(x) \in L^2(\mathbb{R})$. Si $|x| > \pi/2$, entonces $\frac{\operatorname{sen}^2(x)}{x^2} < \frac{1}{x^2}$, y si $|x| \leq \pi/2$, la función está acotada por ser continua en ese compacto, por lo tanto:

$$\int_{\mathbb{R}} \operatorname{sinc}^2(x)\, dx \leq \int_{|x|\leq\pi/2} M\, dx + \int_{|x|>\pi/2} \frac{1}{x^2}\, dx < \infty.$$

Pero esta función no está en $L^1(\mathbb{R})$, ya que la integral $\int_{\mathbb{R}} \operatorname{sin}(x)/x\, dx$ no es absolutamente convergente. Por lo tanto, no podemos definir la transformada de Fourier en el sentido usual, ya que $|f(x)e^{-i\omega x}| \leq |f(x)|$. Sin embargo, si podremos extender la transformada de Fourier a L^2 mediante un proceso de paso al límite. Para ello, comenzamos demostrando una fórmula análoga a (1.1) para el caso de la transformada:

Lema 2.10 (Identidad de Parseval)**.** Sean $f, g \in L^1(\mathbb{R})$, con $\hat{f}, \hat{g} \in L^1(\mathbb{R})$. Entonces

$$\langle \hat{f}, \hat{g} \rangle_2 = 2\pi \langle f, g \rangle_2, \quad \text{y por lo tanto,} \quad \|\hat{f}\|_2 = \sqrt{2\pi}\, \|f\|_2. \tag{2.13}$$

Demostración. En primer lugar, $f \in L^2(\mathbb{R})$, ya que por el Teorema de Inversión de Fourier $\mathscr{F}(\hat{f})(x) \in C_0(\mathbb{R})$ y es igual a f (o lo es en casi todo punto, aunque f no sea continua a trozos, por la extensión que ya enunciamos). Entonces se tiene que

$$\int_{\mathbb{R}} |f(x)|^2\, dx = \int_{\mathbb{R}} |\mathscr{F}(\hat{f})(x)||f(x)|\, dx \leq \left\|\mathscr{F}(\hat{f})\right\|_{C_0(\mathbb{R})} \|f(x)\|_1 < \infty.$$

De la misma forma, $g, \hat{f}$ y $\hat{g} \in L^2(\mathbb{R})$. Ahora podemos aplicar el Teorema de Inversión de Fourier y el Teorema de Fubini para obtener la igualdad:

$$\begin{aligned}\langle \hat{f}, \hat{g} \rangle_2 &= \int_{\mathbb{R}} \hat{f}(\omega)\overline{\hat{g}(\omega)}\, d\omega = \int_{\mathbb{R}} \hat{f}(\omega) \overline{\int_{\mathbb{R}} g(x)e^{-i\omega x}\, dx}\, d\omega \\ &= \int_{\mathbb{R}} \overline{g(x)} \int_{\mathbb{R}} \hat{f}(\omega)e^{i\omega x}\, d\omega\, dx = 2\pi \int_{\mathbb{R}} \overline{g(x)} f(x)\, dx = 2\pi \langle f, g \rangle_2.\end{aligned}$$

□

Ahora, sea $f \in L^2(\mathbb{R})$. Sabemos por el Teorema 1.20 que existe una sucesión $\{f_n\}_{n\in\mathbb{N}}$ de funciones $C^\infty(\mathbb{R})$ de soporte compacto tales que $\lim_{n\to\infty} \|f - f_n\|_2 = 0$. Por estar acotadas en un compacto, tanto f como sus derivadas son de $L^1(\mathbb{R})$. Esto nos asegura además que $\hat{f} \in L^1(\mathbb{R})$, hcho que se puede demostrar a partir de la propiedad $\mathscr{F}(f'')(\omega) = -\omega^2 \hat{f}(\omega)$. Tenemos entonces que $\{f_n\}_{n\in\mathbb{N}}$ es una sucesión de Cauchy. Aplicando la Identidad de Parseval podemos ver que $\{\hat{f}_n\}_{n\in\mathbb{N}}$ también es de Cauchy:

$$\left\|\hat{f}_n - \hat{f}_m\right\| = 2\pi \|f_n - f_m\| \to 0, \quad n, m \to \infty.$$

Pero como $L^2(\mathbb{R})$ es un espacio completo, llegamos finalmente a que esta sucesión convergerá a una función de $L^2(\mathbb{R})$, que definimos como la transformada de Fourier de f:

$$\hat{f} = \mathscr{F}(f) := \lim_{n\to\infty} \hat{f}_n.$$

De esta forma, hemos extendido la transformada al espacio $L^1(\mathbb{R}) \cup L^2(\mathbb{R})$. Nótese que el espacio $L^1(\mathbb{R}) \cap L^2(\mathbb{R})$, donde la transformada ya estaba definida, es denso en $L^2(\mathbb{R})$, ya que $C_c^\infty(\mathbb{R}) \subset L^1(\mathbb{R}) \cap L^2(\mathbb{R})$ y $C_c^\infty(\mathbb{R})$ es denso en $L^2(\mathbb{R})$. Ahora, sin más que utilizar la continuidad del producto interno, podemos extender la identidad de Parseval a $L^2(\mathbb{R})$.

Teorema 2.11 (de Plancherel)**.** La transformada de Fourier definida en $L^1(\mathbb{R}) \cap L^2(\mathbb{R})$ se extiende con unicidad a un operador $\mathscr{F} : L^2(\mathbb{R}) \to L^2(\mathbb{R})$ de forma que, para todo $f, g \in L^2(\mathbb{R})$, se satisface la Identidad de Parseval. De esta forma, $\mathscr{F}$ es una isometría entre los espacios métricos $L^2(\mathbb{R})$.

2.4. Generalización a varias dimensiones

En esta sección, extenderemos la transformada de Fourier al caso multidimensional. Consideramos los vectores variables en los dominios de tiempo y frecuencia $x, \omega \in \mathbb{R}^n$, respectivamente. Definimos la transformada de Fourier *n-dimensional* como:

Definición 2.12. Sea $f \in L^1(\mathbb{R}^n)$, su transformada de Fourier $\hat{f} \equiv \mathscr{F}(f)$ será

$$\hat{f}(\omega) = \int_{\mathbb{R}^n} f(x) e^{-ix\cdot\omega}\, dx. \tag{2.14}$$

En particular, si $n = 2$, tenemos la siguiente expresión en coordenadas cartesianas:

$$\hat{f}(\omega_1, \omega_2) = \int_{\mathbb{R}} \int_{\mathbb{R}} e^{-i(x_1\omega_1 + x_2\omega_2)} f(x_1, x_2)\, dx_1\, dx_2. \tag{2.15}$$

Al igual que antes, considerando la norma de $\mathbb{C}^n$, tenemos $\left\| f(x) e^{-i\omega\cdot x} \right\| \leq \|f(x)\|$ por lo que la integral converge por ser $f \in L^1(\mathbb{R}^n)$.

Lema 2.13. Si $f = \prod_{j=1}^n f_j$, entonces $\hat{f} = \prod_{j=1}^n \hat{f}_j$.

Demostración. Basta usar $e^{-i\omega\cdot x} = \prod_{j=1}^n e^{-i\omega_j x_j}$ y aplicar el teorema de Fubini para descomponer la integral sobre $\mathbb{R}^n$ en un producto de transformadas de Fourier unidimensionales. □

Ejemplo 2.14. Aplicando el anterior resultado, podemos generalizar los Ejemplos 2.4 y 2.5:

$$\mathscr{F}\left((2\pi)^{-n/2} e^{-|x|^2/2}\right)(\omega) = \mathscr{F}\left(\prod_{j=1}^n \frac{1}{\sqrt{2\pi}} e^{-x_j^2/2}\right)(\omega) = \prod_{j=1}^n e^{-\omega_j^2/2} = e^{-|\omega|^2/2},$$

$$\mathscr{F}\left(\chi_{(-a,a)^n}\right)(\omega) = \mathscr{F}\left(\prod_{j=1}^n \chi_{(-a,a)}\right)(\omega) = \prod_{j=1}^n 2a\,\mathrm{sinc}(a\omega_j) = (2a)^n \prod_{j=1}^n \mathrm{sinc}(a\omega_j).$$

En la siguiente proposición, recogemos las propiedades más importantes de la transformada multidimensional, heredadas en su mayor parte del caso unidimensional.

Proposición 2.15. Sea $f \in L^1(\mathbb{R}^n)$:

1. Linealidad: para todo $\alpha, \beta \in \mathbb{R}^n$ se tiene

$$\mathscr{F}(\alpha f + \beta g) = \alpha \hat{f} + \beta \hat{g}.$$

2. Traslación y modulación: para todo $x_0, \omega_0 \in \mathbb{R}^n$ definimos $f(x - x_0)$ la traslación de f por x_0 y $e^{-i\omega_0 \cdot x} f(x)$ la modulación de f por ω_0, entonces

$$\mathscr{F}(f(x - x_0))(\omega) = e^{-ix_0 \cdot \omega} \hat{f}(\omega) \quad \text{y} \quad \mathscr{F}(e^{-i\omega_0 \cdot x} f(x))(\omega) = \hat{f}(\omega_0 + \omega).$$

3. Escalado: para todo $\delta \in \mathbb{R}/\{0\}$, definimos $f_\delta(x) = \delta^{-n} f(x/\delta)$, entonces

$$\hat{f}_\delta(\omega) = \hat{f}(\delta\omega) \quad \text{y} \quad \mathscr{F}(f(\delta x))(\omega) = (\hat{f})_\delta(\omega).$$

4. Diferenciación y producto: si f es continua, $\partial f/\partial x_j$ existe, y además $\partial f/\partial x_j \in L^1(\mathbb{R}^n)$, entonces

$$\mathscr{F}(\partial f/\partial x_j)(\omega) = i\omega_j \hat{f}(\omega).$$

   Por otro lado, si $x_j f(x) \in L^1(\mathbb{R}^n)$, entonces

$$\mathscr{F}(x_j f(x))(\omega) = i\partial \hat{f}/\partial x_j(\omega).$$

5. Teorema de convolución: Sea $g \in L^1(\mathbb{R}^n)$, entonces, por el Teorema 1.15, $f * g \in L^1(\mathbb{R}^n)$, y además

$$\mathscr{F}(f * g)(\omega) = \hat{f}(\omega)\hat{g}(\omega).$$

De forma similar, se puede generalizar el teorema de inversión, cambiando en este caso la función de corte por otra basada en la distribución Gaussiana multivariable.

Teorema 2.16. Si $f \in L^1(\mathbb{R}^n)$ y continua en $\mathbb{R}^n$, entonces

$$f(x) = \lim_{\varepsilon \to 0} \frac{1}{(2\pi)^n} \int_{\mathbb{R}^n} e^{i\omega \cdot x} e^{-\varepsilon^2 |\omega|^2/2} \hat{f}(\omega)\, d\omega, \quad x \in \mathbb{R}^n.$$

Además, si $\hat{f} \in L^1(\mathbb{R}^n)$, entonces

$$f(x) = \frac{1}{(2\pi)^n} \int_{\mathbb{R}^n} e^{i\omega \cdot x} \hat{f}(\omega)\, d\omega, \quad x \in \mathbb{R}^n.$$

Capítulo 3

Transformando señales discretas

3.1. Procesamiento de señales

A lo largo del capítulo previo, hemos analizado la construcción y las propiedades fundamentales de la Transformada de Fourier. Ahora llevaremos los conceptos vistos al caso de señales discretas, que muestrean una función en un conjunto finito de puntos. Las referencias principales de este capítulo serán ([16]) y ([3]), aunque también seguiremos ([9]) en la primera parte.

Hasta ahora, hemos trabajado con funciones reales dentro de espacios funcionales como $L^1(\mathbb{R})$ o $L^2(\mathbb{R})$. En la práctica, normalmente no tenemos una función de variable real o compleja bien definida, sino que tan solo disponemos de una *señal* discreta. Las señales puden ser de muchos tipos: de radio captada en una antena, de audio, de GPS, de Wi-Fi, etc. Normalmente, podemos pensar en la señal como en una función suficientemente regular, pero que tan solo puede ser *muestreada* en un conjunto finito de N puntos, dentro de un intervalo limitado y con una frecuencia de muestreo marcada, que denotaremos como f_s. Por ejemplo, una señal de audio capturada en un micrófono suele tener una frecuencia de muestreo de 8000 Hz a 48000 Hz.

Para poder convertir estas señales a cierto dominio de frecuencia (también llamado espectro) como en el caso continuo, necesitaremos adaptar la Transformada de Fourier. Además, no solo consideramos como señales los datos provenientes de ondas electromagnéticas, sino que cualquier secuencia de datos se puede tratar como tal, como los píxeles de una imagen o los coeficientes de un polinomio.

Todo esto nos lleva a introducir el concepto de Transformada Discreta de Fourier (DFT), que, entre otras motivaciones, busca aproximar la transformada continua de una señal muestreada. A continuación, veremos cómo realizar este proceso de discretización de la transformada de Fourier.

Sea $f \in L^1(\mathbb{R}) \cap C(\mathbb{R})$ y $n \in \mathbb{N}$. Consideremos la rejilla uniforme de puntos

$$A = \left\{ \frac{2\pi j}{N} : j = \frac{-nN}{2}, \dots, \frac{nN}{2} - 1 \right\} \in \mathbb{R}^{nN},$$

en la que muestreamos la función f. Ahora, como $f \in L^1(\mathbb{R})$, para un n suficientemente grande, se puede aproximar la integral de la transformada de Fourier como una integral en $[-n\pi, n\pi)$ y utilizar la regla del rectángulo en ese intervalo para discretizarla

$$\hat{f}(\omega) = \int_{\mathbb{R}} f(x) e^{-i\omega x}\, dx \approx \int_{[-n\pi, n\pi)} f(x) e^{-i\omega x}\, dx \approx \frac{2\pi}{N} \sum_{j=-nN/2}^{nN/2-1} f\left(\frac{2\pi j}{N}\right) e^{-2\pi i j\omega/N}.$$

Ahora muestreamos también el dominio de frecuencia tomando $\omega = \frac{k}{n}$, con índices en el mismo conjunto de antes, $k \in \{-nN/2, \dots, nN/2 - 1\}$, y tendremos:

$$\hat{f}\left(\frac{k}{n}\right) \approx \frac{2\pi}{N} \sum_{j=-nN/2}^{nN/2-1} f\left(\frac{2\pi j}{N}\right) e^{-2\pi i j k/nN} = \frac{2\pi}{N} \sum_{j=-nN/2}^{nN/2-1} f\left(\frac{2\pi j}{N}\right) \omega_{nN}^{jk} := \hat{a}_k \in \mathbb{C}, \tag{3.1}$$

donde $\omega_{nN} = e^{-2\pi i/nN}$ es la nN-ésima raíz primitiva de la unidad. Esta aproximación solo tiene sentido bajo dos circunstancias importantes:

- N es suficientemente grande, ya que entonces $e^{-i\omega x}$ no variará demasiado en $\left[\frac{2\pi j}{N}, \frac{2\pi(j+1)}{N}\right]$ y la regla del rectángulo tendrá una precisión razonable. Es decir, necesitaremos un gran número de datos para obtener una buena aproximación de la transformada de Fourier.
- $k \in \{-nN/2, \dots nN/2 - 1\}$. Esto ocurre ya que los $\hat{a}_k$ son nN-periódicos, $\hat{a}_k = \hat{a}_{k+nN}$, ya que $\omega_{nN}^{j(k+nN)} = \omega_{nN}^{jk}(\omega_{nN}^{nN})^j = \omega_{nN}^{jk}$. Sin embargo, aplicando el Lema 2.6, $\hat{f}(\omega) \to 0$ cuando $|\omega| \to \infty$, por lo que la aproximación deja de funcionar fuera del primer periodo.

Por lo tanto, a partir de nN valores de f muestreados uniformemente con una resolución temporal o intervalo de muestreo de $\Delta t = \frac{2\pi}{N}$, podemos aproximar nN valores de $\hat{f}$ con una resolución espectral de $\Delta f = \frac{1}{n}$, obteniendo así una aplicación $\mathbb{C}^{nN} \to \mathbb{C}^{nN}$. Para obtener una buena aproximación, necesitaremos tanto un n como un N suficientemente grandes. Este desarrollo se puede hacer igual considerando la definición alternativa (2.12). En ese caso, el intervalo de muestreo sería $\Delta t = \frac{1}{N}$ y la resolución espectral $\Delta f = \frac{1}{n}$. A lo largo de este capítulo desarrollaremos las propiedades de esta aplicación, que reflejarán las vistas para el caso continuo.

3.2. La transformada discreta de Fourier (DFT)

Definición 3.1. Dado un vector $a \in \mathbb{C}^N$, su *transformada discreta de Fourier de orden N*, $\mathrm{DFT}(N)$, es la aplicación lineal $\mathbb{C}^N \to \mathbb{C}^N$ definida como

$$\mathrm{DFT}(N) \equiv \hat{a}, \quad \text{con } \hat{a}_k = \sum_{j=0}^{N-1} a_j\, \omega_N^{jk}, \quad k = 0, 1, \dots N-1. \tag{3.2}$$

Definición 3.2. Llamamos *matriz de Fourier de orden* N, F_N, a la matriz asociada a la DFT(N), $\hat{a} = F_N a$, con expresión en coordenadas

$$F_N = (\omega_N^{jk})_{j,k=0}^{N-1} = \begin{pmatrix} 1 & 1 & \dots & 1 \\ 1 & \omega_N & \dots & \omega_N^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \dots & \omega_N \end{pmatrix}.$$

Ejemplo 3.3. Para $N = 4$, la matriz de Fourier tiene la siguiente forma:

$$F_4 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{pmatrix}.$$

Aplicando esta matriz sobre el vector $a = (1, 2, 3, 4) \in \mathbb{R}^4$, obtenemos su transformada discreta $\hat{a} = F_4 a = (10, -2 + 2i, -2, -2 - 2i)$.

Volviendo a la ecuación (3.1), desplazando los índices, tendremos que si $a_j = f\left(\frac{2\pi j}{N}\right)$, entonces

$$\hat{f}\left(\frac{k}{n}\right) \approx \frac{2\pi}{N}\hat{a}_k. \tag{3.3}$$

Existe un argumento más formal para la introducción de la transformada discreta que afirma que, generalizando la transformada de Fourier a la teoría de distribuciones, la DFT no es más que un caso particular de la transformada continua, véase ([3, Sección 6.2]). Por otro lado, la DFT tiene aplicaciones directas en el cálculo numérico de todo tipo de elementos del análisis de Fourier, tales como la aproximación de los coeficientes de series de Fourier o la interpolación de polinomios trigonométricos, véase ([16, Sección 3.1]).

A partir de la Definición 3.2, es inmediato ver que F_N es simétrica. Además, por las propiedades de las raíces de la unidad, solo tiene N valores distintos. Para $N > 2$, F_N no es Hermitiana, ya que no todos los elementos en su diagonal principal son reales.

Lema 3.4. El conjunto de N vectores exponenciales formando las columnas de F_N, $e_k = (\omega_N^{jk})_{j=0}^{N-1}$, $k = 0, \dots, N-1$, forma una base ortogonal de $\mathbb{C}^N$, con $\|e_k\|_2^2 = N$. La misma propiedad se cumple para el conjunto de vectores exponenciales conjugados $\overline{e_k}$, $k = 0 \dots N-1$.

Demostración. Como cada componente de e_k tiene norma 1, claramente $\|e_k\|_2^2 = N$. Ahora, dados $k, l \in \{0, \dots, N-1\}, k \neq l$, tenemos

$$\langle e_k, e_l \rangle = \sum_{j=0}^{N-1} \omega_N^{(l-k)j} = \sum_{j=0}^{N-1} e^{2\pi i(k-l)j/N} = \frac{1 - e^{2\pi i(k-l)}}{1 - e^{2\pi i(k-l)/N}} = 0,$$

y por lo tanto la base formada por los e_k es ortogonal. La prueba se realiza de forma análoga para los conjugados. □

Como consecuencia de este lema y de la simetría de F_N, tenemos que $F_N^T \overline{F_N} = NI_N$, donde I_N es la matriz identidad de orden N. Llegamos entonces al siguiente teorema de inversión de la DFT:

Teorema 3.5. La matriz de Fourier F_N es invertible y cumple que

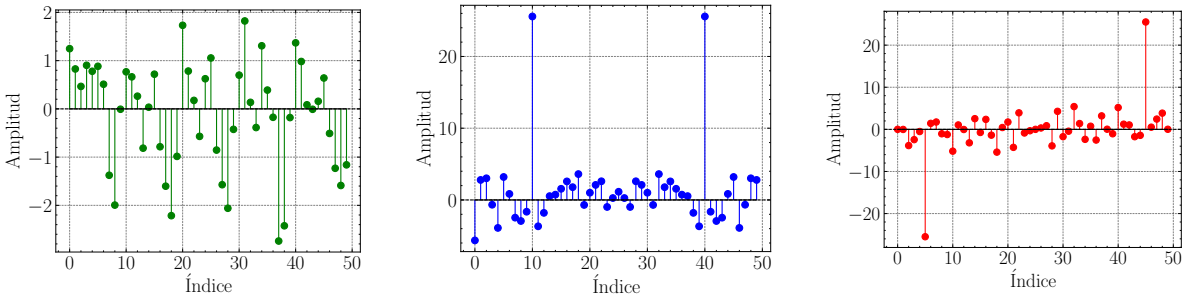
$$F_N^{-1} = \frac{1}{N}\overline{F_N} = \frac{1}{N}(\omega_N^{-jk})_{j,k=0}^{N-1}.$$

En resumen, podemos expresar cada una de las componentes la transformada discreta de Fourier mediante el siguiente par de ecuaciones:

$$\hat{a}_k = \langle a, \overline{e_k} \rangle = \sum_{j=0}^{N-1} a_j \, \omega_N^{jk}, \tag{3.4}$$

$$a_k = \frac{1}{N} \langle \hat{a}, e_k \rangle = \frac{1}{N} \sum_{j=0}^{N-1} \hat{a}_j \, \omega_N^{-jk}. \tag{3.5}$$

Ejemplo 3.6. Ahora visualizaremos la DFT de una señal con $N = 50$ muestras, siguiendo la función real $f(x) = \sin(2\pi x/5) + \cos(2\pi x/10) + h(x)$, donde h es una función de ruido desconocida. En la Figura 3.1 observamos como la DFT captura las 2 frecuencias dominantes en la señal, apareciendo el coseno, de frecuencia $10/2\pi$ en el índice 10 de la parte real de la DFT y el seno, de frecuencia $5/2\pi$, en el índice 5 de la parte imaginaria. El ruido de fondo también aparece en la DFT, pero como frecuencias mucho más bajas. Además, observamos simetría par en la parte real e impar en la parte imaginaria respecto al índice 25, es decir, el punto medio de la DFT. Más adelante, analizaremos más en detalle estas simetrías.



(a) Señal original muestreada (b) Parte real de la DFT (c) Parte imaginaria de la DFT

Figura 3.1: Ejemplo de la DFT de una señal con ruido desconocido.

Teorema 3.7. Sea P_N la matriz asociada a la permutación $\sigma = (0, N-1, \dots, 1)$,

$$P_N = (\delta_{(j+k) \bmod N})_{j,k=0}^{N-1} = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 0 & \dots & 1 \\ \vdots & \vdots & \iddots & 0 \\ 0 & 1 & \dots & 0 \end{pmatrix}.$$

Se tiene entonces que $F_N^2 = NP_N$, $F_N^4 = N^2 I_N$ y $F_N^{-1} = \frac{1}{N} P_N F_N$.

Demostración. Denotemos $F_N^2 = (c_{jl})_{j,l=0}^{N-1}$, entonces

$$c_{jl} = \sum_{k=0}^{N-1} \omega_N^{jk}\, \omega_N^{kl} = \sum_{k=0}^{N-1} e^{-2\pi i (j+l)k/N} = \begin{cases} N, & \text{si } j+l \equiv 0 \bmod N, \\ 0, & \text{si } j+l \not\equiv 0 \bmod N \end{cases} = N\delta_{(j+l) \bmod N}.$$

Por tanto, $F_N^2 = NP_N$, y como $P_N^2 = I_N$, entonces

$$F_N^4 = F_N^2 F_N^2 = (NP_N)(NP_N) = N^2 (P_N)^2 = N^2 I_N.$$

Finalmente, utilizando las dos expresiones anteriores, obtenemos

$$F_N^{-1} = \frac{1}{N^2} F_N^3 = \frac{1}{N} P_N F_N.$$

□

Observación 3.8. El teorema previo es muy útil en la práctica. Como la reordenación y división $\frac{1}{N} P_N$ tienen una implementación trivial, se puede reutilizar un algoritmo de cálculo de DFT para la transformada inversa.

Una segunda forma de ver la transformada discreta de Fourier es a través de la evaluación de un polinomio $p(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{N-1} x^{N-1} \in \mathbb{C}[x]$ en las raíces de la unidad. En concreto, dado un $k = 0, 1, \dots N-1$, se tiene $p(\omega_N^k) = \hat{a}_k$.

Definición 3.9. La *convolución cíclica* de dos vectores $a, b \in \mathbb{C}^N$ es el vector $c = (c_n)_{n=0}^{N-1} = a * b \in \mathbb{C}^N$ con componentes:

$$c_n = \sum_{k=0}^{N-1} a_k b_{(n-k) \bmod N} = \sum_{k=0}^{n} a_k b_{n-k} + \sum_{k=n+1}^{N-1} a_k b_{N+n-k}, \quad n = 0, \dots, N-1. \tag{3.6}$$

Una forma alternativa de definir esta convolución es considerar a y b como sucesiones con periodo N, y tomar cada c_n sin necesidad del operador módulo, $c_n = \sum_{k=0}^{N-1} a_k b_{n-k}$.

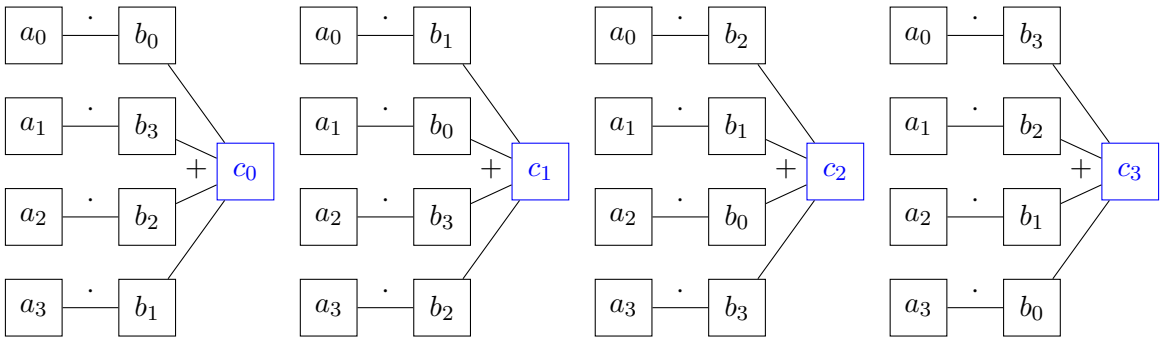


Figura 3.2: Visualización de la convolución cíclica $a * b = c$ en el caso $N = 4$.

En la Figura 3.2 podemos observar cómo funciona esta operación, en la que el vector b rota para producir los elementos de c.

Denotaremos también como $a \circ b = (a_k b_k)_{k=0}^{N-1}$ al vector obtenido multiplicando a y b componente a componente. A partir de estas operaciones, podemos dar una versión discreta del teorema de convolución.

Teorema 3.10 (de convolución discreta)**.** Dados $a, b \in \mathbb{C}^N$ se tiene:

$$\mathrm{DFT}(N)(a * b) = \hat{a} \circ \hat{b}, \quad \mathrm{DFT}(N)(a \circ b) = \frac{1}{N}(\hat{a} * \hat{b}). \tag{3.7}$$

Demostración. Sea $c = a * b$, comenzaremos calculando las componentes de su transformada discreta $\hat{c} = (\hat{c}_k)_{k=0}^{N-1}$:

$$\begin{aligned}\hat{c}_k &= \sum_{j=0}^{N-1} \left(\sum_{n=0}^{N-1} a_n b_{((j-n) \bmod N)} \right) \omega_N^{jk} = \sum_{n=0}^{N-1} a_n \, \omega_N^{nk} \left(\sum_{j=0}^{N-1} b_{((j-n) \bmod N)} \, \omega_N^{((j-n) \bmod N)k} \right) \\ &= \left(\sum_{n=0}^{N-1} a_n \, \omega_N^{nk} \right) \hat{b}_k = \hat{a}_k \hat{b}_k.\end{aligned}$$

Por lo tanto $\hat{c} = \hat{a} \circ \hat{b}$. Para la otra igualdad, primero obtenemos los componentes de a y b utilizando el Teorema 3.5:

$$a_j = \frac{1}{N} \sum_{k=0}^{N-1} \hat{a}_k \, \omega_N^{-jk}, \quad b_j = \frac{1}{N} \sum_{l=0}^{N-1} \hat{b}_l \, \omega_N^{-jl}, \quad \text{para } j = 0, \ldots, N-1.$$

Ahora, denotando $d = a \circ b$, tenemos que

$$\begin{aligned}d_j = a_j b_j &= \frac{1}{N^2} \left(\sum_{k=0}^{N-1} \hat{a}_k \, \omega_N^{-jk} \right) \left(\sum_{l=0}^{N-1} \hat{b}_l \, \omega_N^{-jl} \right) = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} \hat{a}_k \hat{b}_l \, \omega_N^{-j(k+l)} \\ &= \frac{1}{N^2} \sum_{n=0}^{N-1} \left(\sum_{k=0}^{N-1} \hat{a}_k \hat{b}_{((n-k) \bmod N)} \right) \omega_N^{-jn},\end{aligned}$$

y por lo tanto, $d = \frac{1}{N} F_N^{-1}(\hat{a} * \hat{b})$ y entonces $N\hat{d} = \hat{a} * \hat{b}$. □

Definición 3.11. Sean los vectores $a, b \in \mathbb{C}^N$, que extendemos con ceros hasta formar $c, d \in \mathbb{C}^{2N-1}$, con $c_i = a_i$ y $d_i = b_i$, para $i < N$ y $c_i = d_i = 0$, para $i \geq N$. Llamamos *convolución acíclica* de a y b, a la convolución cíclica de c y d:

$$(c * d)_n = \sum_{k=0}^{N-1} c_k d_{(n-k) \bmod N} = \sum_{k=0}^{n} a_k b_{n-k}, \quad n = 0, \ldots, 2N-1. \tag{3.8}$$

Volviendo a la interpretación polinómica de la transformada, si tenemos $p, q \in \mathbb{C}[x]$ de grado $N \in \mathbb{N}$ con vectores de coeficientes $a, b \in \mathbb{C}^N$, entonces los coeficientes de su producto $h = pq$ son los de la convolución acíclica de a y b.

$$p(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1} \Longrightarrow c = (a_0, a_1, \ldots, a_{n-1}, 0, \ldots, 0) \in \mathbb{C}^{2N-1},$$
$$q(x) = b_0 + b_1 x + \cdots + b_{n-1}x^{n-1} \Longrightarrow d = (b_0, b_1, \ldots, b_{n-1}, 0, \ldots, 0) \in \mathbb{C}^{2N-1}.$$

De esta forma, podemos aplicar el Teorema 3.10 y calcular h mediante la multiplicación punto a punto, $\hat{c} \circ \hat{d}$. Más adelante, analizaremos la complejidad computacional de este método con respecto al algoritmo trivial de multiplicación de polinomios. Nótese que es sencillo extender todo esto a polinomios de grados diferentes N_1 y N_2. En ese caso, tan solo necesitamos extender hasta $N_1 + N_2 - 1$ sus vectores de coeficientes.

Ejemplo 3.12. Sean $p(x) = x^2 + 2x + 1$ y $q(x) = 3x^3 + 5x^2 + 2$, con vectores de coeficientes $a = (1, 2, 1)$ y $b = (2, 0, 5, 3)$. Tomamos $N = 4$ como el mayor grado de los polinomios y para calcular su convolución acíclica, ampliamos con ceros hasta tener vectores de longitud $2N$[1], $c = (1, 2, 1, 0, 0, 0, 0, 0)$, $d = (2, 0, 5, 3, 0, 0, 0, 0)$. Ahora calculamos sus respectivas DFTs y el producto elemento a elemento $\hat{c} \circ \hat{d}$:

$$\hat{c} \approx (4, 2.41 - 2.41i, -2i, -0.41 - 0.41i, 0, -0.41 + 0.41i, 2i, 2.41 + 2.41i),$$
$$\hat{d} \approx (10, -0.12 - 7.12i, -3 + 3i, 4.12 + 2.88i, 4, 4.12 - 2.88i, -3 - 3i, -0.12 + 7.12i),$$
$$\hat{c} \circ \hat{d} \approx (40, -17.49 - 16.9i, 6 + 6i, -0.51 - 2.9i, 0, -0.51 + 2.9i, 6. - 6i, -17.49 + 16.9i).$$

Tomando la transformada inversa de $\hat{c} \circ \hat{d}$ obtenemos

$$\mathrm{DFT}^{-1}(8)(\hat{c} \circ \hat{d}) = (2, 4, 7, 13, 11, 3, 0, 0) = c * d,$$

efectivamente, los elementos de este vector se corresponden con los coeficidentes del polinomio producto $h(x) = p(x)q(x) = 3x^5 + 11x^4 + 13x^3 + 7x^2 + 4x + 2$.

A continuación, discutimos brevemente las propiedades de simetría de la DFT que ya hemos observado en algunos de los ejemplos anteriores.

[1]También sirve $2N - 1$, tomamos aquí $2N = 8$ porque así los resultados intermedios son algo más sencillos y, como veremos en el siguiente capítulo, las DFTs de longitud 2^d, $d \in \mathbb{N}$, tienen un buen comportamiento.

Definición 3.13. Un vector $a \in \mathbb{C}^N$ es *par* si $a = P_N a$, esto es, $a_j = a_{N-j}$ para $j = 1, \ldots, N-1$, y es *impar* si $a = -P_N a$, es decir, $a_0 = 0$ y $a_j = -a_{N-j}$ para $j = 1, \ldots, N-1$.

Cuando el vector de entrada a es real, par o impar, tenemos ciertas simetrías en $\hat{a}$. La siguiente proposición, con demostración en ([16, Corol. 3.27 y Corol. 3.28]), las resume.

Proposición 3.14. Sea $a \in \mathbb{R}^N$, entonces $\overline{\hat{a}_j} = \hat{a}_{N-j}$ para todo $j = 1, \ldots N-1$. Esto es, Re $\hat{a}$ es par e Im $\hat{a}$ es impar. Si además a es par, entonces $\hat{a} \in \mathbb{R}^N$ y es par, y si a es impar, entonces $\hat{a} = i$ Im $\hat{a} \in i\mathbb{R}^n$ y es impar. En cualquier caso, la información en los coeficientes $\hat{a}_{\lceil N/2 \rceil + 1}, \ldots, \hat{a}_{N-1}$ se puede obtener a partir de posiciones previas.

Ejemplo 3.15 (Vector de entrada real)**.** Sea $a = (0, 1, 2, 3, 4, 5, 6, 7) \in \mathbb{R}^8$, entonces su DFT será $\hat{a} \approx (28, -4 + 9.65i, -4 + 4i, -4 + 1.65i, -4, -4 - 1.65i, -4 - 4i, -4 - 9.65i)$. Se cumple que $\hat{a}_0, \hat{a}_4 \in \mathbb{R}$ ya que, por definición, $\hat{a}_0$ no es más la suma de los valores de a y por la anterior proposición $\hat{a}_4 = \overline{\hat{a}_4}$. Para los demás índices, también se cumple la primera parte de la proposición.

Ejemplo 3.16 (Vector de entrada real y par)**.** Sea $a = (0, 1, 2, 3, 4, 3, 2, 1) \in \mathbb{R}^8$ y con simetría par, entonces $\hat{a} \approx (16, -6.82, 0, -1.17, 0, -1.17, 0, -6.82)$. Se cumple que $\hat{a} \in \mathbb{R}^8$ y además $\hat{a}$ es par.

3.3. DFTs bidimensionales

De forma similar al caso continuo, podemos ampliar el dominio de las transformadas discretas de Fourier al espacio multidimensional. En esta sección, trabajaremos en 2 dimensiones, y veremos como su cálculo se puede reducir al unidimensional. Más allá de su relevancia teórica, estas transformadas tienen aplicaciones cruciales en el procesamiento digital de imágenes. Introduciremos aquí los conceptos fundamentales y unas primeras visualizaciones, que serán de utilidad en el capítulo de aplicaciones.

Definición 3.17. Sean $N_1, N_2 \in \mathbb{N} \setminus \{1\}$ e $I_{N_j} = \{0, \ldots, N_j - 1\}$, $j = 1, 2$, los respectivos conjuntos de índices. La *transformada discreta de Fourier bidimensional*, $\mathrm{DFT}(N_1, N_2)$, es la aplicación lineal $\mathbb{C}^{N_1 \times N_2} \to \mathbb{C}^{N_1 \times N_2}$ que lleva una matriz $A \in \mathbb{C}^{N_1 \times N_2}$, con coeficientes $(a_{k_1,k_2})_{k_1,k_2=0}^{N_1-1,N_2-1}$, a la matriz $\hat{A} = F_{N_1} A F_{N_2} \in \mathbb{C}^{N_1 \times N_2}$, con coeficientes

$$\hat{a}_{n_1,n_2} = \sum_{k_1=0}^{N_1-1} \sum_{k_2=0}^{N_2-1} a_{k_1,k_2}\, \omega_{N_1}^{k_1 n_1}\, \omega_{N_2}^{k_2 n_2}, \quad n_j \in I_{N_j},\ j = 1, 2. \tag{3.9}$$

Las propiedades de la transformada bidimensional son similares al caso unidimensional ([16, Sección 4.5.2.]), y no las enunciaremos en detalle. También podemos extender la expresión de

los coeficientes en (3.9) a todo $\mathbb{Z}^2$, obteniendo periodicidad en ambas dimensiones $\hat{a}_{n_1,n_2} = \hat{a}_{n_1+l_1N_1,n_2+l_2N_2}$, con $l_j \in \mathbb{Z}$ y $n_j \in I_{N_j}$, $j = 1, 2$.

En el Capítulo 4, desarrollaremos métodos rápidos para el cálculo de DFT(N). Es interesante reducir el caso de la transformada bidimensional al cálculo de varias transformadas unidimensionales, debido a que de esta forma se reduce la complejidad y número de algoritmos a desarrollar. Para ello, desarrollamos el llamado *método de fila-columna* factorizando la expresión de la DFT(N_1, N_2), véase ([19, Sección 5.1]),

$$\hat{a}_{n_1,n_2} = \sum_{k_1=0}^{N_1-1} \sum_{k_2=0}^{N_2-1} a_{k_1,k_2}\, \omega_{N_1}^{k_1 n_1}\, \omega_{N_2}^{k_2 n_2} = \sum_{k_2=0}^{N_2-1} \left(\sum_{k_1=0}^{N_1-1} a_{k_1,k_2}\, \omega_{N_1}^{k_1 n_1} \right) \omega_{N_2}^{k_2 n_2}.$$

Observemos que la suma interna no es más que una DFT(N_1) sobre cada columna de A. Denotaremos cada una de estas DFTs como b_{n_1}, con $n_1 \in I_{N_1}$. Una vez calculadas, podemos hallar $\hat{a}_{n_1,n_2}$ realizando una DFT(N_2) sobre cada una de las filas de $B = (b_0|b_1|\dots|b_{N_2-1})$,

$$\hat{a}_{n_1,n_2} = \sum_{k_2=0}^{N_2-1} b_{n_1,k_2}\, \omega_{N_2}^{k_2 n_2}.$$

De esta forma, para el cálculo completo de la matriz $\hat{A}$ necesitaremos realizar N_2 DFTs de orden N_1 y N_1 DFTs de orden N_2.

Ejemplo 3.18. Consideremos la siguiente matriz $A \in \mathbb{C}^{4\times 4}$, cuyos coeficientes aumentan linealmente en las columnas 1 y 3 y siguen un patrón en forma de coseno en cada fila:

$$A = \begin{pmatrix} 1 & 0 & -1 & 0 \\ 2 & 0 & -2 & 0 \\ 3 & 0 & -3 & 0 \\ 4 & 0 & -4 & 0 \end{pmatrix} \in \mathbb{C}^{4\times 4}.$$

Hallamos la matriz B aplicando sobre cada columna una DFT(4). Por ejemplo, para la primera columna tenemos $b_0 = \text{DFT}(4)(1, 2, 3, 4) = (10, -2 + 2i, -2, -2 - 2i)$. Finalmente, podemos obtener $\hat{A}$ realizando una DFT(4) sobre cada fila de B:

$$B = \begin{pmatrix} 10 & 0 & -10 & 0 \\ -2+2i & 0 & 2-2i & 0 \\ -2 & 0 & 2 & 0 \\ -2-2i & 0 & 2+2i & 0 \end{pmatrix} \longrightarrow \hat{A} = \begin{pmatrix} 0 & 20 & 0 & 20 \\ 0 & -4+4i & 0 & -4+4i \\ 0 & -4 & 0 & -4 \\ 0 & -4-4i & 0 & -4-4i \end{pmatrix}.$$

Visualización de DFTs bidimensionales

Una de las áreas de aplicación más importantes de la transformada de Fourier es el procesamiento de imágenes digitales. Simplificando masivamente, una cámara digital no es más que un mecanismo para proyectar el espacio tridimensional sobre un sensor plano. Este sensor muestrea la intensidad de la luz en cada punto proyectado. Se obtiene así una matriz de píxeles A midiendo la intensidad y el color de cada muestra. Por simplicidad, en esta sección nos centraremos en imágenes monocromáticas de 8 bits, donde $a_{k_1,k_2} \in \{0, 1, \ldots, 255\}$, representando 0 el negro puro y 255 el blanco puro. En las siguientes imágenes, los valores de la entrada y salida de las DFTs bidimensionales son representadas normalizando y discretizando su módulo en $\{0, 1, \ldots, 255\}$.

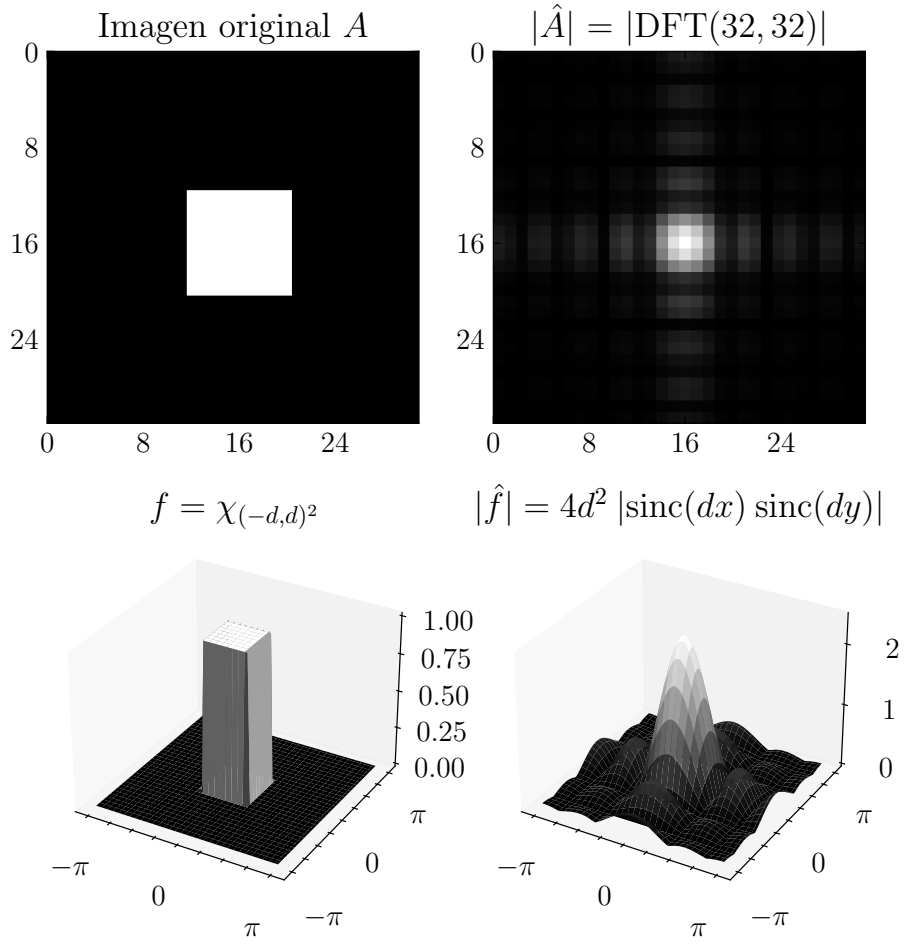


Figura 3.3: Versiones discretas y continuas de la función característica y su transformada. El espacio continuo es $[-\pi, \pi]$, que trasladamos a $[0, 2\pi]$ y muestreamos uniformemente con $N = 32$. En este caso definimos el radio del cuadrado de la imagen como $r = 4$, entonces $d = \frac{2r\pi}{N} = \frac{\pi}{4}$.

En la Figura 3.3, representamos la DFT de un rectángulo blanco sobre fondo negro. Tene-

mos que la matriz de entrada A es par en cada dimensión (ya que la imagen está centrada en $N/2, N/2$), y además es real. Mediante una generalización directa de la Proposicion 3.14, tendremos que su DFT es real y simétrica par en cada eje. Desde el punto de vista continuo, estamos aproximando la transformada continua de la función característica $f = \chi^2_{(-d,d)}$. Como vimos en el Ejemplo 2.14, $\hat{f}(\omega_1, \omega_2) = 4d^2 \operatorname{sinc}(d\omega_1) \operatorname{sinc}(d\omega_2)$. Efectivamente, al igual que la DFT, $\hat{f}$ es real y par en cada dimensión[2]. Como se puede observar, las gráficas de los módulos de f y $\hat{f}$ se ven aproximadas por la imagen del rectángulo y su DFT, respectivamente.

En la Figura 3.4 repetimos esta visualización sobre la conocida imagen *cameraman* ($N = 256$). Para poder distinguir mejor los cambios en el espectro de frecuencia, aplicamos un logaritmo sobre la magnitud de $\hat{A}$. Observamos así que ciertos patrones de la imagen se reflejan en su transformada discreta. Los componentes de baja frecuencia, es decir, las grandes áreas de color uniforme como el fondo o el cuerpo del fotógrafo, se transforman en la región central de la DFT, que es de mayor magnitud. Por otro lado, los componentes de alta frecuencia, asociados a los bordes y texturas de los objetos, aparecen como regiones brillantes alejadas del centro de la imagen espectral.

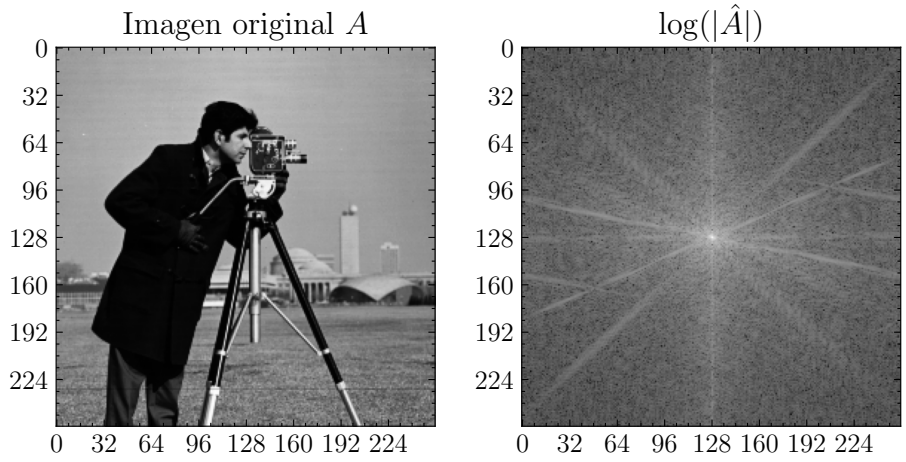


Figura 3.4: Imagen *cameraman* junto al logaritmo del módulo de su DFT.

Finalmente, describimos cómo obtener una transformada con impulsos aislados. Desde el punto de vista discreto, podemos generarlos creando un patrón periódico que muestree una función periódica real, por ejemplo, $f(x_1, x_2) = \cos(ax_1 + bx_2)$, con $a, b \in \mathbb{R}$.

La interpretación continua de este caso es más delicada. La transformada de Fourier se puede extender a las llamadas *distribuciones temperadas*, entre las cuales se encuentra la función δ de Dirac, ya mencionada en el primer capítulo. Aunque un tratamiento formal de estas distribu-

[2]Esto justifica que llamemos *simetría par* a la propiedad $a_j = a_{N-j}$ para $j \geq 1$ (ídem. para la *simetría impar*).

ciones, como el realizado en ([16, Capítulo 4]), se escapa de los objetivos de este trabajo, es ilustrativo mencionarlas. Resulta que funciones periódicas como las sumas de cosenos y senos (o bien finitas, o bien series de Fourier), a pesar de no ser de L^1, tienen transformadas en forma de sumas de distribuciones δ. La interpretación de este hecho es sencilla, estas funciones ya se encuentran en su forma descompuesta en términos de frecuencias, por lo que al calcular la integral de Fourier tendremos divergencia en las frecuencias presentes, mientras que el valor será 0 en las no presentes.

Consideremos, por ejemplo $f(x_1, x_2) = \cos(x_1 + x_2) = \frac{1}{2}\left(e^{i(x_1+x_2)} + e^{-i(x_1+x_2)}\right)$, en $A = [-\pi, \pi]^2$. Podemos entonces evaluar el integrando de la integral de Fourier como

$$e^{-i(x_1\omega_1+x_2\omega_2)} f(x_1, x_2) = \frac{1}{2}\left(e^{-ix_1(\omega_1-1)} e^{-ix_2(\omega_2-1)}\right) + \frac{1}{2}\left(e^{-ix_1(\omega_1+1)} e^{-ix_2(\omega_2+1)}\right),$$

y recordando que $\int_{[-\pi,\pi]} e^{-ixk}\, dx$ se anula si $k = 0$ y es 2π si $k = 0$, llegamos a que $\hat{f}(\omega_1, \omega_2)$ se anula si $(\omega_1, \omega_2) \notin \{(-1,-1), (1,1)\}$ y diverge si $(\omega_1, \omega_2) \in \{(-1,-1), (1,1)\}$. Por lo tanto, $\hat{f}(\omega_1, \omega_2) = \delta(\omega_1 + 1, \omega_2 + 1) + \delta(\omega_1 - 1, \omega_2 - 1)$.

En la Figura 3.5 podemos apreciar que en el entorno de estos puntos, tras la transformación de coordenadas, la DFT presenta impulsos aislados. Si aumentáramos la resolución N, obtendríamos una mayor concentración alrededor de esos puntos.

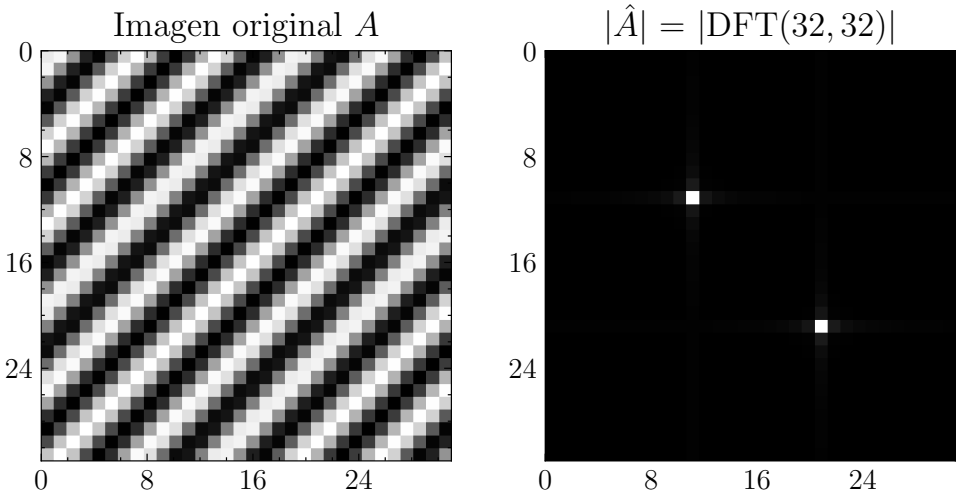


Figura 3.5: DFT de una imagen muestreando $f(x) = \cos(x_1 + x_2)$.

Se pueden encontrar muchos más ejemplos de este tipo de visualización en ([2, Fig. 13.3]). En el Capítulo 5, se expondrá como estas ideas, junto a un paso de cuantización, conduce al algoritmo de compresión de imágenes fotográficas usado en el formato *JPEG*.

Capítulo 4

La transformada rápida de Fourier

Como hemos visto, la DFT es una de las herramientas más potentes en el Análisis de Fourier, permitiéndonos materializar transformadas de Fourier de funciones desconocidas, lo cual desemboca en un amplio rango de aplicaciones en el procesamiento de señales e imágenes, la combinatoria, el análisis de EDPs, y muchos otros campos. Sin embargo, existe un grave inconveniente a la hora de calcular DFTs: la gran cantidad de datos necesaria.

Dependiendo del caso de uso, el tamaño necesario cambiará, pero si seguimos con el ejemplo de procesamiento de imágenes y buscamos calcular la DFT de una imagen en alta resolución (1920×1080), entonces necesitaremos realizar, utilizando el método de fila-columna, 1080 DFTs de orden 1920 y 1920 DFTs de orden 1080. Al tratarse de un producto matriz-vector, el cálculo directo de una DFT(N) requiere N^2 multiplicaciones y $N(N-1)$ sumas complejas. Por lo tanto, necesitaremos realizar, aproximadamente, 10^{10} operaciones.

Aunque los procesadores actuales son extremadamente potentes y podrían acabar este cálculo en un tiempo razonable, sería inviable poder realizar un gran número de DFTs por segundo o calcular DFTs con un orden de magnitud mayor de muestras, lo cual es indispensable en otras aplicaciones como en el filtrado y procesamiento de audio o vídeo. La siguiente definición es fundamental para razonar sobre el tiempo de ejecución de un algoritmo.

Definición 4.1. Sea una función $T : \mathbb{N} \to \mathbb{N}$ que mide el número de operaciones [1] que necesita un algoritmo dependiente de una variable N para completar su ejecución. Decimos que $T(N) \in \mathcal{O}(f(N))$, o que el algoritmo tiene un *orden de complejidad acotado superiormente por* $f(N)$, si existe una función $f : \mathbb{N} \to \mathbb{N}$ y unas constantes $c \in \mathbb{N}$, $N_0 \in \mathbb{N}$ tales que

$$0 \leq T(N) \leq cf(N), \quad \text{para todo } N \geq N_0.$$

[1] El significado exacto de *operación* puede variar dependiendo del contexto. En este trabajo nos referiremos a operaciones aritméticas complejas (tanto sumas como multiplicaciones).

Si además tenemos una cota asintótica inferior, es decir, existe otra constante $d \in \mathbb{N}$ tal que $0 \leq d\, f(N) \leq T(N) \leq c\, f(N)$, para todo $N \geq N_0$, entonces decimos que $T(N) \in \Theta(f(N))$, o que el algoritmo tiene un *orden de complejidad* $f(N)$. Claramente, $\mathcal{O}(f(N)) \subset \Theta(f(N))$.

Como una DFT(N) requiere $N^2 + N(N-1)$ operaciones, es evidente que DFT$(N) \in \Theta(N^2)$. Generalmente, para un producto matriz-vector donde no tenemos mayor información sobre los N^2 coeficientes de la matriz, no es posible mejorar ese orden de complejidad. Sin embargo, para la DFT, podemos aprovecharnos de una propiedad fundamental: tan solo existen N valores distintos en la matriz F_N. Este hecho, junto a otras simetrías presentes en su definición, nos darán la clave para crear algoritmos rápidos que reduzcan el orden de complejidad de la DFT. Podemos introducir ahora la definición que le da nombre a este trabajo.

Definición 4.2. Una *transformada rápida de Fourier* (FFT) de orden N es un algoritmo que permite calcular la transformada discreta de Fourier sobre cualquier vector $a \in \mathbb{C}^N$ manteniendo un orden de complejidad *cuasilineal*, $\mathcal{O}(N \log N)$.

Nótese que es innecesario especificar la base del logaritmo en $\mathcal{O}(N \log N)$, ya que el cambio de base solo genera una constante. A lo largo de este capítulo describiremos varias FFTs, empezando por los algoritmos más sencillos y comúnes en la práctica. Como referencias principales, utilizaremos el Capítulo 5 de ([16]), que presenta un desarrollo formal y preciso de diversas FFTs, así como el Capítulo 3 de ([19]), que contiene análisis detallados para valores de N bajos y los Capítulos 10-13 de ([3]). Para los aspectos computacionales y un punto de vista más práctico, hacemos referencia a ([6]).

4.1. FFTs en base 2

Comenzamos con el caso $N = 2^d$, $d \in \mathbb{N} \cup \{0\}$, el cual, como veremos, es el más sencillo de tratar y uno de los más convenientes para el cálculo en un ordenador, al permitir representar cada uno de los índices $k \in \{0, 1, \ldots, N-1\}$ como un número binario de d bits.

4.1.1. El algoritmo de Cooley-Tukey

Uno de los primeros algoritmos publicados para el cálculo rápido de una DFT, y que continúa siendo el más conocido y usado en la práctica, es el algoritmo de Cooley-Tukey ([5]). Este método se basa en el patrón conocido como *divide y vencerás*, en el que una tarea se descompone en subtareas más pequeñas que, junto a un paso de combinación suficientemente rápido, permiten acelerar la ejecución de un algoritmo. En este caso, buscaremos descomponer una DFT(N) en

varias DFTs más pequeñas. Para llegar a esto, podemos manipular la expresión en forma de suma (3.2). Dado un $k \in \{0, 1, \ldots, N-1\}$, tenemos

$$\hat{a}_k = \sum_{j=0}^{N-1} a_j\, \omega_N^{jk} = \sum_{j \text{ par}} a_j\, \omega_N^{jk} + \sum_{j \text{ impar}} a_j\, \omega_N^{jk} = \sum_{r=0}^{N/2-1} a_{2r}\, \omega_N^{2rk} + \omega_N^k \sum_{r=0}^{N/2-1} a_{2r+1}\, \omega_N^{2rk}.$$

Además, como $\omega_N^{2rk} = e^{\frac{-4\pi irk}{N}} = e^{\frac{-2\pi irk}{N/2}} = \omega_{N/2}^{rk}$, entonces

$$\hat{a}_k = \sum_{r=0}^{N/2-1} a_{2r}\, \omega_{N/2}^{rk} + \omega_N^k \sum_{r=0}^{N/2-1} a_{2r+1}\, \omega_{N/2}^{rk}, \quad k = 0, 1, \ldots, N-1. \tag{4.1}$$

En la ecuación previa, observamos que ambos sumandos no son más que DFTs de tamaño $N/2 = 2^{d-1}$ sobre los vectores formados por los índices pares e impares $b = (a_0, a_2, \ldots, a_{N-2})$ y $c = (a_1, a_3, \ldots, a_{N-1})$. Finalmente, como $\omega_N^{N/2} = -1$, entonces $\omega_N^k = -\omega_N^{k-N/2}$ cuando $k \geq N/2$. De esta forma, llegamos a la siguiente descomposición de la DFT(N) en dos DFT($N/2$):

$$\hat{a}_k = \begin{cases} \hat{b}_k + \omega_N^k\, \hat{c}_k, & \text{si } k < N/2, \\ \hat{b}_k - \omega_N^k\, \hat{c}_k, & \text{si } k \geq N/2. \end{cases} \tag{4.2}$$

Con este primer paso de subdivisión conseguimos reducir el número de productos complejos de N^2 a $N^2/2 + N/2$, donde los $N^2/2$ primeros corresponden al cálculo de ambas DFT($N/2$) y los otros $N/2$ provienen del producto con los *factores de giro* ω_N^k. Aunque ahorramos unas cuantas operaciones, se mantiene el orden de complejidad $\mathcal{O}(N^2)$. Sin embargo, podemos seguir subdiviendo el cálculo de los vectores $\hat{b}$ y $\hat{c}$ con el mismo método hasta llegar al caso base $N = 1$, donde la DFT no es más que la identidad. Llegamos entonces a un proceso recursivo descrito en el Algoritmo 1, conocido como el algoritmo de Cooley-Tukey, que expresamos en forma de pseudocódigo.

Existen formas alternativas de llegar a este y otros muchos algoritmos rápidos para la DFT. En este caso, hemos usado la expresión en forma de suma, pero también se puede proceder mediante la interpretación polinómica o la factorización de la matriz de Fourier F_N en *matrices sparse*. Esta última interpretación es más técnica y la omitiremos en este trabajo, para no tener que introducir nuevos conceptos. Sin embargo, sí usaremos la interpretación polinómica más adelante. En la Figura 4.1 observamos el proceso de subdivisión del vector de entrada a para el caso $N = 8$. Cada celda en el nivel l, con $0 \leq l \leq d$, contiene los valores pasados a cada una de las $DFT(2^{d-l})$ construidas en ese nivel.

Más adelante, comprobaremos que el algoritmo de Cooley-Tukey tiene un orden de complejidad $\Theta(N \log N)$, por lo que, efectivamente, es una FFT. El número exacto de operaciones

Algoritmo 1 Algoritmo de Cooley-Tukey, versión recursiva

Entrada: $a = (a_0, a_1, \ldots, a_{N-1}) \in \mathbb{C}^N$: vector de entrada, N tamaño de los datos.
Salida: $\hat{a} = (\hat{a}_0, \hat{a}_1, \ldots, \hat{a}_{N-1}) \in \mathbb{C}^N$: DFT($N$) de a.

procedure COOLEY-TUKEY-FFT(a, N)
  if $N = 1$ **then return** a
  end if
  $factorgiro \leftarrow 1$
  $\hat{b} \leftarrow$ COOLEY-TUKEY-FFT($(a_0, a_2, \ldots a_{N-2}), N/2$)
  $\hat{c} \leftarrow$ COOLEY-TUKEY-FFT($(a_1, a_3, \ldots, a_{N-1}), N/2$)
  for $k \leftarrow 0 \ldots N/2 - 1$ **do**
    $\hat{a}_k \leftarrow \hat{b}_k + factorgiro \cdot \hat{c}_k$
    $\hat{a}_{k+N/2} \leftarrow \hat{b}_k - factorgiro \cdot \hat{c}_k$
    $factorgiro \leftarrow factorgiro \cdot \omega_N$
  end for
  return $\hat{a}$
end procedure

complejas requeridas se puede consultar en ([16]), y se expresa en función de logaritmos en base 2, reflejando la subdivisión recursiva de las DFTs en 2 partes.

El aumento de rendimiento al pasar de la DFT trivial, cuadrática, a la FFT de Cooley-Tukey, *cuasilineal*, es enorme. Para el anterior ejemplo en el que calculamos una DFT(1920×1080), necesitaremos alrededor de $1920 \cdot 1080 \log 1080 + 1080 \cdot 1920 \log 1920 \approx 10^7$ operaciones complejas, por lo que obtenemos una mejora de 3 órdenes de magnitud.

En cuanto al cálculo rápido de la transformada discreta inversa, podemos proceder de dos formas: o bien cambiar los factores de giro ω_N^k por ω_N^{-k} y dividir los valores obtenidos por cada DFT entre 2, o bien emplear la Observación 3.8 y tan solo ejecutar el Algoritmo 1 tras dividir entre N y reordenar el vector $\hat{a}$.

4.1.2. Decimación en tiempo (DIT) y en frecuencia (DIF)

Ahora analicemos qué frecuencias hemos recuperado en cada paso realizado por el algoritmo. Utilizando el convenio (2.12) para la transformada y siguiendo el análisis realizado en la Sección 3.1., si la frecuencia de muestreo de la señal es $f_s = \frac{1}{T}$ y el número de muestras es N, entonces la frecuencia espectral obtenida será $f_\omega = N f_s = \frac{1}{NT}$. En la Tabla 4.1 podemos observar qué ocurre en los primeros niveles del algoritmo de Cooley-Tukey.

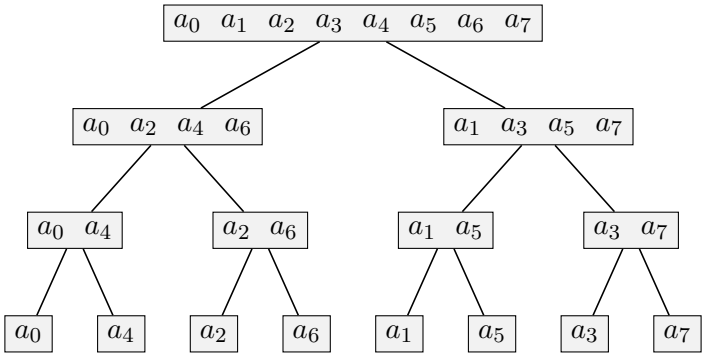


Figura 4.1: Árbol de ejecución para el algoritmo de Cooley-Tukey, con $N = 8$.

Tabla 4.1: Frecuencias temporales y espectrales por nivel en el algoritmo de Cooley-Tukey.

Nivel	Ejemplo de entrada	Muestras	f_s	f_ω
0	$\{a_0, a_1, \ldots, a_{N-1}\}$	N	$1/T$	$1/NT$
1	$\{a_0, a_2, \ldots, a_{N-2}\}$	$N/2$	$1/2T$	$1/NT$
2	$\{a_0, a_4, \ldots, a_{N-4}\}$	$N/4$	$1/4T$	$1/NT$
3	$\{a_0, a_8, \ldots, a_{N-8}\}$	$N/8$	$1/8T$	$1/NT$

Como vemos, la frecuencia temporal se divide en dos en cada paso realizado, mientras que la frecuencia espectral se mantiene. Este hecho hace que el algoritmo de Cooley-Tukey se clasifique como algoritmo de *decimación en tiempo* (DIT-FFT). Cada algoritmo de decimación en tiempo tiene un algoritmo de *decimación en frecuencia* (DIF-FFT) dual, en el que se mantiene la frecuencia temporal mientras se dobla la frecuencia espectral obtenida en cada paso. Para obtener la versión DIF del algoritmo de Cooley-Tukey, procedemos de una forma similar:

$$\begin{aligned}\hat{a}_k &= \sum_{j=0}^{N-1} a_j\, \omega_N^{jk} = \sum_{j=0}^{N/2-1} a_j\, \omega_N^{jk} + \sum_{j=N/2}^{N-1} a_j\, \omega_N^{jk} = \sum_{r=0}^{N/2-1} a_r\, \omega_N^{rk} + (-1)^k \sum_{r=0}^{N/2-1} a_{r+N/2}\, \omega_N^{rk} \\ &= \sum_{r=0}^{N/2-1} (a_r + (-1)^k a_{N/2+r})\, \omega_N^{rk}, \quad k = 0, 1, \ldots, N-1.\end{aligned}$$

Ahora, definiendo los vectores $b = (a_r + a_{N/2+r})_{r=0}^{N/2-1}$ y $c = ((a_r - a_{N/2+r})\, \omega_N^r)_{r=0}^{N/2-1}$ y dividiendo $\hat{a}_k$ en índices pares e impares, podemos completar la descomposición:

$$\begin{cases} \hat{a}_{2k} = \hat{b}_k, & k = 0, 1, \ldots, N/2-1, \\ \hat{a}_{2k+1} = \hat{c}_k, & k = 0, 1, \ldots, N/2-1. \end{cases} \tag{4.3}$$

Al igual que con Cooley-Tukey, podemos aplicar este proceso recursivamente y obtener un algoritmo FFT. En la Tabla 4.2 representamos las frecuencias obtenidas para esta variante.

Tabla 4.2: Frecuencias temporales y espectrales por nivel para la DIF-FFT en base 2.

Nivel	Ejemplo de entrada	Muestras	f_s	f_ω
0	$\{a_0, a_1, \ldots, a_{N-1}\}$	N	$1/T$	$1/NT$
1	$\{a_0, a_1, \ldots, a_{N/2-1}\}$	$N/2$	$1/T$	$2/NT$
2	$\{a_0, a_1, \ldots, a_{N/4-1}\}$	$N/4$	$1/T$	$4/NT$
3	$\{a_0, a_1, \ldots, a_{N/8-1}\}$	$N/8$	$1/T$	$8/NT$

4.1.3. Análisis de complejidad

A continuación, demostraremos que el algoritmo de Cooley-Tukey es una FFT. Para esto, utilizaremos uno de los casos del llamado *teorema maestro* para el análisis de algoritmos recursivos, véase ([6]).

Teorema 4.3 (Teorema maestro)**.** Sean $a > 0$ y $b > 1$ constantes enteras. Consideremos un algoritmo recursivo cuya entrada es un vector de tamaño N y que necesita $T(N)$ pasos para completar su ejecución, de forma que

$$T(N) = aT(N/b) + f(N),$$

donde $f(N)$ es el número de operaciones necesarias para combinar los resultados de las a llamadas recursivas sobre ciertos vectores de tamaño N/b. Si existe $k \geq 0$ tal que $f(N) = \Theta(N^{\log_b a} \log N^k)$, entonces $T(N)$ tiene orden de complejidad $\Theta(N^{log_b a} \log N^{k+1})$.

En el caso del algoritmo de Cooley-Tukey, tenemos $a = b = 2$, ya que subdividimos cada DFT de tamaño N en $a = 2$ DFTs de tamaño $N/b = N/2$. Además, $f(N) \in \Theta(N)$, ya que necesitamos $N/2$ iteraciones de bucle en las que efectuamos unas pocas operaciones complejas para recombinar ambas DFTs (véase el bucle de las líneas 7-11 en el Algoritmo 1). Por lo tanto, podemos aplicar el Teorema maestro con $k = 0$ y obtener nuestro resultado principal:

Corolario 4.4. El algoritmo de Cooley-Tukey es $\Theta(N \log N)$ para $N = 2^d$, $d \in \mathbb{N} \cup \{0\}$. Por lo tanto, es una FFT.

El razonamiento es idéntico para la variante DIF. De hecho, ambas variantes requieren el mismo número exacto de operaciones complejas, véase ([16, Sección 5.2.5.]). Terminamos este apartado con una pequeña nota: ningún algoritmo puede calcular una DFT de tamaño N con un orden de complejidad menor que $\Theta(N \log N)$. La demostración de este hecho se escapa de los objetivos de este trabajo, pero se pude consultar en ([15]). Como consecuencia, en la Definición 4.2 es indiferente escribir $\Theta(N \log N)$ u $\mathcal{O}(N \log N)$. La FFT de Cooley-Tukey conlleva un número de operaciones asintóticamente ideal.

4.1.4. Diagramas de mariposa y computación *in-place*

Ahora centraremos nuestra atención en otra forma de interpretar la FFT de Cooley-Tukey, que nos conducirá a una nueva versión del Algoritmo 1 con dos mejoras que reducen, generalmente, el tiempo de ejecución: el cálculo *in-place*, es decir, sin necesidad de crear vectores en memoria para almacenar los resultados intermedios de las DFT($N/2$), y sin recursividad explícita, es decir, calculando *de abajo a arriba* en el árbol ilustrado en la Figura 4.1.

Para lograr esto, introducimos una visualización denominada *diagrama de mariposa*, que muestra el flujo de datos entre cada paso de una FFT. En la Figura 4.2 se muestra el base, que se corresponde con la DFT(2)$(a_0, a_1) = (a_0 + a_1, a_0 - a_1)$. Las aristas incidiendo sobre cada nodo indican suma, mientras que las anotaciones realizadas sobre ellas indican factores multiplicativos. Para reducir el número de anotaciones, el factor multiplicativo de -1 se verá representado por una traza discontinua.

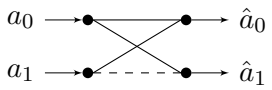


Figura 4.2: Diagrama de mariposa para una DFT(2).

La observación clave que nos permite representar la FFT de Cooley-Tukey con estos diagramas es la siguiente: el proceso de recombinación de los resultados de ambas DFT($N/2$), realizado en la ecuación (4.2), se puede descomponer en dos pasos: la multiplicación por el factor de giro ω_N^k de la segunda DFT($N/2$) y una DFT(2) sobre $(\hat{b}_k, \omega_N^k \hat{c}_k)$. De esta forma, podemos representar el primer nivel del algoritmo de Cooley-Tukey para el caso $N = 8$ como en la Figura 4.3, utilizando 4 mariposas como las de la Figura 4.2 tras la multiplicación por el factor de giro.

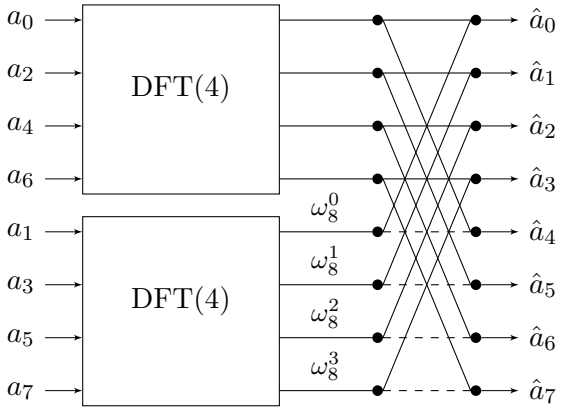


Figura 4.3: Diagrama de mariposa para el primer paso de la FFT de Cooley-Tukey con $N = 8$.

Si continuamos descendiendo en el árbol de ejecución, nos encontraremos con que debemos dividir en partes pares e impares cada uno de los subvectores de entrada de las DFTs, cada vez más pequeñas. Continuamos trazando el diagrama de mariposa hacia la izquierda, cambiando el orden de los elementos de entrada según son enviados a las DFT($N/2$), y añadiendo los $N/2$ productos por factores de giro y las $N/2$ mariposas necesarias en cada nivel para la combinación de los resultados. Finalmente, llegamos a la Figura 4.4.

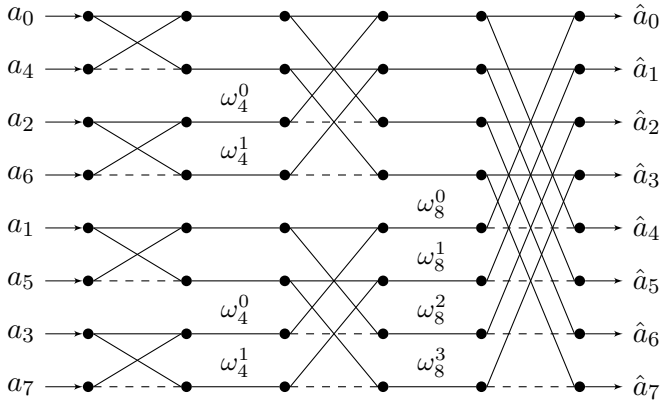


Figura 4.4: Diagrama de mariposa para la FFT de Cooley-Tukey con $N = 8$.

Ahora observemos que el orden de entrada de los valores al diagrama previo es el mismo al orden de la última fila del árbol de ejecución en la Figura 4.1. Pues bien, ¿cuál es este orden?. Si podemos reordenar de esta forma los elementos de a, entonces podríamos ejecutar el circuito descrito por el diagrama de mariposa *de izquierda a derecha* o, equivalentemente, el árbol de la Figura 4.1 *de abajo a arriba*, sin necesidad de hacer llamadas recursivas. Además, como cada par de elementos alterados por las mariposas de cada paso son independientes, podríamos calcular cada nivel sin necesidad de crear vectores intermedios. Para hallar este orden, emplearemos la representación binaria de cada uno de los índices:

$$j = (j_{d-1}j_{d-2}\dots j_1 j_0)_2 = 2^{d-1}j_{d-1} + 2^{d-2}j_{d_2} + \dots + 2j_1 + j_0,\ \ j_l \in \{0,1\},\ \ l = 0,1,\dots,d-1.$$

En el primer nivel del árbol de ejecución 4.1, separamos los elementos pares, con $j_0 = 0$, y los enviamos a la primera DFT, mientras que los impares, con $j_0 = 1$, van a la segunda. De esta forma, obtenemos el orden de índices del segundo nivel: $(0, 2, 4, 6, 1, 3, 5, 7)$, donde la primera mitad de índices tendrá $j_0 = 0$ y la segunda $j_0 = 1$. En resumen: hemos realizado una *permutación circular* de los bits de cada elemento al pasar del primer al segundo nivel: $(j_{d-1}j_{d_2}, \dots, j_1 j_0) \to (j_0 j_{d-1}, \dots, j_2 j_1)$. En el siguiente nivel, realizaremos una permutación circular de los $d-1$ primeros bits, y así sucesivamente hasta llegar al nivel final. Observamos que, tras acabar con las permutaciones, los índices aparecen en el *orden de inversión de bits*,

dado por $j' = 2^{d-1}j_0 + 2^{d-2}j_1 + \cdots + 2j_{d-2} + j_{d-1}$. Para $N = 8$, realizamos las permutaciones $(\underbrace{j_3 j_2 j_1 j_0}) \to (\underbrace{j_2 j_1 j_0} j_3) \to (\underbrace{j_1 j_0} j_2 j_3) \to (j_0 j_1 j_2 j_3)$ y el orden final es $(0, 4, 2, 6, 1, 5, 3, 7)$, con representación binaria $(000_2, 100_2, 010_2, 110_2, 001_2, 101_2, 011_2, 111_2)$.

En resumen, para implementar *in-place* y sin recursividad explícita la FFT de Cooley-Tukey, debemos preparar los datos realizando primero esta permutación de inversión de bits y luego realizar cada una de las $N/2$ mariposas correspondientes al nivel $l = 1, \ldots, d$ del algoritmo. En el Algoritmo 2 expresamos el pseudocódigo de esta implementación, donde utilizamos algunas variables adicionales para obtener los índices necesarios en cada una de las mariposas.

Algoritmo 2 FFT de Cooley-Tukey, versión in-place e iterativa

Entrada: $a = (a_0, a_1, \ldots, a_{N-1}) \in \mathbb{C}^N$: vector de entrada, N tamaño de los datos.
Salida: $\hat{a} = (\hat{a}_0, \hat{a}_1, \ldots, \hat{a}_{N-1}) \in \mathbb{C}^N$: DFT($N$) de a.

1: **procedure** COOLEY-TUKEY-FFT(a, N)
2: $a \leftarrow$ BIT-REVERSAL(a) ▷ Reordenamos el vector a en orden de bits invertido.
3: **for** $l \leftarrow 1 \ldots \log_2 N$ **do** ▷ Iteramos sobre cada nivel del árbol de abajo a arriba.
4: $s \leftarrow 2^l$
5: **for** $j \leftarrow 0 \ldots N/s - 1$ **do** ▷ Bucle sobre los 2^{d-l} bloques de mariposas.
6: $factorgiro \leftarrow 1$ ▷ Inicializamos el factor de giro del bloque.
7: **for** $k \leftarrow 0 \ldots s/2 - 1$ **do** ▷ Bucle sobre las 2^{l-1} mariposas de cada bloque.
8: $p \leftarrow j \cdot s + k$ ▷ Primer índice de la mariposa.
9: $q \leftarrow p + s/2$ ▷ Segundo índice de la mariposa.
10: $tmp_1 \leftarrow a_p$
11: $tmp_2 \leftarrow a_q \cdot factorgiro$
12: $a_p \leftarrow tmp_1 + tmp_2$
13: $a_q \leftarrow tmp_1 - tmp_2$
14: $factorgiro \leftarrow factorgiro \cdot \omega_s$
15: **end for**
16: **end for**
17: **end for**
18: **return** $\hat{a}$
19: **end procedure**

La implementación específica de la reordenación en orden de inversión de bits ha sido estudiada ampliamente en la literatura, y existen implementaciones $\Theta(N)$ *in-place* eficientes ([7]). Además de conllevar un consumo de memoria menor, esta implementación de la FFT suele ser preferida debido a que las llamadas recursivas entre funciones y la copia de datos es costosa, es-

pecialmente al trabajar con vectores de datos grandes. Por otro lado, los diagramas de mariposas son muy útiles para entender cómo funcionan e implementar algoritmos FFT más generales o complicados, como los que desarrollaremos en la siguiente sección.

4.2. Otras FFTs

4.2.1. Cooley-Tukey para enteros compuestos

En la publicación inicial de Cooley y Tukey ([5]) realmente se describe un algoritmo más general que la FFT en base 2, en concreto para un entero compuesto $N = N_1N_2$ con $N_1, N_2 \in \mathbb{N}$. Desarrollaremos la variante de decimación en tiempo de este algoritmo sobre la forma polinómica de la DFT, aunque, por supuesto, esto se puede hacer sobre la forma de suma o matricial.

Sean entonces $j_1 \equiv j \pmod{N_1}$ y $j_2 = \lfloor j/N_1 \rfloor$ para cada $j = 0, 1, \ldots, N-1$. Definimos también los índices $k_1 \equiv k \pmod{N_1}$ y $k_2 = \lfloor k/N_1 \rfloor$ para un $k = 0, 1, \ldots, N-1$. Utilizaremos las siguientes expresiones para indexar la fórmula de la DFT:

$$\begin{aligned} j &= j_1 + j_2 N_1, \\ k &= k_1 N_2 + k_2. \end{aligned}$$

Ahora sea $p(z) \in \mathbb{C}[z]$ de grado $N-1$ con coeficientes $a_0, a_1, \ldots, a_{N-1}$. Para calcular su DFT, debemos evaluarlo en las raíces N-ésimas de la unidad $p(\omega_N^k)$, $k = 0, 1, \ldots, N-1$. Utilizando los anteriores índices, obtenemos

$$p(\omega_N^{k_1N_2+k_2}) = \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} a_{j_1+j_2N_1}\, \omega_N^{(k_1N_2+k_2)(j_1+j_2N_1)}, \tag{4.4}$$

y como $\omega_N^{(k_1N_2+k_2)j_2N_1} = \omega_N^{N_1N_2j_2k_1}\, \omega_N^{k_2j_2N_1} = \omega_N^{Nj_2k_1}\, \omega_N^{j_2k_2N/N_2} = \omega_{N_2}^{j_2k_2}$, entonces

$$p(\omega_N^{k_1N_2+k_2}) = \sum_{j_1=0}^{N_1-1} \omega_N^{j_1(k_1N_2+k_2)} \sum_{j_2=0}^{N_2-1} a_{j_1+j_2N_1}\, \omega_{N_2}^{j_2k_2}. \tag{4.5}$$

Siguiendo con la interpretación polinómica, definimos $p^{(k_2)} \in \mathbb{C}[x]$, $k_2 = 0, 1, \ldots, N_2 - 1$ representando la evaluación en los N_1 puntos dados por $\omega_N^{k_1N_2+k_2}$, con $k_1 = 0, 1, \ldots, N_1 - 1$:

$$p^{(k_2)}(z) := \sum_{j_1=0}^{N_1-1} b_{j_1}^{(k_2)} z^{j_1}, \quad k_2 = 0, \ldots, N_2 - 1, \tag{4.6}$$

cuyos coeficientes son, precisamente, la suma interna de (4.5),

$$b_{j_1}^{(k_2)} = \sum_{j_2=0}^{N_2-1} a_{j_1+j_2N_1}\, \omega_{N_2}^{k_2j_2}, \quad j_1 = 0, \ldots, N_1 - 1. \tag{4.7}$$

De esta forma, el algoritmo calcula primero N_1 DFTs de orden N_2 para obtener los coeficientes $b_{j_1}^{(k_2)}$, para cada $j_1 = 0, 1, \ldots, N_1 - 1$. Una vez hecho esto, podemos evaluar los N_2 polinomios $p^{(k_2)}$ en los N_1 puntos $\omega_N^{k_1 N_2 + k_2}$ mediante N_2 DFTs de orden N_1(desplazadas por el factor de giro $\omega_N^{k_2}$), para cada $k_2 = 0, 1, \ldots, N_1 - 1$. Podemos continuar de forma recursiva hasta llegar a los factores primos de N, que se recuperarían mediante el cálculo directo de la DFT.

4.2.2. FFTs en otras bases y para números primos

Existe un problema con el anterior algoritmo: solo funciona bien para números altamente compuestos, con los que tenemos mayor nivel de recursividad. Además, los factores primos deben ser suficientemente bajos, ya que $\mathrm{DFT}(p)$ es $\Theta(p^2)$, lo cual puede incurrir en tiempo de ejecución demasiado alto. Es por esto que se comporta mejor cuando $N = p^d$ con p primo suficientemente bajo, ya que entonces podemos llegar hasta el final. Como vimos, en el caso $p = 2$, llegamos a la complejidad óptima $\Theta(N \log N)$, mientras que para otros enteros $N = p_1^{d_1} p_2^{d_2} p_3^{d_3} \ldots$, con p_i primos, el numero de operaciones realizadas por el algoritmo se puede expresar como una suma ponderada de las cantidades $N p_i \log_{p_i}(N)$.

Volviendo al caso $N = 2^d$, siendo d par, se puede desarrollar la *FFT de Cooley-Tukey en base 4*, cuya forma DIT parte de la descomposición

$$\hat{a}_k = \underbrace{\sum_{r=0}^{N/4-1} a_{4r}\, \omega_{N/4}^{rk}}_{\textbf{(I)}} + \omega_N^k \underbrace{\sum_{r=0}^{N/4-1} a_{4r+1}\, \omega_{N/4}^{rk}}_{\textbf{(II)}} + \omega_N^{2k} \underbrace{\sum_{r=0}^{N/4-1} a_{4r+2}\, \omega_{N/4}^{rk}}_{\textbf{(III)}} + \omega_N^{3k} \underbrace{\sum_{r=0}^{N/4-1} a_{4r+3}\, \omega_{N/4}^{rk}}_{\textbf{(IV)}}. \quad (4.8)$$

Para reducir aún más el número de operaciones complejas, podemos observar que los sumandos **(I)** y **(III)** se pueden combinar en el primer sumando de una FFT de radio 2, ahorrando la multiplicación compleja por el factor de giro ω_N^{2k} en **(III)**:

$$\hat{a}_k = \sum_{r=0}^{N/2-1} a_{2r}\, \omega_{N/2}^{rk} + \omega_N^k \sum_{r=0}^{N/4-1} a_{4r+1}\, \omega_{N/4}^{rk} + \omega_N^{3k} \sum_{r=0}^{N/4-1} a_{4r+3}\, \omega_{N/4}^{rk}. \quad (4.9)$$

Esta descomposición da lugar a la *FFT de base dividida* (véase ([26])) la cual, hasta recientemente ([13]), fue la FFT con el número de operaciones mínimo para el caso $N = 2^d$. También existen implementaciones iterativas con diagramas de mariposa para esta FFT. Aunque sí se conoce que el orden de complejidad óptimo para el cálculo de una DFT es $\Theta(N \log N)$, continúa siendo una pregunta abierta cuál es el número de operaciones mínimo.

Por otro lado, existen otros algoritmos como la *FFT de Rader* ([18]) o la *FFT de Bluestein* ([1]) que permiten obtener el orden de complejidad $\Theta(N \log N)$ para un N cualquiera, incluso

primo. Describimos a continuación el algoritmo de Bluestein, que se basa en la siguiente identidad:

$$jk = \frac{-(j-k)^2}{2} + \frac{j^2}{2} + \frac{k^2}{2}.$$

Aplicando esto al cálculo de $\hat{a}$, obtenemos:

$$\hat{a}_k = \sum_{j=0}^{N-1} a_j\, \omega_N^{jk} = \omega_N^{k^2/2} \sum_{j=0}^{N-1} \left(a_j\, \omega_N^{j^2/2}\right) \omega_N^{-(k-j)^2/2}, \quad k = 0, 1, \ldots, N-1.$$

Si ahora definimos las sucesiones

$$b = \left(a_j\, \omega_N^{j^2/2}\right)_{j=0}^{N-1}, \quad h = \left(\omega_N^{-j^2/2},\right)_{j=0}^{N-1},$$

entonces, como $h_{j-k} = h_{k-j}$, podemos expresar cada elemento de la DFT como una convolución cíclica desplazada por un factor de giro $\omega_N^{k^2/2}$:

$$\hat{a}_k = \omega_N^{k^2/2} \sum_{j=0}^{N-1} b_j h_{(j-k) \bmod N} = \omega_N^{k^2/2}(b * h), \quad k = 0, 1, \ldots, N-1.$$

Para calcular esta convolución eficientemente, primero debemos extender con ceros b y h a $\mathbb{C}^{2N-1}$, para poder calcular la convolución en su forma acíclica. Extendiendo de nuevo hasta la potencia de 2 más cercana $M = 2^t$, con $2N - 1 \leq M < 4N$, podemos aplicar el Teorema 3.10 y la FFT de Cooley-Tukey para pasar ambos vectores extendidos al dominio de frecuencia. Finalmente, debemos multiplicarlos punto a punto, realizar una DFT inversa y multiplicar por los factores de giro de la expresión anterior. De esta forma, hemos convertido el cálculo de DFT(N) a tres DFT(M), que podemos calcular en tiempo $\Theta(M \log M) = \Theta(4N \log 4N) = \Theta(N \log N)$. Como los demás pasos son lineales, efectivamente el algoritmo de Bluestein es una FFT.

4.2.3. Otros detalles y librerías para el cálculo de FFTs

En la práctica, el cálculo de DFTs en el menor tiempo posible conlleva muchos otros factores que no hemos considerado. Además del orden de complejidad y la memoria utilizada, es importante minimizar los desplazamientos de datos en memoria, facilitar la paralelización (especialmente relevante en DFTs multidimensionales) y poder elegir un algoritmo FFT adecuado para cada tamaño, permitiendo cambiar de algoritmo en cada paso. También se suelen utilizar muchos trucos de implementación para acelerar el cálculo, tales como precomputar los factores de giro y los circuitos de las DFTs de tamaños pequeños.

Una de las librerías actuales más eficientes para la computación de alto rendimiento de DFTs es FFTW ([8]), que incorpora un paso previo en el que se realizan medidas exhaustivas sobre el

rendimiento del sistema y se genera un plan de ejecución. Este plan de ejecución describe cómo descomponer cada DFT(N), qué algoritmo utilizar en cada paso (Cooley-Tukey, base dividida, Bluestein, etc.), la elección entre el cálculo *in-place* o *out-of-place*, qué factores de giro es necesario precomputar, cómo acceder a la información eficientemente, y otras muchas consideraciones prácticas.

Otra implementación popular, que ha sido de gran ayuda para la realización de este trabajo, es el módulo *fft* del paquete SciPy ([23]). Esta librería utiliza el algoritmo de Cooley-Tukey excepto para tamaños N difíciles de factorizar, para los que recurre al algoritmo de Bluestein.

Por otro lado, existen otras variantes y generalizaciones de la DFT como las Transformadas de Cosenos/Senos Discretas (DCT/DST), que trabajan exclusivamente con muestras y frecuencias reales y la Transformada de Teoría de Números (NTT), que trabaja sobre el cuerpo finito $\mathbb{Z}/p$, con p primo. Usaremos algunas de estas variantes en el siguiente capítulo. El desarrollo de algoritmos rápidos para todas estas variantes es similar al de las FFTs usuales.

Capítulo 5

Aplicaciones

A lo largo de este capítulo final, nos adentraremos en dos de los grandes campos de aplicación de la DFT y la FFT: el procesamiento de señales y la compresión de información.

5.1. Filtros digitales de convolución

Los filtros digitales son algoritmos que actúan sobre una señal digital expresada como una secuencia de valores, normalmente reales, $a \in \mathbb{R}^N$. Por ejemplo, esta señal puede ser de audio, captada por un micrófono, o eléctrica, medida en un electrocardiograma. Sus objetivos principales son la separación de señales que fueron previamente combinadas, o la restauración de una señal distorsionada de alguna forma, usualmente con ruido de fondo ([22]). Los filtros digitales se clasifican en dos grandes categorías:

1. Filtros de convolución o de respuesta a impulso finita (FIR): caracterizados por generar una salida finita a partir de un impulso finito. Son más lentos, pero altamente precisos. Su forma general sigue la ecuación de convolución discreta acíclica (3.11) y, por tanto, son el objeto de nuestro estudio.

2. Filtros recursivos o de respuesta a impulso infinita (IIR): caracterizados por crear una señal teóricamente infinita a partir de un impulso finito. Su implementación suele ser recursiva, dependiendo de salidas anteriores, es decir, $\hat{a}_k$ se calcula a partir de valores $\hat{a}_l$ con $l < k$. Son mucho más rápidos pero menos precisos. Su forma general sigue la siguiente ecuación:

$$\hat{a}_k = \sum_{j=0}^{P-1} b_j a_{k-j} + \sum_{j=1}^{Q-1} c_j \hat{a}_{k-j}, \quad b \in \mathbb{R}^P, c \in \mathbb{R}^{Q-1}, P \in \mathbb{N}, Q \in \mathbb{N}.$$

El filtro FIR más sencillo es la media móvil, que suaviza la señal mediante la convolución

con un kernel constante $K = (1/L, 1/L, \ldots, 1/L) \in \mathbb{R}^L$. Por otro lado, tenemos los filtros de paso bajo, que buscan eliminar las frecuencias de la señal con amplitud mayor a una f_c llamada *frecuencia de corte*. Esta frecuencia se expresa normalizada, como una fracción de la frecuencia de muestreo de la señal f_s. Debido al Teorema de muestreo de Nyquist-Shannon ([16]), $f_c \leq 0.5$, ya que una función con una frecuencia de muestro f_s solo tiene frecuencias significativas, sin *aliasing*, en el rango $[0, f_s/2]$. Para obtener la expresión del filtro de paso bajo, basta replicar el cálculo de la integral del Ejemplo 2.4, tan solo cambiando el signo del exponente para calcular la transformada inversa. También, por convenio propio del campo de procesamiento de señales, cambiamos a la forma (2.12) de la transformada. Como es de esperar, el filtro de paso bajo ideal se expresa en función del seno cardinal:

$$l(x) = \int_{-f_c}^{f_c} e^{2\pi i \omega x}\, d\omega = \frac{1}{\pi x} \sin(2\pi f_c\, x) = 2\pi \operatorname{sinc}(2\pi f_c\, x). \tag{5.1}$$

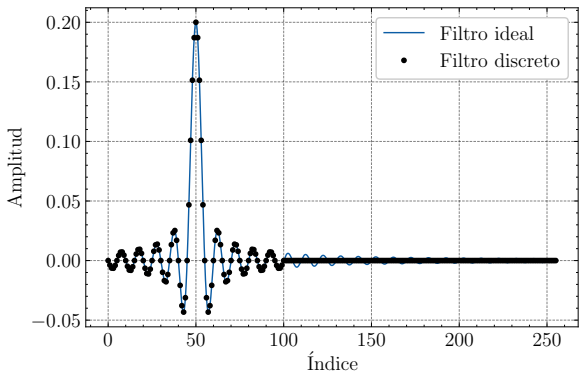
Sin embargo, nuestras señales son discretas y finitas, lo cual introduce cierto error. Para construir un filtro práctico a partir de (5.1), primero elegimos una longitud L, normalmente impar, para poder incluir el pico del seno cardinal, y muestreamos en los enteros:

$$l_j = \frac{\sin(2\pi f_c j)}{\pi j}, \quad j \in \{-(L-1)/2, \ldots, (L-1)/2\}. \tag{5.2}$$

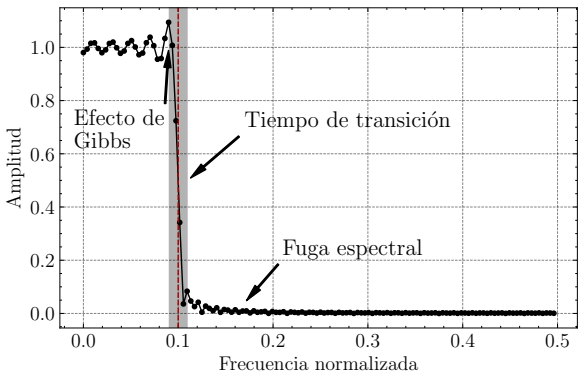
Como detalle de implementación, es común desplazar los índices $S = (L-1)/2$ unidades para pasarlos al rango positivo $0, \ldots, L-1$. Además, como ya hemos visto, para poder realizar una convolución mediante la FFT es necesario extender ceros hasta cierto $M \geq N + L - 1$. En la Figura 5.1 mostramos el núcleo obtenido con $L = 101$, extendido hasta $M = 256$, junto a su DFT. Señalamos los 3 tipos de error fundamentales: el tiempo de transición (*roll-off*), la ondulación producida por el efecto de Gibbs y la fuga espectral (*spectral leakage*) en las frecuencias con valor absoluto mayor a f_c.

Para mejorar la precisión del filtro, además de aumentar L, podemos multiplicar por una *función ventana*. Estas son funciones suaves, basadas en polinomios trigonométricos y especialmente diseñadas para reducir la intensidad del filtro suavemente, eliminando gran parte de la distorsión en su espectro. Existen varias funciones ventana como la de Blackman o la de Hamming, cada una de las cuáles presenta diferentes perfiles espectrales. En la Figura 5.2 visualizamos las DFTs de nuestro filtro (5.2) multiplicado por diferentes funciones ventana, representando las amplitudes espectrales en decibelios. Esta unidad no es más que un logaritmo en base 10 escalado, y se emplea aquí por convenio y por claridad en la representación de las frecuencias, $|\hat{l}_j|\ [dB] = 20 \log_{10} |\hat{l}_j|$.

Observamos que las ventanas de Hamming y Blackman suavizan el efecto de Gibbs, y reducen el tiempo de transición y las fugas espectrales. En particular, la ventana de Hamming tiene un



(a) Filtro de paso bajo ideal con $f_c = 0.1$ frente al discreto.



(b) Magnitud del espectro $|\hat{h}|$ con defectos señalados.

Figura 5.1: Filtro de paso bajo con $L = 101$, $M = 256$ y $f_c = 0.1$.

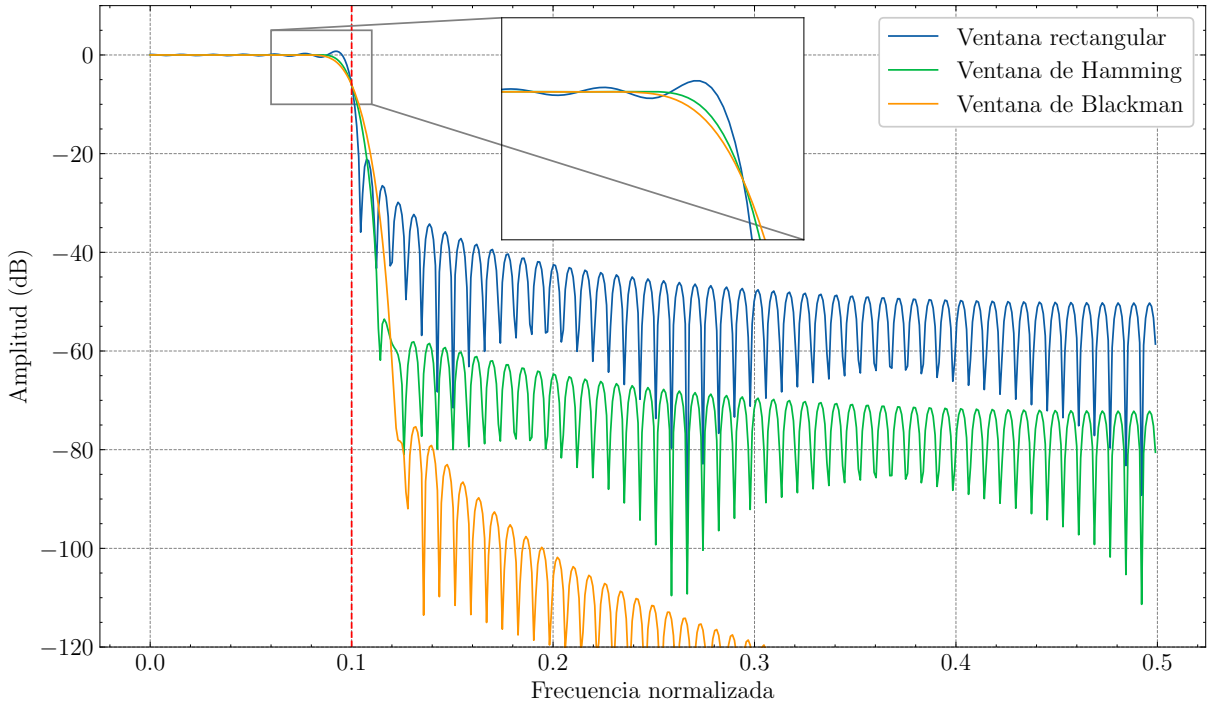


Figura 5.2: Perfiles espectrales de diferentes ventanas para el filtro de paso bajo con $L = 101$, $M = 1024$ y $f_c = 0.1$. La "ventana rectangular" se corresponde con no aplicar ninguna función de ventana.

menor tiempo de transición pero una mayor fuga espectral, mientras que la ventana de Blackman presenta la propiedad contraria. Un análisis más exhaustivo sobre las ventajas de cada tipo de ventana se puede encontrar en ([10]). Utilizando, por ejemplo, la ventana de Blackman, el filtro

de paso bajo de longitud L y con frecuencia de corte f_c tiene la siguiente expresión:

$$l_j^{(f_c)} = \frac{\sin(2\pi f_c(j-S))}{\pi(j-S)}\left(0.42 - 0.5\cos\left(\frac{2\pi j}{L}\right) + 0.08\cos\left(\frac{4\pi j}{L}\right)\right), \quad j \in \{0,\dots,L-1\}. \tag{5.3}$$

Con esto, completamos la construcción del filtro de paso bajo. A partir de este, podemos construir un filtro de paso alto, que conserva solo las frecuencias con magnitud mayor o igual a f_c. Para ello, definimos un impulso corto δ como $\delta_S = 1$, $\delta_j = 0$ si $j \neq S$ y definimos $h_j^{(f_c)} = \delta_j - l_j^{(f_c)}$, $j = 0,\dots,L-1$. El impulso δ representa una versión discreta y finita de la distribución delta, con nuestros índices desplazados. Este proceso se suele denominar *inversión espectral* ([22, Capítulo 14]). A partir de estos, podemos construir también filtros de rechazo de banda, que eliminan todas las frecuencias dentro del rango $[f_{c1}, f_{c2}]$, $br_j = l_j^{(f_{c1})} + h_j^{(f_{c2})}$ y de paso de banda, que eliminan todas las frecuencias fuera de ese rango, $bp_j = \delta_j - br_j$.

Cuando L es suficientemente bajo, no hay ningún problema en realizar la convolución con el filtro FIR directamente en el dominio temporal, ejecutando en un orden de $\mathcal{O}(NL)$ operaciones sobre una señal de longitud N. Sin embargo, cuando necesitamos una precisión más alta, podemos aplicar el Teorema de convolución discreta 3.10, extendiendo los vectores adecuadamente para poder utilizar un algoritmo FFT. Como ya hemos anotado, para convolucionar una señal de longitud N con un núcleo de longitud L, necesitamos extender primero a $M = N+L-1$ y luego, si queremos usar el algoritmo de Cooley-Tukey, a la potencia de 2 más cercana. De esta forma, conseguimos un tiempo de ejecución de $\mathcal{O}(N \log N)$.

Ejemplo 5.1 (Aislando notas musicales)**.** En este ejemplo, buscamos ilustrar los conceptos anteriores mediante un caso práctico en el que intentamos aislar dos notas musicales en una grabación digital. Disponemos de 2 segundos de audio, donde suenan simultánea y continuamente un Re en la tercera octava, con frecuencia fundamental $f_{\text{Re3}} \approx 146.83$ Hz, tocado por una guitarra acústica, así como un Mi en la sexta octava, con frecuencia fundamental $f_{\text{Mi6}} \approx 1318.51$ Hz, tocado por un teclado. La señal consta de $N = 118700$ muestras, y la frecuencia de muestreo es de $f_s = 48000$ Hz.

En la Figura 5.3 mostramos la magnitud del espectro de la señal en decibelios. Resaltamos tanto las frecuencias fundamentales f_{Re3} y f_{Mi6} como sus *frecuencias harmónicas*. Estas son, para un instrumento perfectamente afinado, múltiplos de las frecuencias fundamentales, y su distribución y amplitud relativa define el timbre característico del instrumento. Para diseñar un filtro que logre aislar, por ejemplo, la nota tocada por el teclado, es necesario eliminar tanto la frecuencia fundamental f_{Re3} como sus harmónicos.

Primero, veamos cómo eliminar el tono fundamental f_{Re3}. Para ello, construimos un filtro de rechazo de banda $br^{(1)} \in \mathbb{R}^L$ mediante senos cardinales con ventana de Blackman. Sea $R > 0$, el radio del ancho de banda, tomamos $f_{c1} = (f_{\text{Re3}} - R)/f_s$, $f_{c2} = (f_{\text{Re3}} + R)/f_s$ y entonces $br_j^{(1)} = l_j^{(f_{c1})} + h_j^{(f_{c2})}$ para $j = 0, 1, \dots, L-1$.

De igual forma, se pueden construir el resto de filtros $br^{(i)}$ para cada frecuencia harmónica $i \cdot f_{\text{Re3}}$, $i \in \mathbb{N}$, limitándonos en este ejemplo a $i \leq 8$. En vez de aplicar cada filtro sobre la señal a en sucesión, podemos utilizar la asociatividad de la convolución acíclica y construir un filtro de "rechazo de banda múltiple", $bm = br^{(1)} * br^{(2)} * \cdots * br^{(8)}$.

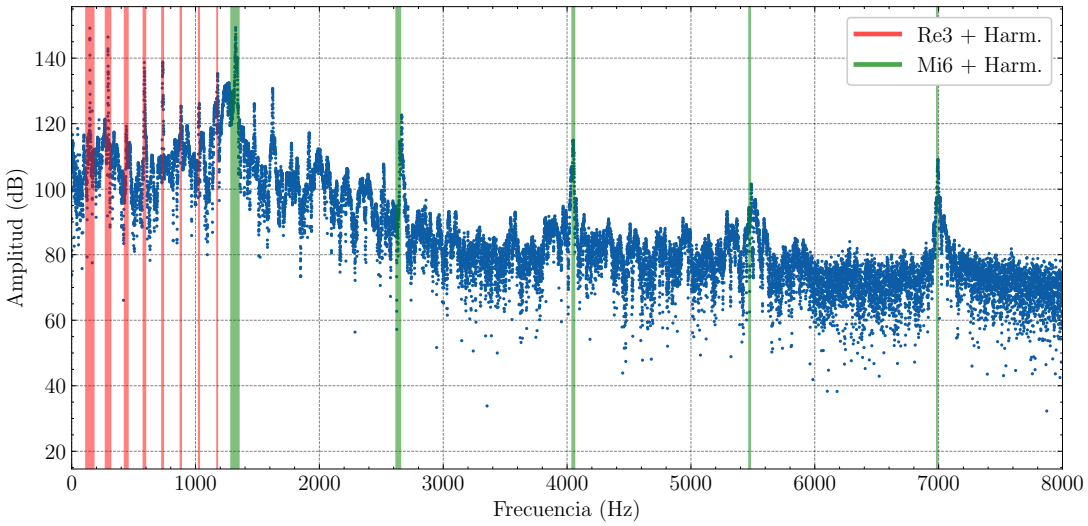


Figura 5.3: Espectro del audio con las frecuencias de Re3 y Mi6 destacadas, junto a sus harmónicos.

El tamaño de bm será $8(L-1)+1$, debido a que la convolución acíclica de dos vectores de longitud L contiene $2L-1$ elementos. Tomando $L = 5001$ y $R = 20$, visualizamos el espectro del filtro en la Figura 5.4, obteniendo una reducción de 35 dB en las frecuencias a eliminar. Escogiendo un L más grande, el filtro se puede hacer más potente, aunque para nuestra muestra de audio no fue necesario.

En el Anexo I se añaden explicaciones adicionales del proceso, así como un enlace al código fuente original y la muestra de audio usada. Tanto estas convoluciones como la convolución final con la señal se pueden realizar en el dominio de frecuencia utilizando la FFT, rellenando con tantos ceros como sea ncesario. Especialmente, en la convolución final $bm * a$, es indispensable utilizar un algoritmo rápido, ya que, en otro caso, estaríamos realizando un gran número de operaciones, en el orden de $8NL \approx 3.79 \cdot 10^{10}$.

5.2. Algoritmos de compresión: formato JPEG

Las aplicaciones de la DFT van más alla de los filtros digitales y el teorema de convolución. En el campo de la compresión de archivos multimedia, es común compactar las características

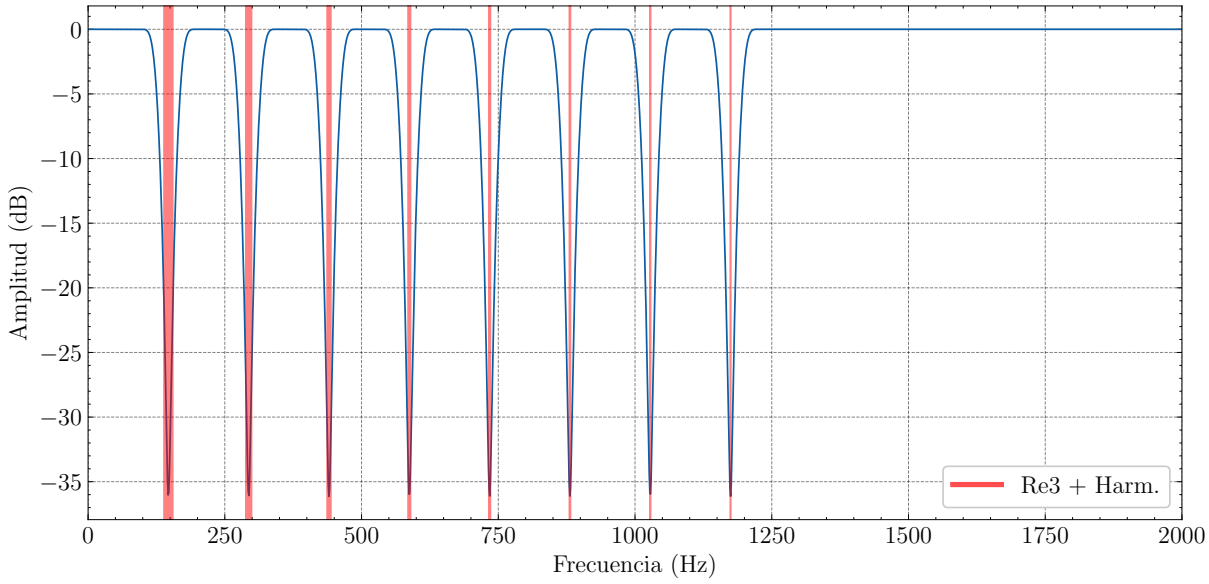


Figura 5.4: Espectro del filtro *bm*, visualizando la disminución de amplitud obtenida sobre cada frecuencia harmónica de Re3.

fundamentales de un audio, una imagen o un vídeo a partir de una representación espectral. El formato *JPEG (Joint Photographics Experts Group)* ([12, 24]) es un buen ejemplo de ello. Con sus más de 30 años de historia, continúa siendo uno de los estándares más utilizados en todo el mundo para la compresión de imágenes, conteniendo casi todo dispositivo cientos de archivos con la extensión `.jpg` que los caracteriza. Pues bien, el algoritmo utilizado para la codificación y decodificación (*códec*) de JPEG se fundamenta en una descomposición de los patrones de la imagen mediante la DCT (Transformada Discreta de Cosenos), que discutimos a continuación. Al igual que en la anterior sección, se puede obtener más información acerca del código fuente y las imágenes usadas en el Anexo I.

Para llegar a la DCT, es conveniente recordar la relación entre la paridad de las funciones y sus series de Fourier. En concreto, sabemos que si tenemos una función par con periodo $2L$, entonces su serie de Fourier contiene tan solo cosenos, en lugar de exponenciales complejas

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} a_n \cos\left(\frac{n\pi x}{L}\right).$$

De esta forma, si tenemos una función f tan solo definida en $[0, L]$, podemos extenderla con paridad a $[-L, L]$ y luego considerar su extensión periódica, para la cual podemos calcular la serie de Fourier (llamada, en este caso, serie de cosenos). Esta misma idea se puede trasladar a la DFT, llegando a la noción de Transformadas Trigonométricas Discretas (DTT), las cuales son la DCT (cosenos) y la DST (senos).

Sin embargo, existen varias formas de extender un vector de $\mathbb{R}^N$ con paridad, dando lugar

a 16 tipos distintos de transformadas trigonométricas discretas: las de cosenos DCT-I, DCT-II, ... DCT-VIII y las de senos DST-I, DST-II, ..., DST-VIII. Estas 16 variaciones surgen de las 4 formas posibles de extender cada uno de los dos extremos de la secuencia, que sirven como puntos origen de simetría. Para el extremo derecho tenemos:

1. Simetría par, sobre muestra entera: $a, b, c, d \to a, b, c, d, c, b, a$.
2. Simetría impar, sobre muestra entera: $a, b, c, d \to a, b, c, d, 0, -d, -c, -b, -a$.
3. Simetría par, sobre media muestra: $a, b, c, d \to a, b, c, d, d, c, b, a$.
4. Simetría impar, sobre media muestra: $a, b, c, d \to a, b, c, d, -d, -c, -b, -a$.

Para el extremo izquierdo, debemos tener en cuenta la extensión periódica realizada implícitamente por la DFT. Cada una de estas combinaciones da lugar a una transformada trigonométrica que actúa sobre un vector de $\mathbb{R}^N$ que, implícitamente, asume esta periodicidad. Gracias a las propiedades de simetría de la DFT (Proposición 3.14), las DTTs están definidas como funciones puramente reales, lo cual es una ventaja computacional. Sin entrar en más detalles sobre las propiedades de cada una de las DTTs posibles, que se pueden consultar en ([20]), definiremos la más usada, la DCT-II (comúnmente llamada *la* DCT), y su inversa, la DCT-III (comúnmente llamada *la* IDCT). [1]

Definición 5.2. Dado $x \in \mathbb{R}^N$, su Transformada de Cosenos Discreta, $X = DCT(x) \in \mathbb{R}^N$ vendrá dada por

$$\hat{X}_k = \sqrt{\frac{2}{N}}\, \xi(k) \sum_{j=0}^{N-1} \cos\left(\frac{k(2j+1)\pi}{2N}\right) x_j, \quad k = 0, 1, \ldots, N-1, \tag{5.4}$$

donde $\xi : \{0, 1, \ldots, N-1\} \to \mathbb{R}$ viene definida por $\xi(0) = \sqrt{\frac{1}{2}}$, $\xi(j) = 1$ si $j \neq 0$. Su transformada inversa será la siguiente:

$$x_j = \sqrt{\frac{2}{N}}\, \xi(j) \sum_{k=0}^{N-1} \cos\left(\frac{j(2k+1)\pi}{2N}\right) X_k, \quad j = 0, 1, \ldots, N-1. \tag{5.5}$$

La DCT-II realmente se corresponde con la simetría par sobre media muestra, aplicada en ambos extremos de la sucesión. Extendiendo apropiadamente el vector de entrada x y ajustando

[1] Algunas DCTs como la I y la IV son su propia inversa salvo normalización. El motivo de elegir el par DCT-II, DCT-III viene de dos razones: conveniencia computacional, al ser sus expresiones casi idénticas, y el hecho de que las discontinuidades causadas por las extensiones impares son inconvenientes, de forma similar a lo que ocurre con las discontinuidades puntuales en las series de Fourier.

las constantes de normalización, es posible expresarla como un caso particular de la DFT. También es inmediato adaptar ambas expresiones (5.4) y (5.5) al caso $d-$dimensional. En concreto, JPEG utiliza una DCT de tamaño 8×8 ([24]), con expresión

$$X_{k_1,k_2} = \frac{1}{4}\,\xi(k_1)\xi(k_2) \sum_{j_1=0}^{7} \sum_{j_2=0}^{7} \cos\left(\frac{k_1(2j_1+1)}{16}\right) \cos\left(\frac{k_2(2j_2+1)}{16}\right) x_{j_1,j_2}, \tag{5.6}$$

$$x_{j_1,j_2} = \frac{1}{4}\,\xi(j_1)\xi(j_2) \sum_{k_1=0}^{7} \sum_{k_2=0}^{7} \cos\left(\frac{j_1(2k_1+1)}{16}\right) \cos\left(\frac{j_2(2k_2+1)}{16}\right) X_{k_1,k_2}. \tag{5.7}$$

Por supuesto, estas transformadas se pueden optimizar con algoritmos muy similares a las FFTs, véase ([14]). En el caso 8×8, es común el uso de implementaciones especializadas para cada hardware, atendiendo a la disponibilidad de instrucciones vectorizadas y la arquitectura concreta del procesador. Veamos ahora cómo se utiliza esta transformada dentro del *códec* de JPEG. En primer lugar, se establecen algunas suposiciones sobre cómo percibe el ojo humano las imágenes fotográficas:

I. El ojo es más sensible al cambio entre brillo (luminancia o *luma*) que al cambio entre color (crominancia o *chroma*).

II. El ojo es más sensible a cambios en regiones de baja frecuencia (áreas planas de color o intensidad) que en regiones de alta frecuencia (bordes y texturas).

Consideremos una imagen I de tamaño $N_1 \times N_2$ que queremos comprimir. Habitualmente, los colores de cada píxel p_{j_1,j_2} de la imagen se encuentran en el espacio de color tridimensional sRGB, donde las dimensiones representan el rojo (R), el verde (G) y el azul (B), y los valores son enteros en el rango $[0, 255]$. Por ejemplo, un rojo puro sería $(255, 0, 0)$, un naranja $(255, 207, 0)$ y el negro puro $(0, 0, 0)$. El primer paso realizado por JPEG es la conversión de sRGB al espacio Y'CbCr, cuya componente Y' representa la luminancia del píxel y las componentes Cb y Cr representan su crominancia, separada en un espacio bidimensional donde Cb se corresponde con tintes azulados y Cr con tintes rojizos, véase Figura 5.5. Esta conversión se realiza mediante una transformación lineal, y es útil gracias a I., ya que permite considerar la crominancia por separado, a la cual el ojo humano es más insensible. Representamos cada una de estas componentes como una matriz, $M_{Y'}, M_{Cb}, M_{Cr} \in \mathcal{M}_{N_1 \times N_2}([0, 255] \cap \mathbb{Z})$. En ocasiones, siguiendo la misma suposición I., se suele efectuar un *submuestreo de crominancia*. En este paso se igualan, típicamente en bloques de tamaño 2×2, las componentes Cb y Cr de los píxeles, reduciendo a una cuarta parte la cantidad de información en M_{Cb} y M_{Cr}.

Una vez hecho esto, es hora del paso principal del algoritmo. En este, fragmentamos cada

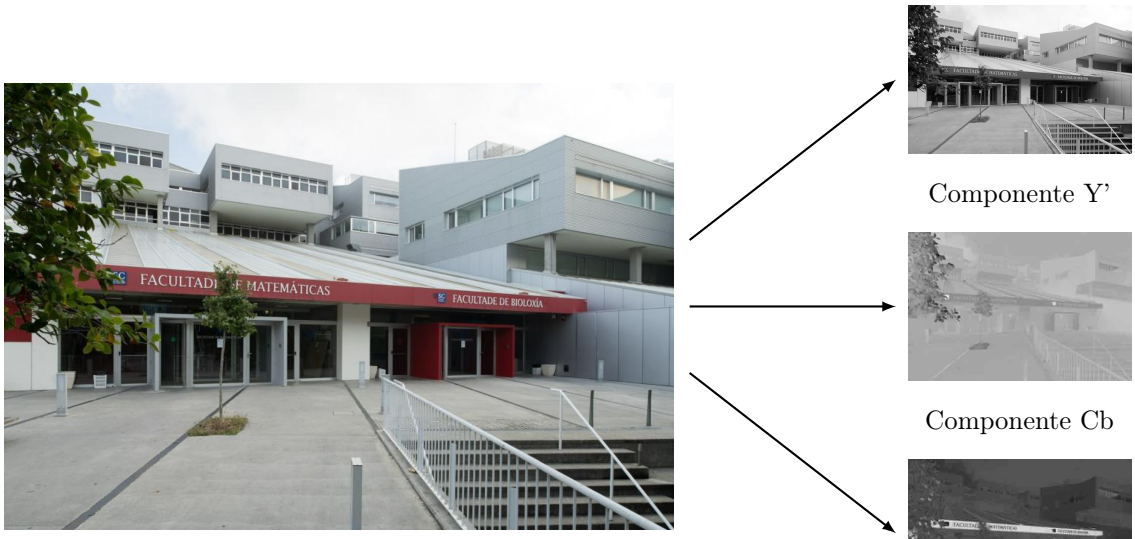


Figura 5.5: Separación de la imagen original en los componentes Y', Cb y Cr.

matriz en bloques de tamaño 8×8 [2] y restamos 128 para pasar al rango de valores $[-128, 127]$, Fig. 5.7a. Sobre cada uno de estos bloques, aplicamos la DCT bidimensional (5.6) mediante un algoritmo optimizado basado en la FFT, Fig. 5.7b. Para interpretar qué información contienen realmente cada uno de los coeficientes de la transformada, X_{k_1,k_2}, es útil la visualización de la Figura 5.6, donde cada subimagen muestra las intensidades de cada elemento de la base de $\mathcal{M}_{8\times 8}(\mathbb{R})$ de la transformada. Recordemos que las transformadas discretas no son más que transformaciones lineales, y en este caso, la intensidad de cada uno de los píxeles del bloque original no es más que una suma ponderada por los coeficientes X_{k_1,k_2} de cada uno de estos patrones.

Una vez transformado el bloque, es hora de emplear las suposición II. para comprimir la imagen. Para ello, JPEG utiliza una idea muy sencilla, llamada *cuantización*. En este proceso, dividimos cada uno de los X_{k_1,k_2} por los valores de una tabla $T \in \mathcal{M}_{8\times 8}$ y redondeamos al entero más cercano. Los coeficientes de T serán más grandes para las frecuencias altas, con k_1 o k_2 más elevado. De esta forma, se reduce la información presente en las frecuencias altas, de forma normalmente indistinguible para el ojo humano. La tabla T usada depende de la configuración de calidad deseada, cuyo valor varía entre 1 (calidad mínima) y 100 (calidad máxima) y el canal

[2] Si las dimensiones de la imagen no son divisibles entre 8, entonces se deben tener en cuenta como rellenar los bordes para minimizar la aparición de defectos. No entraremos en detalles sobre esto y supondremos que N_1 y N_2 son múltiplos de 8.

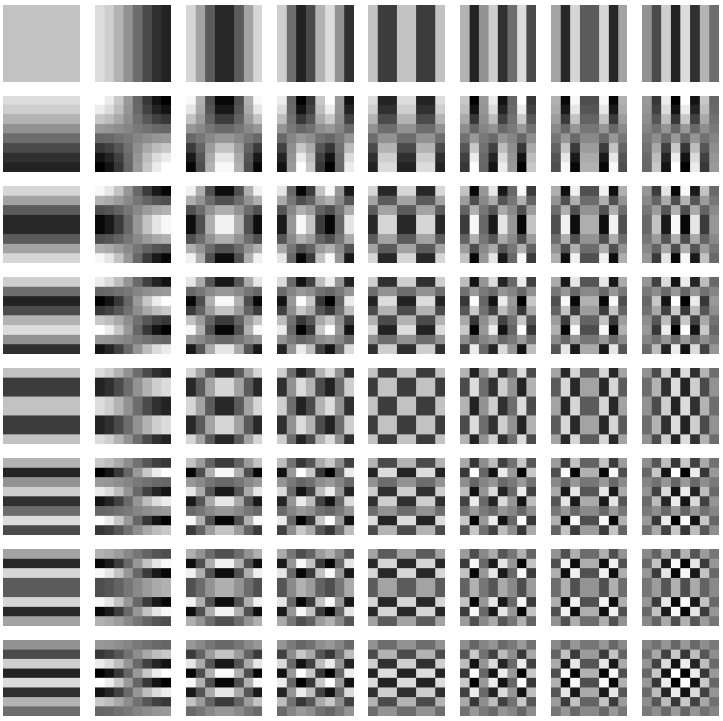
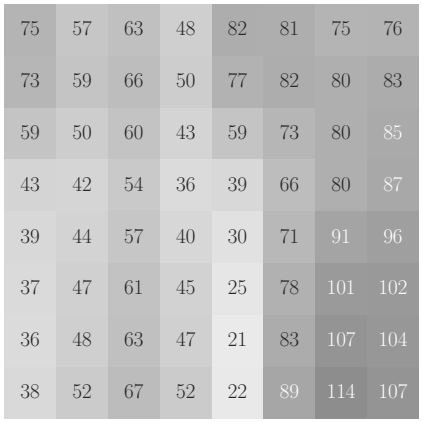


Figura 5.6: Funciones base de la DCT-2 en bloques de 8×8.

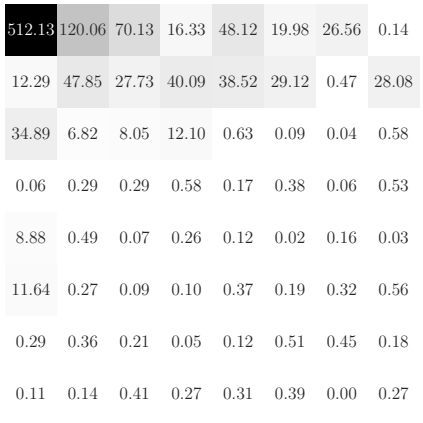
elegido, habiendo diferentes tablas para *luma* (componente Y) y *chroma* (componentes Cb y Cr), denotaremos $T_{lu,q}$ y $T_{ch,q}$ a las tablas correspondientes a una calidad q, Fig. 5.7c, 5.7d. El paso de cuantización es especialmente importante, ya que (además del submuestreo cromático) es el único donde realmente se pierde información sobre la imagen original, al ser la función de redondeo no biyectiva.

Tras la cuantización, es común tener un gran número de ceros en la parte correspondiente a las altas frecuencias del bloque. Esto se debe a la llamada propiedad de *compactación de energía* de la DCT. En este contexto, definimos la energía de un bloque como la suma del módulo de cada uno de sus componentes al cuadrado. Es común que la DCT de cada bloque concentre una gran parte de la energía de la señal en unos pocos elementos, y suele hacerlo en los correspondientes a las frecuencias bajas, como observamos en la Figura 5.7b. De hecho, este fenómeno no ocurre solo en imágenes fotográficas. En un gran número de señales provenientes de mediciones físicas es común encontrar una relación $|\hat{a}_k|^2 \propto 1/k^\alpha$, con $\alpha \approx 1$. Se suele decir que estas señales presentan *ruido* $1/f$ o *ruido rosa*. Los motivos detrás de su aparición en todo tipo de sistemas no relacionados entre sí es aún desconocido, siendo objeto de numerosos estudios.

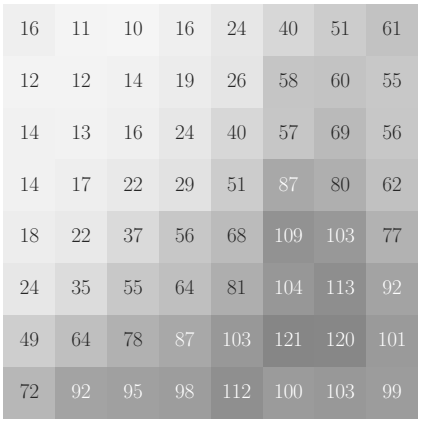
Para finalizar el proceso, linealizamos el bloque mediante el orden zig-zag representado en la Figura 5.7e, y realizamos un proceso de *codificación entrópica* sobre el vector resultante $X_L \in \mathbb{Z}^{64}$. La codificación entrópica es una técnica de compresión sin pérdidas proveniente del campo de la



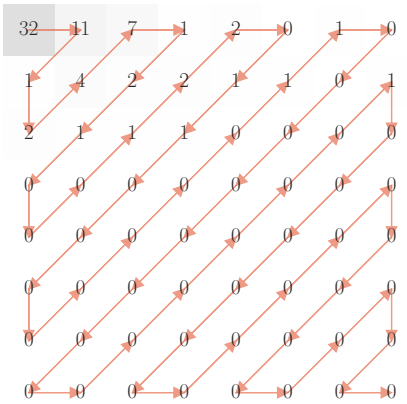
(a) Bloque original de $M_{Y'}$.



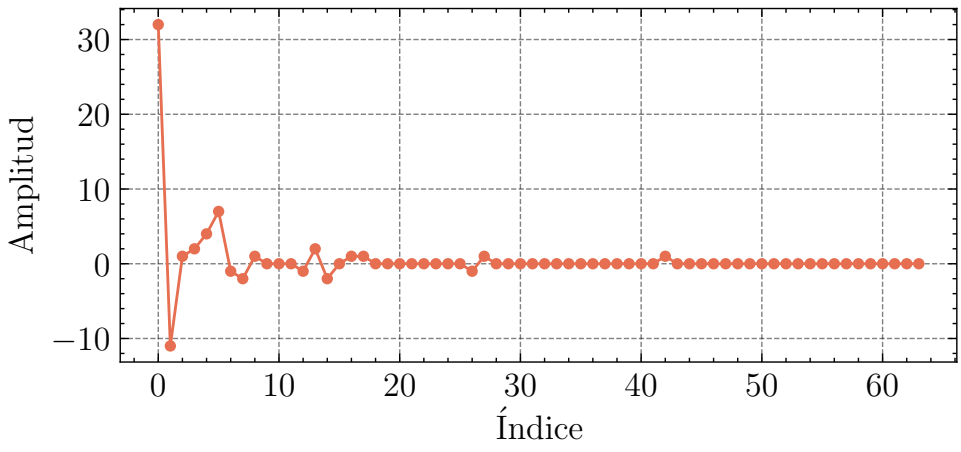
(b) DCT del bloque.



(c) Tabla de cuantización $T_{lu,50}$.



(d) Bloque cuantizado y zig-zag.



(e) Linealización del bloque, con varios ceros consecutivos.

Figura 5.7: Algoritmo JPEG para la codificación y cuantización de bloques.

teoría de la información que busca representar conjuntos de datos de forma compacta e invertible, utilizando únicamente la información contenida en la propia secuencia.

En JPEG, normalmente se utilizan dos algoritmos de codificación entrópica. En primer lugar, se emplea *run-length encoding*, que simplemente anota el número de enteros iguales consecutivos. Por ejemplo, el siguiente vector: $X_L = (122, 23, 12, 0, 0, 1, 0, 0, \ldots, 0)$ se transforma en dos vectores $X_{val} = (122, 23, 12, 0, 1, 0), X_{rep} = (1, 1, 1, 2, 1, 58)$, conteniendo los valores y el número de veces que se repite cada uno de ellos. Como el orden en zig-zag hace que aparezcan muchos ceros consecutivos correspondientes a las altas frecuencias, este paso suele reducir bastante el número de enteros a almacenar. Finalmente, se emplea la *codificación Huffman* para comprimir aún más la información restante de los dos vectores anteriores. De forma muy sencilla, el algoritmo de Huffman busca reducir la representación binaria de un conjunto de símbolos, que en nuestro caso son cada uno de los valores disponibles en el anterior paso, asignando códigos binarios más cortos a caracteres que se repiten más frecuentemente. La *tabla de Huffman* necesaria para realizar el proceso inverso se suele guardar en el encabezado del archivo `.jpg`, junto a las tablas $T_{lu,q}$ y $T_{ch,q}$ usadas en la cuantización.

El proceso de descodificación es prácticamente simétrico. En primer lugar, se reconstruyen los bloques cuantizados deshaciendo la codificación entrópica y el zig-zag. Una vez hecho esto, se multiplica cada elemento cuantizado $X^q_{k_1,k_2}$ por T_{k_1,k_2}, obteniendo el valor $\tilde{X}_{k_1,k_2}$, que aproxima el componente espectral original X_{k_1,k_2}. Finalmente, realizamos la transformada inversa (5.7) y la transformación del espacio de color Y'CbCr a sRGB, recuperando así una aproximación de la imagen original. En la Figura 5.8 comparamos algunas imágenes recuperadas para distintas configuraciones de calidad, junto a las tablas de cuantización $T_{q,lu}$ y el tamaño final en kilobytes.

Por supuesto, las suposiciones I. y II. no son ciertas para todas las imágenes. JPEG se ideó originalmente para fotografías digitales realizadas por una cámara, en las que los cambios de color suelen ser suaves y es difícil notar la cuantización (o el submuestreo cromático) al menos que se emplee una calidad muy baja. No obstante, el formato se hizo inmensamente popular en la web, al reducir el tamaño de las imágenes mucho más que otras alternativas, lo cual beneficia al rendimiento de los servidores y la red. Es común encontrar imágenes sintéticas como iconos, gráficos o capturas de pantalla comprimidas con JPEG, aunque los códecs sin pérdidas como PNG sean más adecuados para estas. Utilizar JPEG en imágenes sintéticas hace aparecer *artefactos* como la difuminación de líneas o texto, que pueden ser visibles incluso en calidades altas.

La transformada de cosenos no se limita a la compresión de imágenes, sino que, gracias a su capacidad para concentrar la energía de una señal, es una herramienta clave en el procesamiento de todo tipo de señales multimedia. Por ejemplo, en el campo de la compresión de audio, una variación conocida como Transformada de Cosenos Modificada (MDCT) es la base de codificadores

$q = 80$, tamaño = 83 KB



$q = 30$, tamaño = 40 KB



$q = 5$, tamaño = 14 KB



$T_{lu,80}$

6	4	4	6	10	16	20	24
5	5	6	8	10	23	24	22
6	5	6	10	16	23	28	22
6	7	9	12	20	35	32	25
7	9	15	22	27	44	41	31
10	14	22	26	32	42	45	37
20	26	31	35	41	48	48	40
29	37	38	39	45	40	41	40

$T_{lu,30}$

27	18	17	27	40	66	85	101
20	20	23	32	43	96	100	91
23	22	27	40	66	95	115	93
23	28	37	48	85	144	133	103
30	37	61	93	113	181	171	128
40	58	91	106	134	173	188	153
81	106	129	144	171	201	199	168
120	153	158	163	186	166	171	164

$T_{lu,5}$

160	110	100	160	240	255	255	255
120	120	140	190	255	255	255	255
140	130	160	240	255	255	255	255
140	170	220	255	255	255	255	255
180	220	255	255	255	255	255	255
240	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255

$T_{ch,80}$

6	7	10	19	40	40	40	40
7	8	10	26	40	40	40	40
10	10	22	40	40	40	40	40
19	26	40	40	40	40	40	40
40	40	40	40	40	40	40	40
40	40	40	40	40	40	40	40
40	40	40	40	40	40	40	40
40	40	40	40	40	40	40	40

$T_{ch,30}$

27	30	40	78	164	164	164	164
30	35	43	110	164	164	164	164
40	43	93	164	164	164	164	164
78	110	164	164	164	164	164	164
164	164	164	164	164	164	164	164
164	164	164	164	164	164	164	164
164	164	164	164	164	164	164	164
164	164	164	164	164	164	164	164

$T_{ch,5}$

160	180	240	255	255	255	255	255
180	210	255	255	255	255	255	255
240	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255

Figura 5.8: Comparación de imágenes comprimidas por JPEG para distintas calidades.

tan conocidos como MP3, para música y audio general, u Opus, optimizado para la voz humana. Por otro lado, estándares de compresión de vídeo como H.264/AVC se basan en variantes enteras de la transformada de cosenos (IntDCT). Detrás de todos los detalles técnicos y adaptaciones necesarias para cada algoritmo y formato, se encuentran las ideas fundamentales detrás de la Transformada Discreta de Fourier, impulsada por los algoritmos FFT que hemos descrito en este trabajo.

Anexo I

Código fuente

Todo el código fuente utilizado para la obtención de las figuras y mediciones del trabajo está disponible en el repositorio de GitHub `https://github.com/derivada/bachelor_thesis_fft`. En este Anexo, resumimos el código utilizado para las demostraciones del Capítulo 5, disponibles en la carpetas *audio* y *jpeg* del repositorio. No daremos la implementación de los algoritmos FFT vistos en el Capítulo 4, al ya haberse aportado pseudocódigo detallado para las variantes principales del algoritmo de Cooley-Tukey y disponer de implementaciones optimizadas en la librería SciPy/NumPy ([23]).

I.1. Implementación del filtro digital de audio

Para la construcción de los filtros digitales de audio, es necesario realizar convoluciones acíclicas mediante de forma eficiente mediante algoritmos FFT. En el Código I.1 describimos una implementación con dos "modos" de ejecución posibles. El primero, ejecutado cuando *mode == 'full'* es el visto a lo largo del trabajo, en el que el resultado de la convolución acíclica de los vectores $a \in \mathbb{R}^{N_1}$ y $b \in \mathbb{R}^{N_2}$ es de tamaño $N_1 + N_2 - 1$. El segundo, ejecutado cuando *mode == 'same'*, tan solo mantiene los índices centrales con respecto a a, devolviendo un vector de tamaño N_1. Este último es útil cuando queremos filtrar frecuencias sin extender la señal y lo usaremos al convolucionar el audio original con el filtro final. El modo completo se usará al construir el filtro de harmónicos de Re3 mediante la convolución iterativa de filtros de paso de banda.

Código I.1: Implementación de convoluciones acíclicas rápidas con la FFT

```
1 import numpy as np
2 def fft_convolve(a, b, mode):
3     # FFT con relleno con zeros
4     n = len(a) + len(b) - 1
5     A = np.fft.fft(a, n)
```

```
    B = np.fft.fft(b, n)
    # Producto elemento a elemento
    C = A * B
    # FFT inversa para obtener el array de salida
    # Cogemos la parte real, la imaginaria deberia ser 0,
    # pero la quitamos explicitamente por si hay errores numericos
    c = np.fft.ifft(C).real
    # Modo full: devolver todo el vector de N1+N2-1 elementos
    # Modo same: devolver solo los indices centrales (N1 elementos)
    if mode == 'full':
        return c
    elif mode == 'same':
        start = (len(b) - 1) // 2
        end = start + len(a)
        return c[start:end]
    else:
        raise ValueError("mode must be 'full' or 'same'")
```

En el siguiente Código I.2, detallamos la implementación del filtro digital de audio para frecuencias harmónicas de la nota Re3 tocada por la guitarra.

Código I.2: Construcción del filtro de harmónicos

```
import numpy as np
# Frecuencias fundamentales de los 2 tonos
D3_FREQ = 146.83
E6_FREQ = 1318.51

# Harmonicos
harmonics_d3 = [D3_FREQ * i for i in range(1, 9)]
harmonics_e6 = [E6_FREQ * i for i in range(1, 7)]

# Correcion de los harmonicos del piano (no esta afinado perfectamente)
for i in range(2, 6):
    harmonics_e6[i] += 95*(i-1)
harmonics_e6[3] += 10
harmonics_e6[4] += 110

# Radio de las bandas, longitud del kernel e indice S para inversion espectral
BAND_RADIUS = 20
kernel_len = 5001
S = kernel_len // 2
# Hace falta conocer la frecuencia de muestreo fs para construir el filtro.
# Realmente, primero se lee el archivo de audio para hallarlo,
# pero aqui colocamos eso en el siguiente bloque de codigo, por claridad
h_harmonics = 0
delta = np.zeros(kernel_len) # Funcion impulso para inversion espectral
```

```
delta[S] = 1
# Iteramos por cada harmonico de Re3
for i, fft_freqs in enumerate(harmonics_d3):
    # Definimos las frecuencias alta y baja del filtro de pase de banda
    fc_low = fft_freqs - BAND_RADIUS
    fc_high = fft_freqs + BAND_RADIUS
    # Construimos los filtros de paso bajo para ambas frecuencias
    t = np.arange(-S, S+1)
    h_low = 2 * fc_low / fs * np.sinc(2 * fc_low * t / fs)
    h_low *= np.blackman(kernel_len)
    h_high = 2 * fc_high / fs * np.sinc(2 * fc_high * t / fs)
    h_high *= np.blackman(kernel_len)
    # Construimos el filtro de paso de banda mediante inversion espectral
    h_bandreject = h_low + (delta - h_high)
    # Si no estamos en la primera frecuencia, realizamos una convolucion
    # con el filtro ya obtenido para obtener un filtro de varios pases de banda
    if i == 0:
        h_harmonics = h_bandreject
    else:
        h_harmonics = fft_convolve(h_harmonics, h_bandreject, mode='full')
```

Una vez hecho esto, en el Código I.3 utilizamos el paquete *SoundFile* para leer el archivo de audio original, convolucionarlo con el filtro para eliminar las frecuencias no deseadas y escribirlo de vuelta al disco.

Código I.3: Filtrado de audio

```
import numpy as np
import soundfile as sf
# Leer archivo de audio en .wav
FILENAME = "d3guitar_e6piano.wav"
# Extraer numero de muestreas y frecuencia de muestreo
samples, fs = sf.read(FILENAME)

# Convertir a mono si es estereo
if samples.ndim > 1:
    samples = samples.mean(axis=1)  # Promediar canales para convertir a mono

# Asegurar tipo float32
samples = samples.astype(np.float32)

# Frecuencia de muestreo y numero de muestras
print("Frecuencia de muestreo:", fs, "Hz")
print("Numero de muestras:", samples.size)

# Convolucion del filtro, con el modo 'same' para no extender el audio
filtered = fft_convolve(samples, h_harmonics, mode='same')
```

```
21
22 # Normalizacion al rango [-1, 1] para evitar distorsion
23 filtered = filtered / np.max(np.abs(filtered))
24
25 # Exportar el audio procesado
26 sf.write("piano_filtrado.wav", filtered, fs)
```

I.2. Análisis de la compresión JPEG

También aportamos algo de código con el objetivo de aclarar las explicaciones y las gráficas mostradas en la Sección 5.2. Para el manejo de imágenes, hemos usado los módulos *cv2* de OpenCV (*Open Computer Vision*) y *PIL* (*Python Imaging Library*). En primer lugar, leemos la imagen y la separamos en los canales Y', Cr y Cb en el Código I.4.

Código I.4: lectura de la imagen y separación en canales

```
1  import cv2
2  # Cargar imagen y convertir a RGB
3  img_bgr = cv2.imread(FILE_PATH)
4  img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
5
6  # Separacion componentes RGB
7  r, g, b = cv2.split(img_rgb)
8
9  # Conversion a YCrCb
10 img_ycc = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2YCrCb)
11 y, cr, cb = cv2.split(img_ycc)
```

Una vez hecho esto, definimos en el Código I.5 algunas funciones para extraer un bloque, calcular su transformada DCT de tamaño 8×8 utilizada en JPEG, cuantizarlo y obtener su orden zig-zag.

Código I.5: Funciones auxiliares para el análisis del proceso de compresión JPEG

```
1  import numpy as np
2  from scipy.fftpack import dct, idct
3  # Funcion auxiliar para extraer un bloque 8x8 de una imagen, dado un canal
4  # e indice de bloque
5  def get_block(img_ycc, channel, i, j):
6      return img_ycc[(i*8):((i+1)*8), (j*8):((j+1)*8), channel].astype(np.int16)
7
8  # Funcion para aplicar la DCT 2-dimensional sobre un bloque, nos basamos
9  # en la dct de scipy, que a su vez utiliza un algoritmo tipo FFT para su calculo
10 # https://docs.scipy.org/doc/scipy/reference/generated/scipy.fftpack.dct.html
```

```
def dct2(block):
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

# Transformada inversa de dct2
def idct2(block):
    return idct(idct(block.T, norm='ortho').T, norm='ortho')

# Funcion auxiliar para cuantizar un bloque a partir de una tabla
def quantize_block(dct_block, quant_table):
    return np.round(dct_block / quant_table)

# Funciones auxiliares para desarrollar un bloque mediante orden zigzag y
    obtener sus indices
def zigzag_indices(n=8):
    indices = [[] for _ in range(2*n - 1)]
    for i in range(n):
        for j in range(n):
            indices[i+j].append((j,i) if (i+j)%2 == 0 else (i,j))
    return [item for sub in indices for item in sub]

def zigzag_block(block):
    idx = zigzag_indices(len(block))
    return np.array([block[i, j] for i, j in idx])
```

Para la extracción de las tablas de cuantización dada una calidad y un canal, podemos usar *PIL* para guardar el archivo en formato JPEG en un *buffer* temporal y luego leer los metadatos de este. En el Código I.6 resumimos la implementación. Por supuesto, en una implementación real de JPEG, estas tablas se obtienen directamente siguiendo el estándar ([12]).

Código I.6: Función para la obtención de las tablas de cuantización JPEG

```
import io
import numpy as np
from PIL import Image
# Funcion auxiliar para obtener la tabla de cuantizacion de una imagen
# para un nivel de calidad y un canal especificos
def get_quant_table(quality, channel):
    img_pil = Image.fromarray(img_rgb)
    buf = io.BytesIO()
    img_pil.save(buf, format='JPEG', quality=quality)
    buf.seek(0)
    img_jpeg = Image.open(buf)
    qtables = img_jpeg.quantization
    if channel == 0:
        qtable = qtables[0]
    else:
        qtable = qtables.get(1, qtables[0])
```

```
17
18      quant_table = np.array(qtable).reshape(8, 8)
19      return quant_table
```

Finalmente, vemos qué le pasa a un bloque de la imagen a lo largo del proceso de compresión en el Código I.7. Omitimos el código utilizado en la obtención de las figuras de la sección, que se puede encontrar en el archivo de fuente original.

Código I.7: Proceso JPEG (excepto la codificación entrópica) desglosado en pasos

```
1  # 1. Bloque original del canal Y (indice 0)
2  CHANNEL = 0
3  BLOCK_I = 40
4  BLOCK_J = 30
5  block = get_block(img_ycc, CHANNEL, BLOCK_I, BLOCK_J)
6
7  # 2. Transformada (DCT 8x8) del bloque
8  dct_block = dct2(block)
9
10 # 3. Tabla de cuantizacion
11 QUALITY = 50
12 quant_table = get_quant_table(QUALITY, CHANNEL)
13
14 # 4. Bloque cuantizado
15 quant_block = quantize_block(dct_block, quant_table)
16
17 # 5. Linealizacion del bloque
18 linearized_block = zigzag_block(quant_block)
```

Bibliografía

[1] Bluestein, L. (1970). A linear filtering approach to the computation of discrete Fourier transform. IEEE Transactions on Audio and Electroacoustics, 18(4), 451-455.

[2] Bracewell, R.N. (1966). The Fourier Transform and Its Applications. McGraw-Hill Higher Education.

[3] Brigham, E. O. (1974). The Fast Fourier Transform. Prentice-Hall, Inc.

[4] Cooley, J. W., Lewis, P. A. y Welch, P. D. (1967). Historical notes on the fast Fourier transform. Proceedings of the IEEE, 55(10), 1675-1677.

[5] Cooley, J. W. y Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. Mathematics of computation, 19(90), 297-301.

[6] Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). Introduction to algorithms. MIT press.

[7] Elster, A. C. (1989). Fast bit-reversal algorithms. In International Conference on Acoustics, Speech, and Signal Processing, (pp. 1099-1102). IEEE.

[8] Frigo, M. y Johnson, S. G. (2005). The design and implementation of FFTW3. Proceedings of the IEEE, 93(2), 216-231.

[9] Folland, G. B. (2009). Fourier analysis and its applications (Vol. 4). American Mathematical Soc.

[10] Harris, F. J. (2005). On the use of windows for harmonic analysis with the discrete Fourier transform. Proceedings of the IEEE, 66(1), 51-83.

[11] Heideman, M., Johnson, D., y Burrus, C. (1984). Gauss and the history of the fast Fourier transform. IEEE Assp Magazine, 1(4), 14-21.

[12] ISO/IEC 10918-1 (1994). Information technology — Digital compression and coding of continuous-tone still images — Requirements and guidelines. International Organization for Standardization.

[13] Johnson, S. G. y Frigo, M. (2006). A modified split-radix FFT with fewer arithmetic operations. IEEE Transactions on Signal Processing, 55(1), 111-119.

[14] Lee, B. (1984). A new algorithm to compute the discrete cosine transform. IEEE Transactions on Acoustics, Speech, and Signal Processing, 32(6), 1243-1245.

[15] Morgenstern, J. (1973). Note on a lower bound on the linear complexity of the fast Fourier transform. Journal of the ACM (JACM), 20(2), 305-306.

[16] Plonka, G., Potts, D., Steidl, G. y Tasche, M. (2018). Numerical fourier analysis. Basel: Birkhäuser.

[17] Pouso, R. L. (2019). Series de Fourier y ecuaciones en derivadas parciales: una introducción con Maple y ejercicios resueltos. Universidade de Santiago de Compostela, Servizo de Publicacións.

[18] Rader, C. M. (1968). Discrete Fourier transforms when the number of data samples is prime. Proceedings of the IEEE, 56(6), 1107-1108.

[19] Rao, K. R., Kim, D. N. y Hwang, J. J. (2011). Fast Fourier transform-algorithms and applications. Springer Science y Business Media.

[20] Roma, N. y Sousa, L. (2011). A tutorial overview on the properties of the discrete cosine transform for encoded image and video processing. Signal Processing, 91(11), 2443-2464.

[21] Rudin, W. (1976) Principles of Mathematical Analysis. 3rd Edition, McGraw-Hill, New York, 145.

[22] Smith W. (1997). The scientist and engineer's guide to digital signal processing. California Technical Publishing, USA.

[23] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... y Van Mulbregt, P. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature methods, 17(3), 261-272.

[24] Wallace, G. K. (1992). The JPEG still picture compression standard. IEEE transactions on consumer electronics, 38(1), xviii-xxxiv.

[25] Wheeden, R. L. y Zygmund, A. (1977). Measure and integral (Vol. 26). New York: Dekker.

[26] Yavne, R. (1968, December). An economical method for calculating the discrete Fourier transform. In Proceedings of the December 9-11, 1968, fall joint computer conference, part I (pp. 115-125).